

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ CẦU NÓI AXI-APB VÀ KIỂM TRA
BẰNG MÔI TRƯỜNG UVM**

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

GVHD: **TS. PHẠM VĂN KHOA**

Sinh viên: **BÙI TRẦN ANH KHOA**

MSSV: 21119219

TRẦN LỤC KHÁNH VŨ

MSSV: 21119291

TP. HỒ CHÍ MINH – 6/2025

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG

ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ CẦU NỐI AXI-APB VÀ KIỂM TRA
BẰNG MÔI TRƯỜNG UVM**

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

GVHD: TS. PHẠM VĂN KHOA

Sinh viên: BÙI TRẦN ANH KHOA
MSSV: 21119219

TRẦN LỤC KHÁNH VŨ
MSSV: 21119291

TP. HỒ CHÍ MINH – 6/2025

THÔNG TIN KHÓA LUẬN TỐT NGHIỆP

1. Thông tin sinh viên

Họ và tên sinh viên: Bùi Trần Anh Khoa	MSSV: 21119219
Email: buitranganhkhoea123789456@gmail.com	Điện thoại: 0799657479
Họ và tên sinh viên: Trần Lục Khánh Vũ	MSSV: 21119291
Email: khanhvu1702@gmail.com	Điện thoại: 0857886961

2. Thông tin đề tài

- Tên đề tài: Thiết kế cầu nối AXI-APB và kiểm tra bằng môi trường UVM
- Đơn vị quản lý: Bộ môn Kỹ Thuật máy tính – Viễn Thông, Khoa Điện-Điện Tử, Trường Đại Học Sư Phạm Kỹ Thuật Tp. Hồ Chí Minh.
- Thời gian thực hiện: Từ ngày / / 20... đến ngày / / 20..
- Thời gian bảo vệ trước hội đồng: Ngày / / 20..

3. Lời cam đoan của sinh viên

Nhóm thực hiện cam đoan KLTN là công trình nghiên cứu của tôi, dưới sự hướng dẫn của Tiến Sĩ Phạm Văn Khoa. Kết quả công bố trong KLTN là trung thực và không sao chép từ bất kì công trình nào khác.

Tp. HCM, ngày ... tháng ... năm 20...
SV thực hiện đồ án
(Ký và ghi rõ họ tên)

.....

Giảng viên hướng dẫn xác nhận quyền báo cáo đã được chỉnh sửa theo đề nghị được ghi trong biên bản của Hội đồng đánh giá Khóa luận tốt nghiệp.

.....

Xác nhận của Bộ Môn

Tp. HCM, ngày ... tháng ... năm 20..
Giảng viên hướng dẫn
(Ký, ghi rõ họ tên và học hàm – học vị)

BẢN NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

LỜI CẢM ƠN

Trong quá trình thực hiện đề tài, nhóm chúng tôi đã có cơ hội vận dụng những kiến thức đã học từ các học phần trước, kết hợp với tài liệu tham khảo và trải nghiệm thực tế để hoàn thành đề tài một cách tốt nhất.

Nhóm xin chân thành cảm ơn quý thầy cô Trường Đại học Sư phạm Kỹ thuật TP.HCM đã tận tình giảng dạy và truyền đạt kiến thức trong suốt thời gian học tập tại trường. Đặc biệt, nhóm trân trọng gửi lời cảm ơn đến quý thầy cô Khoa Điện – Điện tử đã hỗ trợ, định hướng và cung cấp nền tảng kiến thức vững chắc, tạo điều kiện thuận lợi để nhóm có thể thực hiện đề tài này.

Nhóm cũng xin cảm ơn Bộ môn Kỹ thuật Máy tính – Viễn thông đã tạo điều kiện và hỗ trợ nhóm trong quá trình thực hiện đồ án tốt nghiệp, giúp chúng tôi có cơ hội vận dụng hiệu quả những kiến thức đã học vào thực tiễn.

Đặc biệt, nhóm xin gửi lời cảm ơn sâu sắc đến thầy TS. Phạm Văn Khoa, giảng viên hướng dẫn, người đã đồng hành, tận tình chỉ bảo và định hướng cho nhóm từ những bước đầu tiên đến khi hoàn thiện đề tài. Sự hướng dẫn tận tâm và những góp ý quý báu của thầy không chỉ giúp nhóm hoàn thành đồ án mà còn là hành trang quý giá cho con đường học tập và làm việc trong tương lai.

Trong quá trình thực hiện đề tài, do còn thiếu kinh nghiệm, nhóm khó tránh khỏi những thiếu sót. Rất mong nhận được sự góp ý và đánh giá từ quý thầy cô để nhóm có thể hoàn thiện hơn trong các công việc sau này.

Xin chân thành cảm ơn.

Người thực hiện đề tài

Bùi Trần Anh Khoa

Trần Lục Khánh Vũ

TÓM TẮT

Trong một thiết kế SoC thì vấn đề giao tiếp giữa các giao thức luôn là một trong những vấn đề rất được quan tâm. Do các nhu cầu truyền tải khác nhau giữa các khối trong SoC, một số khối sẽ yêu cầu giao thức có tốc độ cao trong khi một số khối chỉ yêu cầu giao thức có tốc độ thấp để xử lý với ngoại vi.

Đề tài này sẽ hướng đến việc nghiên cứu cầu nối AXI-APB (AXI-to-APB Bridge), đây là một thành phần thiết yếu trong các hệ thống trên chip (SoC) hiện đại, đóng vai trò trung gian cho phép các khối chủ (master) sử dụng giao thức AXI hiệu năng cao giao tiếp với các khối tớ (slave) sử dụng giao thức APB đơn giản, tiêu thụ ít năng lượng hơn. Giao thức AXI (Advanced eXtensible Interface) được thiết kế cho các kết nối băng thông rộng, tốc độ cao, hỗ trợ nhiều giao dịch đồng thời và truyền dữ liệu theo cụm (burst), thường được sử dụng cho các thành phần như bộ vi xử lý, bộ điều khiển DMA, và bộ nhớ chính. Ngược lại, giao thức APB (Advanced Peripheral Bus) là một giao diện đơn giản hơn, không hỗ trợ pipeline, chủ yếu dùng để truy cập các thanh ghi cấu hình và các ngoại vi tốc độ thấp như UART, SPI, I2C, GPIOs, PWM.

Cầu nối sẽ phục vụ cho nhu cầu truyền tải dữ liệu từ nơi có tần số hoạt động cao (AXI) đến nơi có tần số hoạt động thấp hơn (APB).

Việc sử dụng môi trường UVM để kiểm tra và đánh giá thông qua các trường hợp mô phỏng sẽ giúp đảm bảo các tính năng của cầu nối hoạt động đúng với các yêu cầu đặt ra.

MỤC LỤC

DANH MỤC HÌNH.....	VIII
DANH MỤC BẢNG.....	XI
CÁC TỪ VIẾT TẮT	XII
CHƯƠNG 1 TỔNG QUAN ĐỀ TÀI	1
1.1 GIỚI THIỆU.....	1
1.2 MỤC TIÊU ĐỀ TÀI	2
1.3 PHẠM VI GIỚI HẠN ĐỀ TÀI.....	2
1.4 PHƯƠNG PHÁP NGHIÊN CỨU	2
1.5 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU	3
1.6 BỐ CỤC QUYỀN BÁO CÁO.....	3
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT	4
2.1 TỔNG QUAN VỀ SYSTEM ON CHIP VÀ AMBA	4
2.1.1 Giới thiệu về System on Chip.....	4
2.1.2 Giới thiệu về AMBA	4
2.2 TỔNG QUAN VỀ BUS APB.....	6
2.2.1 Giới thiệu về giao thức APB	6
2.2.2 Mô tả tín hiệu của chuẩn giao tiếp APB4	6
2.2.3 Trạng thái hoạt động của giao thức truyền thông APB4.....	7
2.2.4 Quá trình ghi dữ liệu của giao thức truyền thông APB4.....	9
2.2.5 Quá trình đọc dữ liệu của giao thức truyền thông APB4	10
2.2.6 Mô tả chi tiết một số tín hiệu của giao thức APB4	12
2.3 TỔNG QUAN GIAO THỨC AXI.....	14
2.3.1 Giới thiệu về giao thức AXI	14
2.3.2 Cấu trúc phân kênh của giao thức AXI.....	15
2.3.3 Các cơ chế Valid – Ready (Hand-Shake).....	16
2.3.4 Quá trình đọc và ghi dữ liệu	18
2.3.5 Cơ chế chuyển đổi burst trong giao thức AXI.....	20
2.4 TỔNG QUAN VỀ MÔI TRƯỜNG KIỂM TRA UVM.....	25

2.5	CẤU TRÚC MÔI TRƯỜNG UVM	26
2.5.1	<i>UVM Testbench top</i>	27
2.5.2	<i>UVM Test</i>	28
2.5.3	<i>UVM Environment</i>	28
2.5.4	<i>UVM Agent</i>	28
2.5.5	<i>UVM Driver</i>	28
2.5.6	<i>UVM Monitor</i>	29
2.5.7	<i>UVM Scoreboard</i>	29
2.6	CƠ CHẾ PHA TRONG UVM.....	29
CHƯƠNG 3	THIẾT KẾ HỆ THỐNG.....	31
3.1	YÊU CẦU CỦA HỆ THỐNG	31
3.2	THIẾT KẾ HỆ THỐNG	31
3.2.1	<i>Sơ đồ khối tổng quát</i>	31
3.2.2	<i>Thiết kế khối FIFO</i>	36
3.2.3	<i>Thiết kế khối B_CH</i>	44
3.2.4	<i>Thiết kế khối Write/Read Decoder</i>	45
3.2.5	<i>Thiết kế khối APB Master</i>	47
3.2.6	<i>Thiết kế khối thanh ghi APB (APB Register)</i>	59
3.3	THIẾT KẾ MÔI TRƯỜNG KIỂM TRA UVM.....	60
3.3.1	<i>Xây dựng kế hoạch kiểm tra</i>	60
3.3.2	<i>Xây dựng các khối trong môi trường</i>	64
CHƯƠNG 4	KẾT QUẢ	70
4.1	CÁC TRƯỜNG HỢP KIỂM THỬ CHỨC NĂNG CỦA QUÁ TRÌNH TRUYỀN.....	70
4.2	KẾT QUẢ MÔ PHỎNG.....	79
4.2.1	<i>Testcase 1: AXI – APB ghi đọc dữ liệu 32-bit vào slave 8-bit chế độ INCR với BURST_SIZE = 8, BURST_LENGTH = 32</i>	79
4.2.2	<i>Testcase2: AXI – APB ghi đọc dữ liệu 32-bit vào slave 8-bit chế độ WRAP với BURST_SIZE = 8, BURST_LENGTH = 32</i>	87

4.2.3	<i>Testcase3: AXI – APB ghi đọc dữ liệu 32-bit vào slave 16-bit chế độ WRAP với BURST_SIZE = 8, BURST_LENGTH = 32.....</i>	95
4.2.4	<i>Testcase4: AXI – APB ghi đọc dữ liệu 32-bit vào slave 32-bit chế độ WRAP với BURST_SIZE = 8, BURST_LENGTH = 32.....</i>	103
CHƯƠNG 5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	111
5.1	KẾT LUẬN	111
5.2	HƯỚNG PHÁT TRIỂN	111
PHỤ LỤC		112
TÀI LIỆU THAM KHẢO.....		113

DANH MỤC HÌNH

Hình 2.1	Sơ đồ khối của một hệ thống triển khai theo AMBA	5
Hình 2.2	Máy trạng thái hữu hạn của giao thức APB [2]	8
Hình 2.3	Quá trình ghi không có trạng thái chờ	9
Hình 2.4	Quá trình ghi có trạng thái chờ	10
Hình 2.5	Quá trình đọc không có trạng thái chờ.....	11
Hình 2.6	Quá trình đọc có trạng thái chờ.....	11
Hình 2.7	Mô tả về làn dữ liệu hợp lệ nếu độ dài dữ liệu là 32 bits.....	12
Hình 2.8	Quá trình ghi xảy ra lỗi	13
Hình 2.9	Quá trình đọc xảy ra lỗi	13
Hình 2.10	Giao diện giao tiếp và kết nối bus của giao thức AXI.....	14
Hình 2.11	Các kênh của một kênh truyền AXI [3]	15
Hình 2.12	Minh họa tín hiệu bắt tay ở kênh AXI	17
Hình 2.13	Minh họa tín hiệu bắt tay với tín hiệu VALID có trước	17
Hình 2.14	Minh họa tín hiệu bắt tay với tín hiệu READY có trước.....	17
Hình 2.15	Mối liên hệ giữa các tín hiệu bắt tay ở giao dịch ghi.....	18
Hình 2.16	Mối liên hệ giữa các tín hiệu bắt tay ở giao dịch đọc	18
Hình 2.17	Giao dịch đọc trong giao thức AXI.....	19
Hình 2.18	Giao dịch ghi trong giao thức AXI	19
Hình 2.19	Nhóm kênh phục vụ cho giao dịch đọc.....	21
Hình 2.20	Nhóm kênh phục vụ cho giao dịch ghi	22
Hình 2.21	Kiến trúc môi trường UVM	26
Hình 2.22	Sơ đồ phân cấp trong môi trường UVM	27
Hình 2.23	Danh sách và trình tự các pha trong UVM	29
Hình 3.1	Sơ đồ khối tổng quát của cầu nối AXI-APB	31
Hình 3.2	Các tín hiệu đầu vào và đầu ra của cầu nối.....	33
Hình 3.3	Mô hình FIFO bất đồng bộ	37
Hình 3.4	Hoạt động của một FIFO độ rộng mỗi ô là 8 bit	37
Hình 3.5	Mô tả hoạt động của FIFO_FULL và FIFO_EMPTY	38
Hình 3.6	Sơ đồ kết nối của khối ASFIFO_AR	39

Hình 3.7	Sơ đồ kết nối của khối ASFIFO_AW	40
Hình 3.8	Sơ đồ kết nối của khối ASFIFO_RD	42
Hình 3.9	Sơ đồ kết nối của khối ASFIFO_WD	43
Hình 3.10	Sơ đồ kết nối của khối B_CH	44
Hình 3.11	Sơ đồ khối Write/Read Decoder.	45
Hình 3.12	Mô tả chi mạch của khối write/read decoder	46
Hình 3.13	Sơ đồ kết nối khối APB_Master	47
Hình 3.14	Sơ đồ máy trạng thái FSM của khối APB Master	50
Hình 3.15	Địa chỉ tăng của ngoại vi 8-bit.....	52
Hình 3.16	Địa chỉ tăng của ngoại vi 16-bit.....	53
Hình 3.17	Địa chỉ tăng của ngoại vi 32-bit.....	53
Hình 3.18	Minh họa địa chỉ tăng ở ngoại vi 16-bit theo tín hiệu điều khiển	53
Hình 3.19	Trình tự thay đổi địa chỉ ở kiểu truyền WRAP.....	54
Hình 3.20	Mô hình và tín hiệu của khối APB Register	59
Hình 3.21	Đặc điểm độ dài dữ liệu của từng ngoại vi APB	60
Hình 3.22	Sơ đồ môi trường UVM.....	64
Hình 3.23	Quá trình gửi tín hiệu từ Driver với hoạt động ghi.....	65
Hình 3.24	Quá trình gửi tín hiệu từ Driver với hoạt động đọc	66
Hình 3.25	Sơ đồ khối hoạt động của khối Monitor	67
Hình 3.26	Ví dụ về cách hiển thị báo cáo của scoreboard.....	68
Hình 3.27:	Mô hình APB Slave	69
Hình 4.1	Dạng sóng tín hiệu chuẩn bị quá trình ghi và đọc	79
Hình 4.2	Dạng sóng quá trình ghi ở ngoại vi APB 8-bit	81
Hình 4.3	Dạng sóng quá trình đọc ở ngoại vi APB 8-bit.....	82
Hình 4.4	Dạng sóng thông tin đọc phóng to ở phía AXI tại thời điểm T8...	84
Hình 4.5	Bảng kết quả quá trình ghi từ mô hình UVM.....	85
Hình 4.6	Bảng kết quả quá trình đọc của mô hình UVM	86
Hình 4.7	Dạng sóng tín hiệu chuẩn bị quá trình đọc và ghi	87
Hình 4.8	Dạng sóng quá trình ghi ở APB 8-bit	88
Hình 4.9	Dạng sóng quá trình đọc ở ngoại vi APB 8-bit.....	90

Hình 4.10	Dạng sóng thông tin đọc phóng to ở phía AXI thời điểm T8.....	91
Hình 4.11	Bảng kết quả quá trình ghi từ mô hình UVM.....	92
Hình 4.12	Bảng kết quả cho quá trình đọc từ mô hình UVM.....	93
Hình 4.13	Dạng sóng tín hiệu chuẩn bị quá trình ghi/đọc ở ngoại vi 16-bit	95
Hình 4.14	Dạng sóng quá trình ghi ở ngoại vi APB 16-bit	96
Hình 4.15	Dạng sóng quá trình đọc của ngoại vi APB 16-bit	98
Hình 4.16	Dạng sóng đọc được phóng to ở phía AXI tại thời điểm T8	99
Hình 4.17	Bảng kết quả quá trình ghi từ mô hình UVM.....	100
Hình 4.18	Bảng kết quả quá trình đọc từ mô hình UVM	101
Hình 4.19	Dạng sóng tín hiệu chuẩn bị quá trình ghi và đọc	103
Hình 4.20	Dạng sóng quá trình ghi ở ngoại vi APB 32-bit	104
Hình 4.21	Dạng sóng quá trình đọc ở ngoại vi APB 32-bit.....	106
Hình 4.22	Dạng sóng dữ liệu đọc phóng to ở phía AXI tại thời điểm T8 ..	107
Hình 4.23	Bảng kết quả quá trình ghi từ mô hình UVM.....	108
Hình 4.24	Bảng kết quả quá trình đọc từ mô hình UVM	109

DANH MỤC BẢNG

Bảng 2.1 Các tín hiệu của bus kết nối APB4	6
Bảng 2.2 Kích thước dữ liệu tương ứng với tín hiệu AxSIZE	23
Bảng 2.3 Các loại burst tương ứng với giá trị của tín hiệu AxBURST.....	23
Bảng 2.4 Các loại phản hồi tương ứng với giá trị của RRESP/BRESP	25
Bảng 3.1 Các chân tín hiệu của cầu AXI – APB.....	33
Bảng 3.2: Bảng mô tả chức năng của các tín hiệu khối APB_MASTER	47
Bảng 3.3 Bảng thông số thiết lập cho burst truyền.....	54
Bảng 3.4 Bảng thể hiện nhóm bit tính toán tương ứng với AxLEN	55
Bảng 3.5 Trình tự thay đổi địa chỉ theo cách tính bằng nhóm bit	55
Bảng 3.6 Bảng thay đổi các nhóm bit của ngoại vi 8-bit	56
Bảng 3.7 Bảng mô tả nhóm bit thay đổi với ngoại vi 16-bit	57
Bảng 3.8 Bảng mô tả nhóm bit thay đổi với ngoại vi 32-bit	58
Bảng 3.9 Địa chỉ bắt đầu và kết thúc của ngoại vi APB và thanh ghi APB..	59
Bảng 3.10 Bảng thống kê hạng mục kiểm thử	61
Bảng 4.1 Bảng biểu diễn các hạng mục kiểm tra sẽ được lựa chọn.....	70
Bảng 4.2 Bảng mô tả chi tiết hạng mục kiểm tra	71

CÁC TỪ VIẾT TẮT

AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
AR	Address Read
AW	Address Write
AXI	Advanced eXtensible Interface
FIFO	First In First Out
SoC	System on Chip
UVM	Universal Verification Methodology
DUT	Design Under Test

CHƯƠNG 1

TỔNG QUAN ĐỀ TÀI

1.1 GIỚI THIỆU

Sự phát triển nhanh chóng của các hệ thống vi mạch tích hợp (SoC – System on Chip) đòi hỏi các phương pháp thiết kế và xác minh phần cứng ngày càng phức tạp và hiệu quả hơn. Trong các hệ thống này, việc kết nối giữa các khối chức năng sử dụng các chuẩn giao tiếp khác nhau là vô cùng quan trọng. Một ví dụ điển hình là sự kết hợp giữa hai chuẩn giao tiếp phổ biến trong kiến trúc AMBA của ARM: AXI (Advanced eXtensible Interface) và APB (Advanced Peripheral Bus).

Giao thức AXI được thiết kế cho các khối có yêu cầu cao về băng thông và hiệu năng, như bộ xử lý trung tâm hoặc bộ nhớ, trong khi APB phù hợp với các thiết bị ngoại vi đơn giản, như bộ định thời (timer), UART hoặc GPIO, do thiết kế đơn giản và tiết kiệm tài nguyên. Để truyền dữ liệu giữa hai không gian này một cách hiệu quả, một cầu nối AXI-APB là cần thiết, đóng vai trò chuyển đổi tín hiệu và giao thức giữa hai bus với kiến trúc khác biệt.

Nhằm đáp ứng nhu cầu ngày càng đa dạng trong thiết kế SoC, đặc biệt là SoC trong các thiết bị nhúng, việc phát triển một cầu nối AXI-APB có khả năng tùy biến theo từng loại thiết bị ngoại vi là rất cần thiết, có thể kể đến các ngoại vi có độ dài như sau: độ dài 8-bit như các bộ chuyển đổi cấp thấp hơn như UART, I2C, với 16-bit sẽ là bộ điều khiển SPI chế độ 16-bit và bộ điều chế độ rộng xung (PWM) 16-bit, còn 32-bit sẽ là những bộ kiểm tra lỗi CRC, và những thanh ghi cấu hình cho bộ điều khiển DMA (DMAC). Cách tiếp cận này giúp hệ thống dễ dàng mở rộng hoặc tích hợp nhiều loại ngoại vi khác nhau, tăng sự linh động ở các mô hình nhúng. Trong đề án này, nhóm chúng tôi đề xuất một cấu trúc cầu nối

AXI-APB có khả năng kết nối ngoại vi APB với độ rộng dữ liệu khác nhau (8-bit, 16-bit, 32-bit).

Trong những năm gần đây, khi các thiết kế ngày càng phức tạp thì hình thức kiểm tra truyền thống (direct test) trở nên kém hiệu quả hơn. Vì vậy UVM (Universal Verification Methodology) là một chuẩn kiểm tra theo hướng đối tượng dựa trên System Verilog cho phép kỹ sư có thể xây dựng một môi trường kiểm tra có tính tổng quát, tái sử dụng và tự động hóa cao. Việc áp dụng UVM trong kiểm tra phần cứng thiết kế sẽ nâng cao khả năng phát hiện lỗi thiết kế từ sớm từ đó giảm thời gian thiết kế phần cứng.

Vì vậy trong đề tài này, nhóm thực hiện tập trung vào thiết kế cầu nối AXI-APB và xây dựng môi trường kiểm tra bằng UVM cho cầu nối AXI-APB.

1.2 MỤC TIÊU ĐỀ TÀI

Mục tiêu của đề tài tập trung vào thiết kế cầu nối AXI-APB và xây dựng môi trường kiểm tra UVM bao gồm 2 nhiệm vụ chính sau:

- Thiết kế cầu nối AXI-APB với chức năng hoàn chỉnh.
- Kiểm tra và đánh giá chức năng của cầu nối thông qua mô phỏng bằng môi trường kiểm tra UVM.

1.3 PHẠM VI GIỚI HẠN ĐỀ TÀI

Đề tài được thực hiện bằng cách xây dựng cầu nối và môi trường UVM bằng ngôn ngữ System Verilog. Các file trong đề tài được tổng hợp trên phần mềm Vivado 2024.2.

Cầu nối được mô phỏng kiểm tra chức năng bằng môi trường UVM 1.2 thông qua phần mềm Vivado 2024.2.

1.4 PHƯƠNG PHÁP NGHIÊN CỨU

Đề tài sử dụng các phương pháp nghiên cứu như sau:

- Phương pháp phân tích và tổng hợp.
- Phương pháp phân loại và hệ thống.
- Phương pháp mô phỏng.

1.5 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

Đối tượng nghiên cứu của đề tài này là cầu nối AXI-APB và môi trường kiểm tra UVM.

Phạm vi nghiên cứu về các giao thức AXI, giao thức APB, các khối trong cầu nối AXI-APB và các thành phần trong môi trường kiểm tra UVM.

1.6 BỐ CỤC QUYỀN BẢO CÁO

Bố cục sẽ được nhóm chia thành 5 chương để cho độc giả có thể hình dung một cách cụ thể những gì nhóm thực hiện sẽ làm trong từng chương.

Chương 1. GIỚI THIỆU: Trong chương này, nhóm thực hiện chủ yếu giới thiệu một cách khái quát về đề tài và làm rõ, phân tích các vấn đề mà đề tài này gặp phải, đưa ra các giới hạn đề tài riêng và đặt ra các mục tiêu cụ thể để có thể hoàn thiện đề tài một cách hiệu quả.

Chương 2. CƠ SỞ LÝ THUYẾT: Trong chương 2, nhóm thực hiện làm rõ các khái niệm cơ bản của đề tài, những lý thuyết về giao thức AXI, bus APB và môi trường kiểm tra UVM.

Chương 3. THIẾT KẾ HỆ THỐNG: Trong chương 3, nhóm tập trung phân tích, vẽ lại hệ thống và phân tích các khối cần thiết của cầu nối AXI-APB cùng môi trường kiểm tra UVM

Chương 4. KẾT QUẢ: Trong chương này, nhóm thực hiện sẽ trình bày chi tiết các trường hợp kiểm tra được lựa chọn, chạy mô phỏng và phân tích từng trường hợp nhằm giúp người đọc hiểu về quá trình hoạt động của thiết kế.

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN: Trong chương này, nhóm thực hiện sẽ tổng kết lại toàn bộ quá trình nghiên cứu và phát triển hệ thống. Đồng thời nhóm cũng sẽ nhận xét về các yêu cầu đạt được, đưa ra các hạn chế và phương hướng phát triển của đề tài.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1 TỔNG QUAN VỀ SYSTEM ON CHIP VÀ AMBA

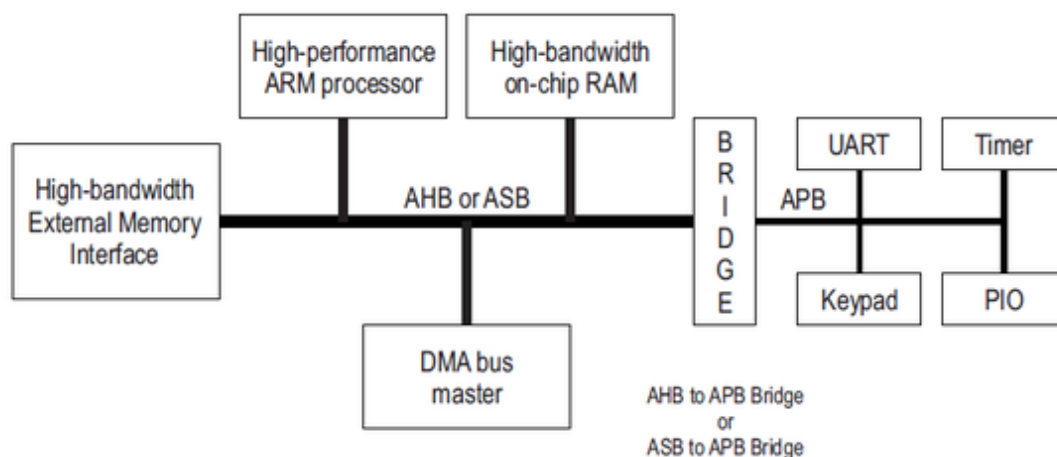
2.1.1 Giới thiệu về *System on Chip*

Hệ thống trên chip (System on Chip hay SoC) là một loại mạch tích hợp, trong đó các hệ thống SoC bao gồm các thành phần như bộ xử lý, bộ nhớ, các cổng ngoại vi được tích hợp sẵn trên một con chip duy nhất. Các thành phần trên được kết nối với nhau thông qua một hệ thống bus kết nối. Các thành phần trong hệ thống SoC thông thường sẽ có tần số hoạt động khác nhau nên các bus kết nối giữa các thành phần sẽ khác nhau tùy theo thành phần. Những đặc trưng đó đã tạo nên kiến trúc bus.

Kiến trúc Bus là một tiêu chuẩn trong việc xây dựng một SoC, trong đó các thành phần bus được quy định rõ ràng về vị trí kết nối. Mỗi công ty thiết kế SoC sẽ có các kiến trúc riêng trong thiết kế của họ. Một số kiến trúc hiện nay là AMBA của ARM, CoreConnect của IBM, Wishbone.

2.1.2 Giới thiệu về *AMBA*

AMBA hay Advanced Microcontroller Bus Architecture là một hệ thống đường truyền dữ liệu nội bộ trên các thiết kế hệ thống trên chip (SoC) được phát triển bởi công ty ARM. AMBA là một tiêu chuẩn mở để kết nối và quản lý các khối chức năng của thiết kế. AMBA có nhiều chuẩn kết nối khác nhau cũng như mục đích sử dụng. Hình 2.1 thể hiện sơ đồ của hệ thống triển khai theo AMBA.



Hình 2.1 Sơ đồ khối của một hệ thống triển khai theo AMBA

Theo hình 2.1, các chuẩn giao tiếp AHB và ASB được sử dụng trong các khối chức năng yêu cầu giao tiếp ở tần số, băng thông cao, trong đó chuẩn giao tiếp APB được sử dụng trong các giao tiếp với ngoại vi có tốc độ xử lý thấp hơn như UART, SPI, I/O. Đặc trưng của kiến trúc AMBA là tại điểm giao giữa 2 chuẩn kết nối có tốc độ xử lý khác nhau thì có một cầu chuyển đổi giao tiếp (Bridge), cầu nối này đóng vai trò chuyển đổi tín hiệu giữa hai chuẩn, từ đó đồng bộ với nhau trong hệ thống.

Qua từng năm, ARM đã phát triển thêm nhiều phiên bản AMBA khác để phục vụ cho yêu cầu ngày càng cao về thiết kế chip, bao gồm có AMBA, AMBA 1, AMBA 2, AMBA 3, AMBA 4 và AMBA 5.

Với AMBA 4, đây là phiên bản được sử dụng rộng rãi nhất trong các hệ thống SoC. AMBA 4 được công bố rộng rãi vào năm 2010 và vẫn được ưa chuộng cho đến ngày nay vì sự đa dạng, linh hoạt, hiệu suất cao và khả năng tương thích tốt với thiết kế công nghiệp hiện đại. AMBA 4 các các chuẩn bus như sau [1]:

- Advanced eXtensible Interface 4 (AXI4)
- Advanced eXtensible Interface 4 Lite (AXI4-Lite)
- Advanced eXtensible Interface 4 Stream (AXI4-Stream)
- Advanced Peripheral Bus (APB4)
- AXI Coherency Extensions (ACE)
- Advanced Trace Bus (ATB)

2.2 TỔNG QUAN VỀ BUS APB

2.2.1 Giới thiệu về giao thức APB

APB (Advanced Peripheral Bus) theo tiếng Việt là đường truyền ngoại vi nâng cao, là một phần của kiến trúc kết nối nâng cao của vi điều khiển (AMBA). Giao thức truyền thông APB được điều khiển do máy trạng thái hữu hạn (FSM) nên việc thiết kế hệ thống và cài đặt trong phần cứng diễn ra đơn giản.

APB hoạt động dựa theo cơ chế MASTER – SLAVE, trong đó SLAVE thường sẽ là cầu nối đến các tín hiệu bên trong với thiết bị ngoại vi. APB không có các kênh dẫn riêng lẻ, chỉ có một kênh dẫn chung để giao tiếp, thích hợp để kết nối với các ngoại vi có tốc độ xử lý và băng thông tương đối thấp hơn so với bộ xử lý.

Qua các năm, APB đã được phát triển thêm nhiều phiên bản khác nhau như APB, APB2, APB3, APB4. Hiện nay APB4 được sử dụng rộng rãi trong các SoC vì độ linh hoạt của kiến trúc này, và trở thành tiêu chuẩn của các thiết kế trên chip hiện đại.

2.2.2 Mô tả tín hiệu của chuẩn giao tiếp APB4

Các tín hiệu thông dụng của bus kết nối APB4 được mô tả theo bảng 2.1:

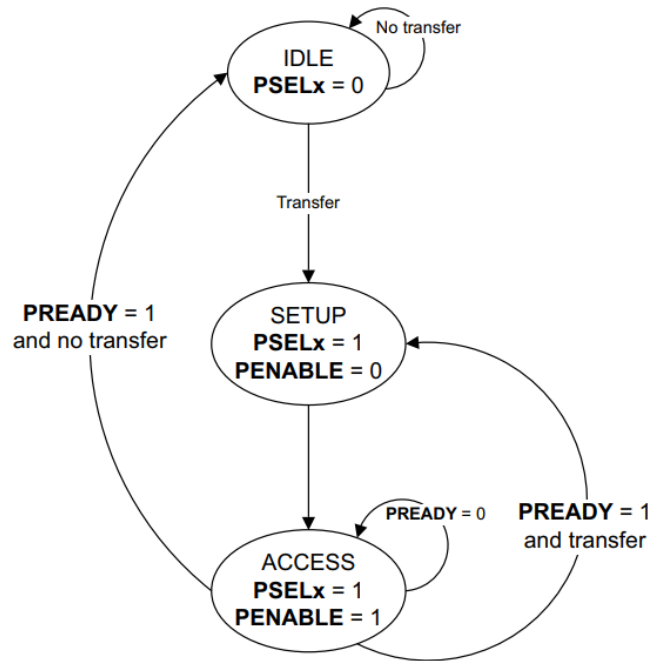
Bảng 2.1 Các tín hiệu của bus kết nối APB4

Tín hiệu	Nguồn của tín hiệu	Mô tả
PCLK	Bộ tạo xung của hệ thống	Là xung vuông của hệ thống, trong đó cạnh lên của xung vuông được sử dụng xuyên suốt quá trình giao tiếp APB
PRESETn	Bus hệ thống	Tín hiệu khởi động lại được tích cực mức thấp, khôi phục APB trở lại trạng thái mặc định
PADDR	APB master	Tín hiệu địa chỉ, nhằm xác định địa chỉ của thanh ghi của khối APB trong quá trình đọc và ghi dữ liệu
PSELx	APB master	Tín hiệu chọn. Tín hiệu chọn cho biết rằng phía master APB quyết định chọn slave APB tương ứng nào

PENABLE	APB master	Tín hiệu cho phép. Tín hiệu này thông báo đến phía APB slave rằng APB master sẵn sàng ghi/nhận dữ liệu. Tín hiệu này đánh dấu chu kỳ thứ 2 của một lần truyền ở APB
PWRITE	APB master	Tín hiệu ghi và đọc. Nếu PWRITE mức thấp thì là đọc dữ liệu, PWRITE cao là ghi dữ liệu
PWDATA	APB master	Tín hiệu dữ liệu ghi. Dữ liệu ghi được điều khiển bởi APB master trong quá trình ghi của APB. Độ rộng dữ liệu có thể là 8 bits, 16 bits hoặc 32 bits.
PSTRB	APB master	Tín hiệu mặt nạ dữ liệu. Tín hiệu này quyết định xem làn byte nào của tín hiệu PWDATA được cập nhật trong quá trình ghi. PSTRB không được kích hoạt trong lúc APB thực hiện quá trình đọc.
PPROT	APB master	Tín hiệu mô tả quyền truy cập. Tín hiệu được điều khiển bởi APB master.
PRDATA	APB slave	Tín hiệu dữ liệu đọc. Dữ liệu được gửi từ APB slave. Độ rộng dữ liệu là 32 bit
PREADY	APB slave	Tín hiệu sẵn sàng. Tín hiệu này được phát ra phía APB slave để kéo dài quá trình truyền
PSLVERR	APB slave	Tín hiệu báo lỗi. Được phát ra phía APB Slave cho biết quá trình giao tiếp thất bại.

2.2.3 Trạng thái hoạt động của giao thức truyền thông APB

Sơ đồ máy trạng thái hữu hạn của giao thức truyền thông APB sẽ được biểu diễn ở hình 2.2:



Hình 2.2 Máy trạng thái hữu hạn của giao thức APB [2]

Ở máy trạng thái của giao thức APB, sẽ có 3 trạng thái: IDLE, SETUP và ACCESS.

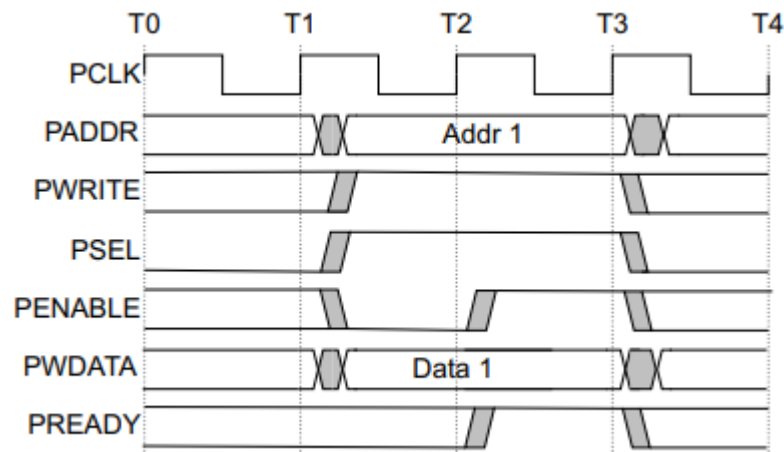
- Trạng thái IDLE: Đây là trạng thái khởi tạo , không có giao dịch xảy ra. Khi thực hiện xong giao dịch thì sẽ quay lại trạng thái này. Sau khi có tín hiệu PSEL, trạng thái sẽ chuyển sang trạng thái SETUP.
- Trạng thái SETUP: Bắt đầu trạng thái cài đặt, khi này những tín hiệu chứa thông tin sẽ được truyền hệ thống vào như PADDR, PWDATA, PWRITE, PSTRB. Sau khi đã truyền thông tin dữ liệu điều khiển vào, trạng thái sẽ chuyển qua trạng thái ACCESS khi có tín hiệu PENABLE.
- Trạng thái ACCESS: Khi ở trạng thái ACCESS, những tín hiệu điều khiển ở pha SETUP không được thay đổi. Thông thường khi APB slave chưa sẵn sàng thì PREADY sẽ ở mức thấp nên sẽ vẫn giữ ở trạng thái ACCESS. Khi PREADY được đưa lên mức cao, chứng tỏ rằng Slave sẵn sàng nhận dữ liệu ghi hoặc xuất ra để đọc. Khi kết thúc một giao dịch, nếu có một giao dịch đang còn chờ tiếp tục, thì sẽ quay lại trạng thái SETUP để đưa thông tin điều khiển mới vào. Nếu không còn giao dịch tiếp tục theo sau thì máy trạng thái sẽ quay về pha ban đầu, pha IDLE.

2.2.4 Quá trình ghi dữ liệu của giao thức truyền thông APB4

Có hai kiểu truyền khi ghi dữ liệu: Quá trình ghi không có trạng thái chờ và quá trình ghi có trạng thái chờ:

- Quá trình ghi không có trạng thái chờ:

Quá trình ghi không có trạng thái chờ được mô tả theo hình 2.3:



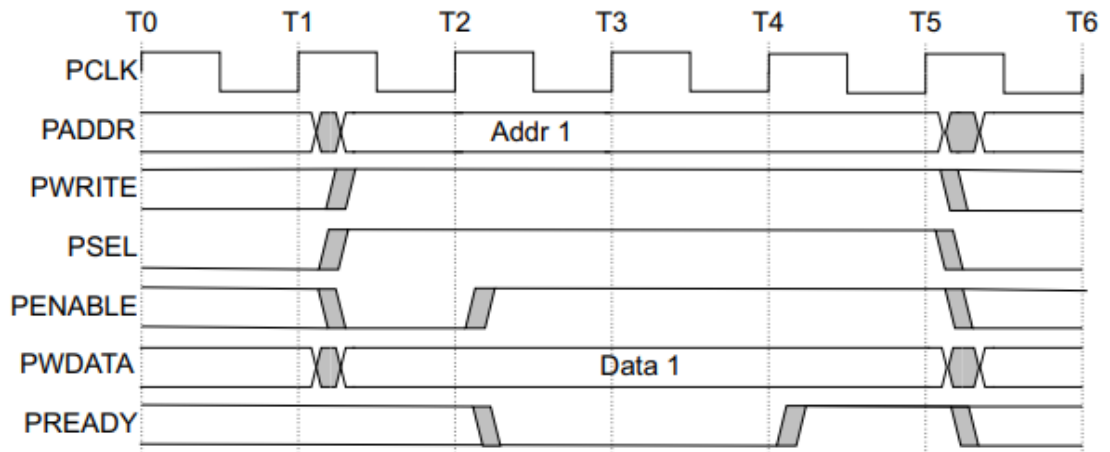
Hình 2.3 Quá trình ghi không có trạng thái chờ

Tại thời điểm T0, hiện tại hệ thống ở trạng thái IDLE nên không có gì xảy ra. Đến thời điểm T1, PSEL được nâng lên mức 1, báo hiệu cho trạng thái SETUP, khi đó các tín hiệu thông tin dữ liệu sẽ được đưa vào, tín hiệu PADDR và PWDATA và PWRITE có thông tin, PENABLE được giữ ở mức thấp. Tại thời điểm T2, khi này tín hiệu PENABLE được đưa lên mức cao nhằm cho biết rằng APB master sẵn sàng giao tiếp với APB slave. Đồng thời PREADY cũng được nâng lên mức cao để cho phép thực hiện ghi vào slave ở chu kỳ T3. Các tín hiệu PWDATA, PADDR và những tín hiệu khác phải được giữ nguyên cho đến lần chu kỳ xung tiếp theo.

Tại thời điểm T3, khi nhận được tín hiệu xung cùng với các tín hiệu điều khiển ở chu kỳ trước, khi này quá trình truyền đã xong nên PENABLE và PSEL được kéo về mức thấp.

- Quá trình ghi có trạng thái chờ:

Quá trình ghi có trạng thái chờ được mô tả theo hình 2.4:



Hình 2.4 Quá trình ghi có trạng thái chờ

Tại thời điểm T0, hiện tại hệ thống ở trạng thái IDLE nên không có gì xảy ra. Đến thời điểm T1, PSEL được nâng lên mức 1, báo hiệu cho trạng thái SETUP, khi đó các tín hiệu thông tin dữ liệu sẽ được đưa vào, tín hiệu PADDR và PWDATA và PWRITE có thông tin, PENABLE được giữ ở mức thấp. Tại thời điểm T2, khi này tín hiệu PENABLE được đưa lên mức cao nhằm cho biết rằng APB master sẵn sàng giao tiếp với APB slave. Các tín hiệu PWDATA, PADDR và những tín hiệu khác phải được giữ nguyên cho đến lần chu kỳ xung tiếp theo.

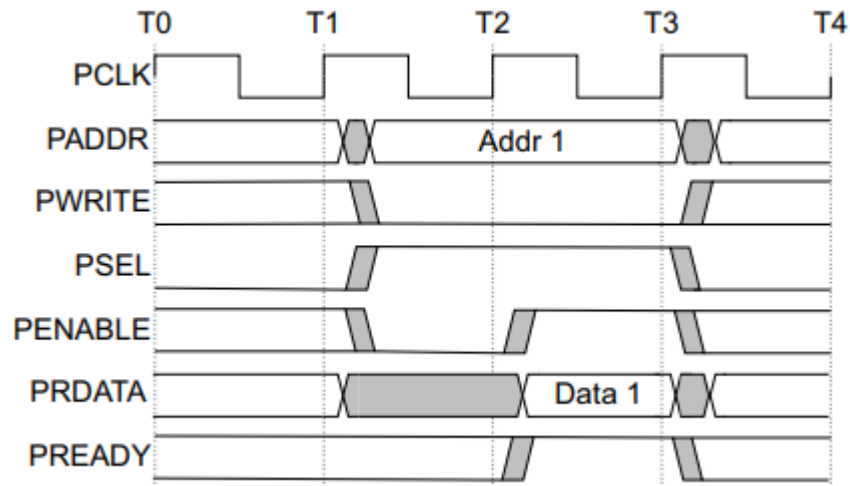
Tuy nhiên qua chu kỳ xung T3, PREADY không được nâng lên mức cao nên các tín hiệu điều khiển bị kéo dài, chờ cho đến khi có tín hiệu PREADY. Tại thời điểm T4, PREADY được nâng lên mức một để chuẩn bị truyền dữ liệu. Tại chu kỳ T5, sau khi PREADY được xác nhận, APB master tiến hành hạ hai xung PSEL và PENABLE xuống mức 0 hoàn thành quá trình truyền.

2.2.5 Quá trình đọc dữ liệu của giao thức truyền thông APB4

Cũng tương tự quá trình ghi, quá trình đọc dữ liệu của giao thức APB cũng được chia ra làm 2 trường hợp: đọc không có trạng thái chờ và đọc có trạng thái chờ.

- Quá trình đọc không có trạng thái chờ:

Quá trình đọc không có trạng thái chờ được mô tả như hình 2.5:

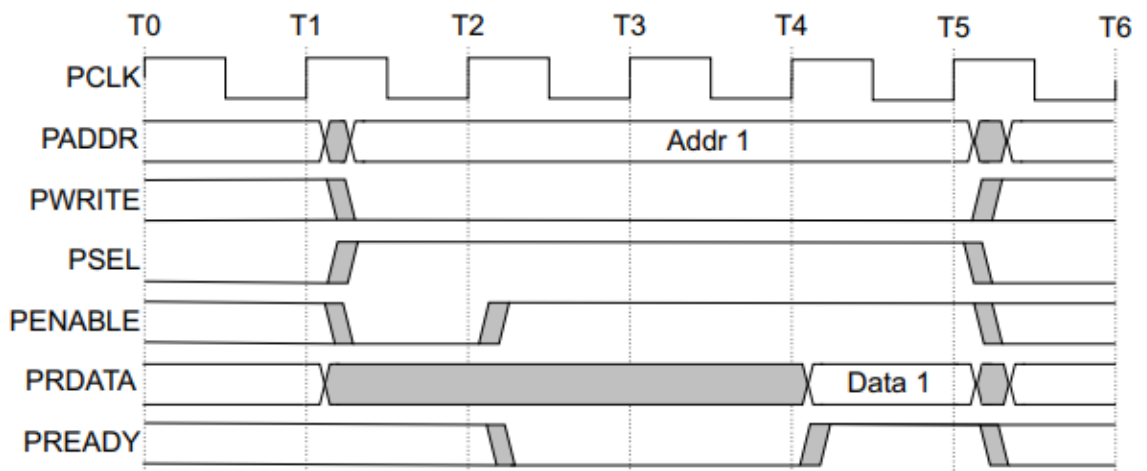


Hình 2.5 Quá trình đọc không có trạng thái chờ

Khác với quá trình ghi, tín hiệu PWRITE được tích cực mức thấp suốt quá trình truyền. Các tín hiệu còn lại đều hoạt động tương tự như quá trình ghi. Tuy nhiên phải thay đổi PWDATA, là tín hiệu dữ liệu ghi do APB master điều khiển, bằng PRDATA, là tín hiệu dữ liệu đọc sẽ được APB slave điều khiển. Tại T2, khi có cả tín hiệu PSEL, PENABLE, đặc biệt là PREADY và được nhận được cạnh lên xung vuông thì bên APB slave sẽ xuất data đến tín hiệu PRDATA phục vụ cho việc đọc và tín hiệu này tồn tại cho đến khi kết thúc trạng thái ACCESS của quá trình truyền.

- Quá trình đọc có trạng thái chờ:

Quá trình đọc có trạng thái chờ được mô tả như hình 2.6:



Hình 2.6 Quá trình đọc có trạng thái chờ

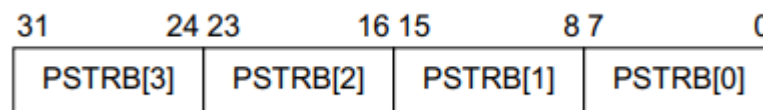
Ở quá trình này, các tín hiệu điều khiển sẽ giống như ở quá trình đọc. Cách hoạt động sẽ tương tự quá trình ghi có trạng thái chờ. Tín hiệu PSEL và PENABLE sẽ được duy trì khi không có quá trình truyền. PREADY sẽ được nâng lên mức 1 tại T4 để xác nhận việc truy cập đọc. Sau đó tại thời điểm T5, đánh dấu kết thúc quá trình truyền bằng việc đưa các tín hiệu PSEL, PENABLE và PREADY về lại mức 0.

2.2.6 Mô tả chi tiết một số tín hiệu của giao thức APB

Một số tín hiệu điều khiển được đặc biệt sẽ được giới thiệu như sau:

- Tín hiệu PSTRB:

Tín hiệu PSTRB (write strobe) có thể quyết định làn dữ liệu hợp lệ của đường dữ liệu trong một quá trình ghi. Bằng cách này, APB master có thể quyết định làn dữ liệu nào của bên APB slave được cập nhật giá trị. Một bit của đường dữ liệu PSTRB đại diện cho 1 byte (8 bits) của đường dữ liệu. Độ dài của PSTRB sẽ dựa vào độ dài của đường dữ liệu ghi PWDATA như sau: $PSTRB[n]$ tương ứng với $PWDATA[(8n+7):8n]$. Mô tả về làn dữ liệu hợp lệ khi độ dài dữ liệu là 32 bit được thể hiện ở hình 2.7:

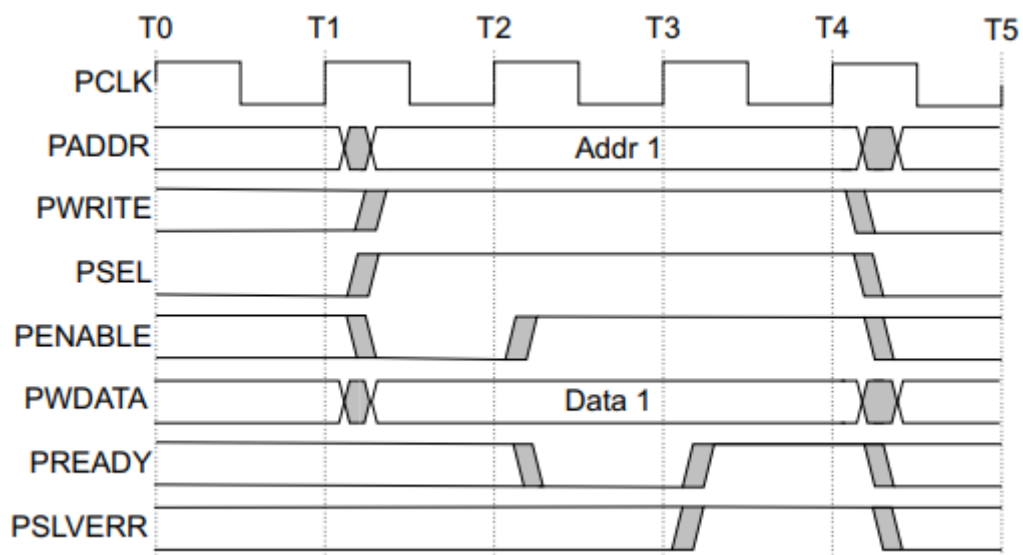


Hình 2.7 Mô tả về làn dữ liệu hợp lệ nếu độ dài dữ liệu là 32 bits.

- Tín hiệu PSLVERR:

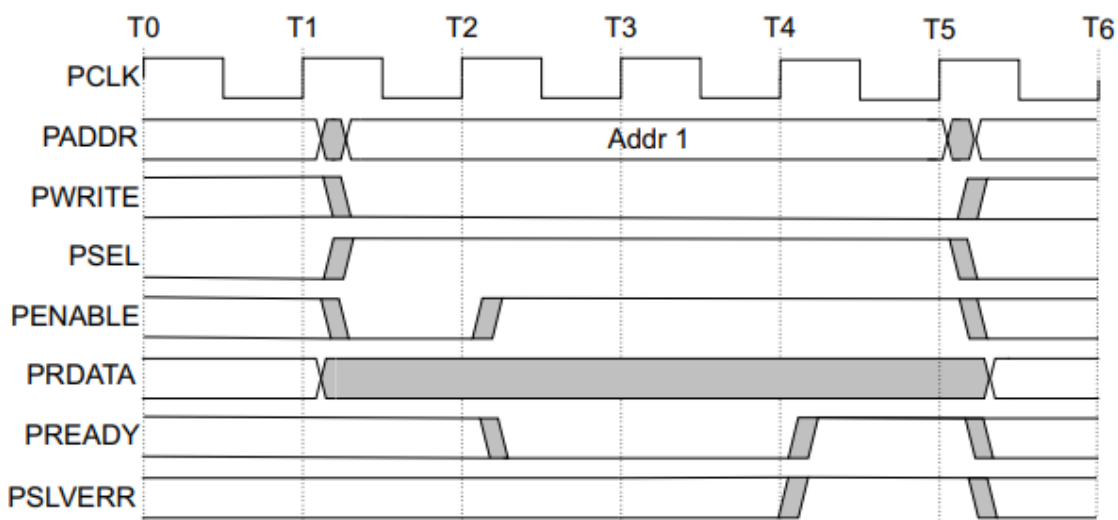
Tín hiệu PSLVERR được sử dụng để thông báo lỗi trong một quá trình truyền ở giao thức APB. Lỗi này có thể xảy ra ở cả giao dịch đọc và ghi. Tín hiệu PSLVERR chỉ được hợp lệ ở chu kỳ cuối cùng của một quá trình truyền, khi mà các tín hiệu PSEL, PENABLE và PREADY ở mức cao.

Khi xảy ra sai sót ở quá trình ghi, điều này không có nghĩa rằng là những ngoại vi không được cập nhật. Khi xảy ra lỗi ở quá trình đọc, kênh PRDATA vẫn có thể đưa ra dữ liệu bị sai. Ngoại vi không cần phải đưa dữ liệu truyền đi xuống 0 khi xảy ra lỗi. Ở phía master không thể dựa vào PSLVERR để chặn việc đọc dữ liệu từ PRDATA. Quá trình ghi xảy ra lỗi được thể hiện ở hình 2.8:



Hình 2.8 Quá trình ghi xảy ra lỗi

Như hình 2.8, khi xảy ra lỗi ở quá trình ghi, PSLVERR được đưa lên mức cao ở T3, khi mà các tín hiệu PSEL, PENABLE, PREADY đã đều lên mức cao. Quá trình ghi xảy ra lỗi được thể hiện ở hình 2.9.



Hình 2.9 Quá trình đọc xảy ra lỗi

Theo hình 2.9, khi xảy ra lỗi ở quá trình đọc, PSLVERR được đưa lên mức cao ở T3, khi mà các tín hiệu PSEL, PENABLE, PREADY đã đều lên mức cao.

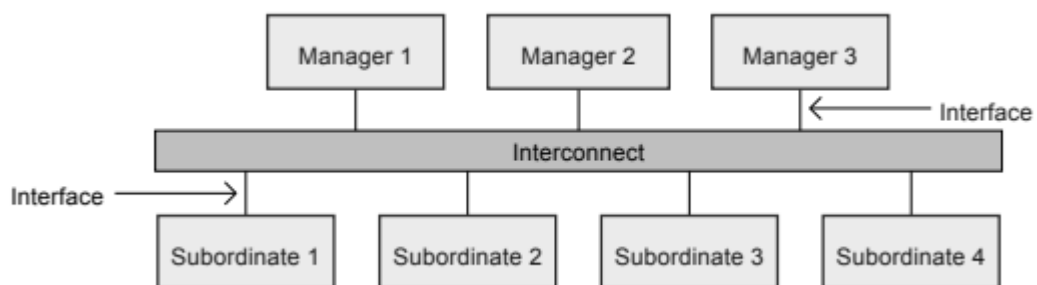
2.3 TỔNG QUAN GIAO THỨC AXI

2.3.1 Giới thiệu về giao thức AXI

Advanced eXtensible Interface hay còn gọi tắt là AXI là một chuẩn giao thức truyền thông tốc độ cao do hãng ARM phát triển. AXI4 là phiên bản giao thức AXI thứ 4 được công bố vào năm 2010 và hiện nay là mặt bằng chung của thiết kế SoC hiện đại, đặc biệt là những hệ thống trên chip sử dụng hệ sinh thái ARM. Thêm vào đó, AXI cũng có thể tương thích với các chuẩn giao tiếp khác cùng họ AMBA như chuẩn AHB và APB. Một số tính năng chính của giao thức AXI bao gồm những tính năng sau:

- Các pha truyền được tách riêng biệt, thành những pha địa chỉ, điều khiển, dữ liệu riêng biệt.
- Hỗ trợ việc chuyển đổi qua trình giao tiếp burst nên chỉ cần truyền địa chỉ bắt đầu.
- Cho phép thiết kế với các tầng thanh ghi giúp thời gian truyền của hệ thống kết nối tốt hơn.
- Hoạt động với tần số cao dựa vào kiểu truyền bắt tay (Hand-shaking) ở mỗi kênh giúp việc truyền dữ liệu đồng bộ hóa, an toàn và không bị thất thoát gói tin, mặc dù giữa các thành phần có thể chạy khác tần số khác nhau.

Giao diện giao tiếp và kết nối của giao thức AXI được thể hiện ở hình 2.10.



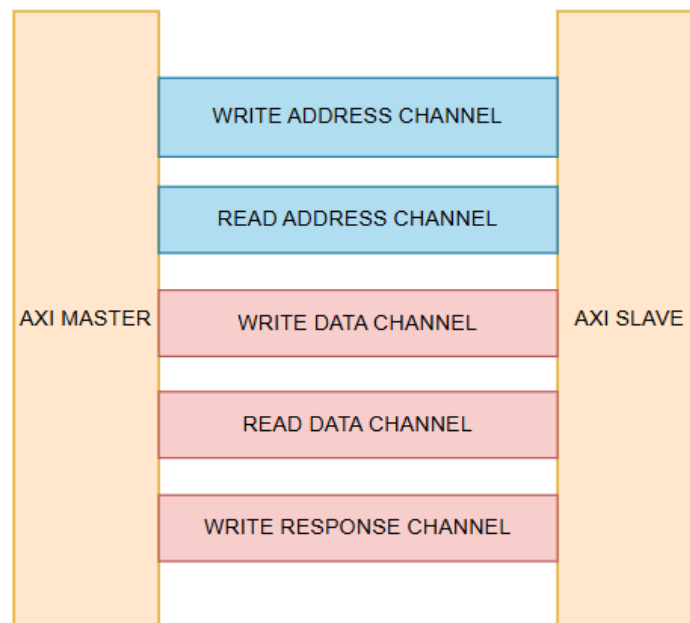
Hình 2.10 Giao diện giao tiếp và kết nối bus của giao thức AXI

Giao thức truyền thông AXI quy định phương thức giao tiếp giữa 1 master với 1 slave với nhau hoặc là giao tiếp thông qua một mạng lưới kết nối gọi là Interconnection. Giống như các giao thức truyền thông khác, AXI cũng có 2 hoạt

động cơ bản là đọc với ghi với việc phía master sẽ truy vấn truy cập ghi vào phía slave và phía master sẽ gửi truy vấn truy cập đọc, phía slave sẽ gửi dữ liệu đọc cho master.

2.3.2 Cấu trúc kênh truyền của giao thức AXI

Giao thức AXI hoạt động dựa trên 5 kênh truyền dữ liệu riêng biệt, mỗi kênh đảm nhiệm một chức năng khác nhau. Việc truyền và nhận tín hiệu giữa các kênh được thực hiện hoàn toàn độc lập, không phụ thuộc vào hoạt động của các kênh còn lại. Năm kênh này bao gồm: kênh địa chỉ ghi (AW), kênh dữ liệu ghi (W), kênh địa chỉ đọc (AR), kênh dữ liệu đọc (R), và kênh phản hồi ghi (B). Các kênh truyền được thể hiện ở hình 2.11.



Hình 2.11 Các kênh của một kênh truyền AXI [3]

Chức năng của các kênh truyền được mô tả như sau:

- Kênh địa chỉ ghi AW: Cho biết địa chỉ và thông tin điều khiển của một chuyển đổi ghi được truyền từ master đến slave.
- Kênh địa chỉ đọc AR: Cho biết địa chỉ và thông tin điều khiển của một chuyển đổi đọc được truyền từ master đến slave.
- Kênh dữ liệu ghi W: Dữ liệu ghi sẽ được truyền từ master đến slave.

- Kênh dữ liệu đọc R: Dữ liệu đọc và thông tin phản hồi của một chuyển đổi đọc được truyền từ slave đến master.
- Kênh phản hồi ghi B: Truyền thông tin phản hồi của một chuyển đổi ghi từ slave đến master.

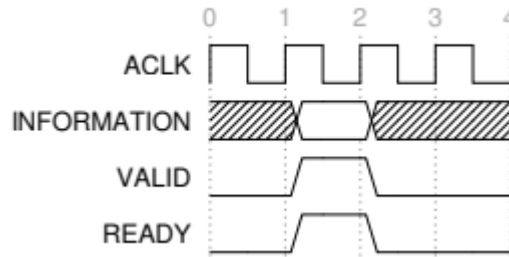
Trong đó, khi thực hiện một giao dịch hay một truy cập, bên master sẽ gửi đi những thông tin sau:

- Dữ liệu địa chỉ: xác định ngoại vi đích đến, nơi mà quá trình giao dịch sẽ được thực hiện.
- Tín hiệu điều khiển: các thuộc tính về kích thước dữ liệu, độ dài dữ liệu, kiểu truyền.
- Thông tin dữ liệu: là những dữ liệu truyền đi hoặc nhận trong giao dịch.
- Tín hiệu phản hồi: kiểm tra trạng thái chuyển đổi có truyền thành công dữ liệu hay không hay bị lỗi.

Một giao dịch AXI bao gồm nhiều phiên truyền từ AXI master đến AXI slave. Ví dụ như ở một giao dịch đọc AXI thì sẽ có một phiên truyền yêu cầu đọc và một hoặc nhiều phiên truyền dữ liệu đọc.

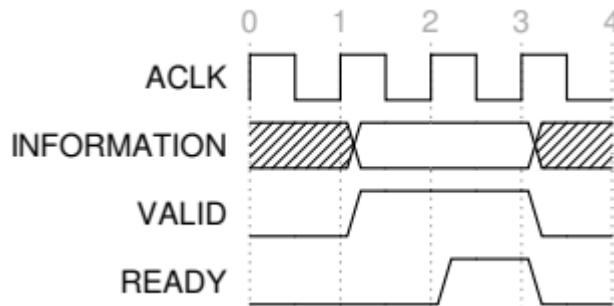
2.3.3 Các cơ chế *Valid – Ready (Hand-Shake)*

Tất cả các kênh truyền của AXI đều sử dụng cơ chế bắt tay valid – ready giống nhau để truyền địa chỉ, dữ liệu và thông tin điều khiển. Cơ chế điều khiển hai chiều có nghĩa rằng cả phía master và slave đều có thể điều khiển lưu lượng thông tin của cả 2 bên. Bên gửi thông tin (Source) sẽ tạo tín hiệu valid để chỉ ra khi nào địa chỉ, dữ liệu và thông tin điều khiển sẵn sàng [4]. Còn bên nhận thông tin (Destination) sẽ phát tín hiệu READY thông báo với bên master rằng đã sẵn sàng nhận thông tin. Việc truyền dữ liệu giữa hai bên chỉ xảy ra khi cả hai tín hiệu valid và ready đều ở mức cao. Ví dụ về cơ chế bắt tay hai chiều được thể hiện ở hình 2.12.



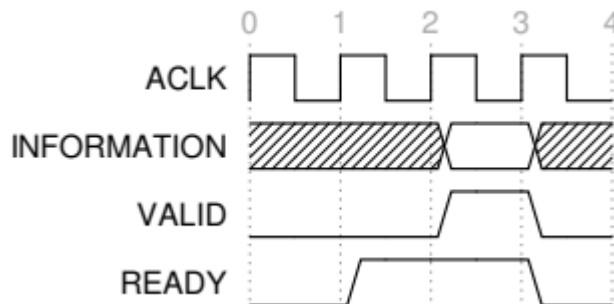
Hình 2.12 Minh họa tín hiệu bắt tay ở kênh AXI

Phía master không cần phải đợi khi có tín hiệu ready thì mới nâng tín hiệu valid. Nhưng mà khi đã có tín hiệu valid, nó phải được giữ nguyên cho đến khi quá trình bắt tay xảy ra tại thời điểm cạnh lên xung vuông khi có 2 tín hiệu valid và ready đồng thời có tín hiệu như hình 2.13 sau.



Hình 2.13 Minh họa tín hiệu bắt tay với tín hiệu VALID có trước

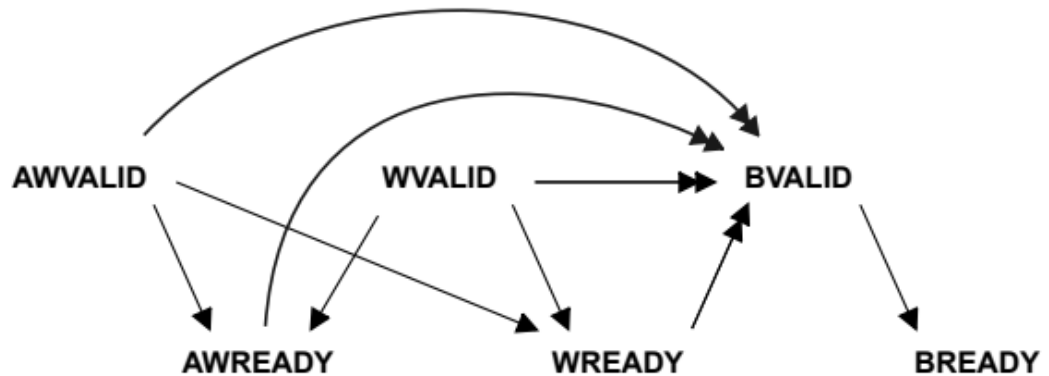
Ở đích đến có thể đợi tín hiệu valid trước khi gửi tín hiệu ready tương ứng. Khi đã gửi tín hiệu ready, đích đến được phép tắt tín hiệu ready trước khi valid được gửi đi như hình 2.14 sau.



Hình 2.14 Minh họa tín hiệu bắt tay với tín hiệu READY có trước

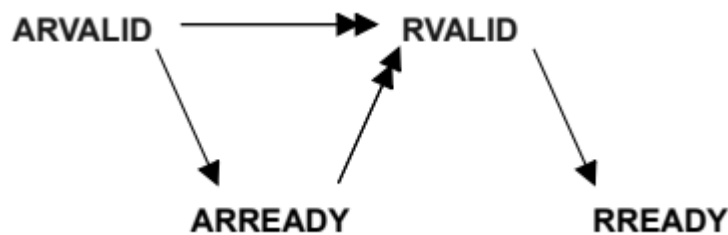
Với cơ chế bắt tay hai chiều, mỗi kênh có khả năng hoàn thành giao dịch của nó độc lập, cho phép tiếp tục thực hiện giao dịch mà không cần chờ đợi những

kênh khác sẵn sàng. Điều này giúp tăng tính đồng thời và hiệu quả của việc giao tiếp của những thành phần trong hệ thống.



Hình 2.15 Mối liên hệ giữa các tín hiệu bắt tay ở giao dịch ghi

Hình 2.14 và 2.15 biểu diễn sự phụ thuộc tín hiệu của quá trình ghi và đọc với mũi tên đơn là tín hiệu có thể được đi trước hoặc sau, còn mũi tên đôi là tín hiệu ở đầu nhọn mũi tên chỉ được phát sau tín hiệu ở phía bắt đầu của mũi tên.



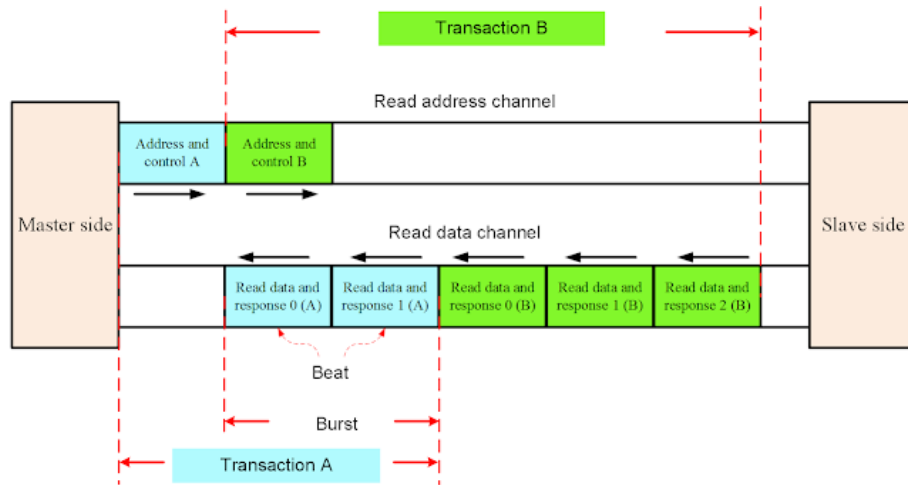
Hình 2.16 Mối liên hệ giữa các tín hiệu bắt tay ở giao dịch đọc

2.3.4 Quá trình đọc và ghi dữ liệu

Một giao dịch đọc trong giao thức AXI được điều phối thông qua hai kênh: kênh địa chỉ đọc (AR) và kênh dữ liệu đọc (R). Trong giao dịch này, dữ liệu và phản hồi được truyền trên cùng một kênh – kênh R. Quá trình bắt đầu khi phía khởi tạo (master) gửi thông tin địa chỉ cùng các tín hiệu điều khiển thông qua kênh AR. Sau đó, phía đích (slave) sẽ phản hồi bằng cách truyền dữ liệu và tín hiệu phản hồi tương ứng trên kênh R, dựa theo các tín hiệu điều khiển đã nhận từ kênh AR.

Việc phân tách các pha giao tiếp thành pha truyền địa chỉ, pha truyền dữ liệu và pha truyền tín hiệu điều khiển giúp cho nhiều giao dịch có thể được khởi tạo kế tiếp nhau mà không cần chờ giao dịch trước hoàn tất. Điều này góp phần nâng cao

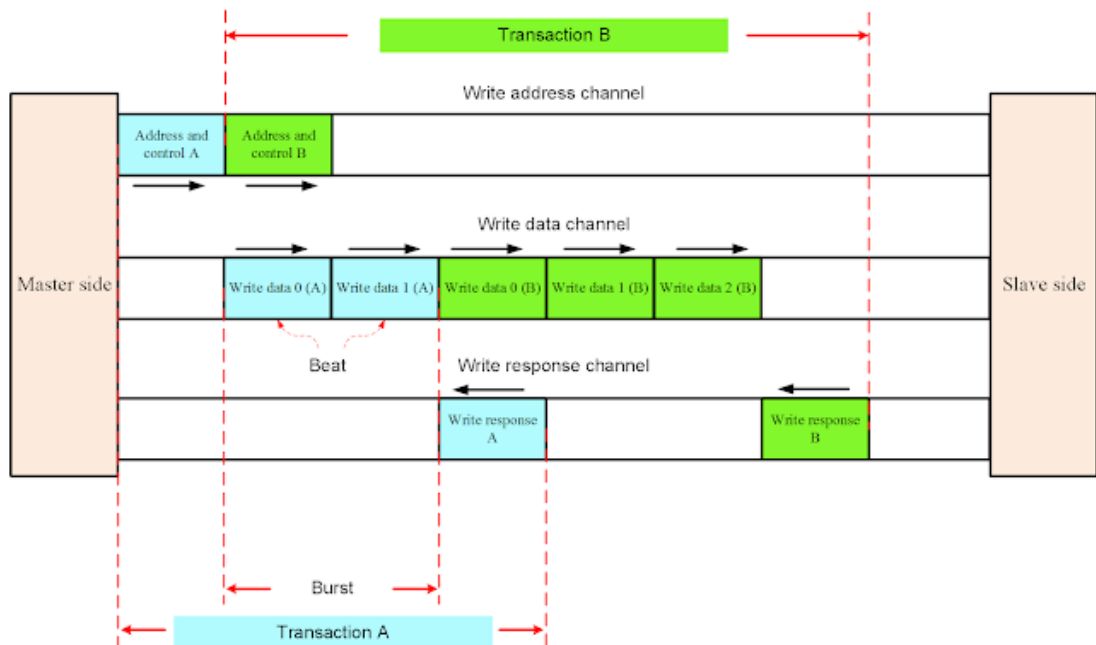
hiệu quả truyền dữ liệu, như được minh họa trong hình 2.17. Hình 2.18 thể hiện cụ thể một giao dịch đọc trong giao thức AXI.



Hình 2.17 Giao dịch đọc trong giao thức AXI

Một khung truyền (burst) là toàn bộ gói tin đi qua trong một giao dịch. Những gói tin trong một khung truyền này có thể gọi là nhịp truyền (beat). Mỗi giao dịch đều có những thông tin điều khiển một burst được quy định bởi các đường điều khiển.

Hình 2.18 mô tả quá trình giao dịch ghi được thực hiện trong giao thức AXI



Hình 2.18 Giao dịch ghi trong giao thức AXI

Đối với giao dịch ghi, phía nguồn sẽ gửi địa chỉ, thông tin điều khiển qua kênh AW, và gửi dữ liệu thông qua kênh W, đồng thời sẽ nhận phản hồi từ đích đến thông qua kênh B. Một giao dịch ghi cần sử dụng đến ba kênh truyền.

Tuy nhiên, chỉ có một tín hiệu phản hồi được gửi về khi cả giao dịch ghi hoàn thành. Còn với giao dịch đọc, mỗi khi có dữ liệu đọc thì sẽ có một phản hồi được gửi về tương ứng, giá trị của các phản hồi có thể giống hoặc khác nhau. Với chuyển đổi A như hình 2.17, phản hồi 0 đang cho thấy dữ liệu đọc đầu tiên "không lỗi" nhưng phản hồi 1 lại đang báo lỗi cho dữ liệu đọc thứ 2.

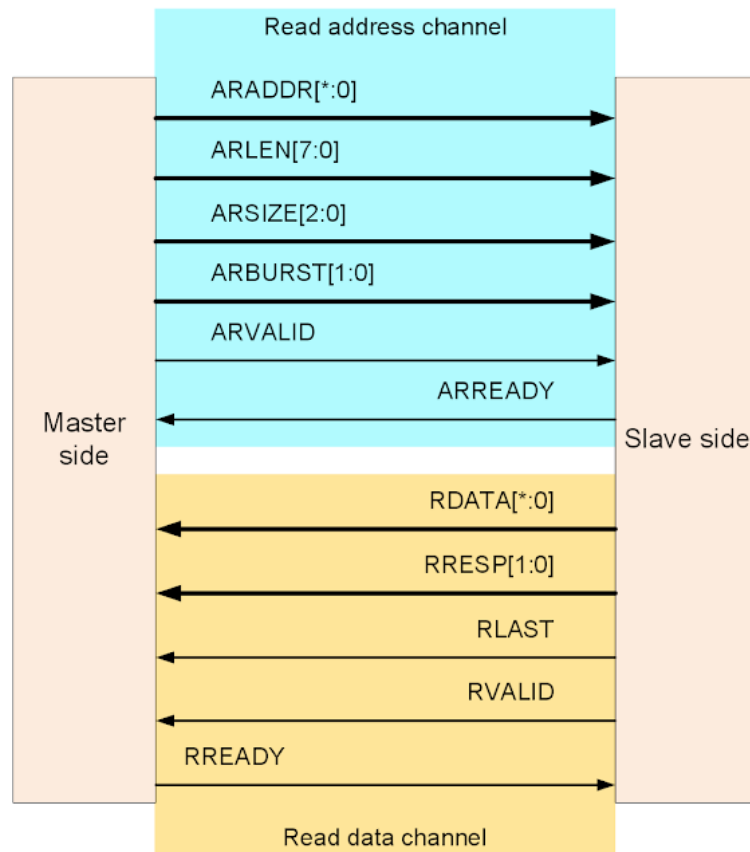
2.3.5 Cơ chế chuyển đổi burst trong giao thức AXI

AXI là một giao thức truyền thông sử dụng cơ chế burst (khung truyền) để kết nối và trao đổi dữ liệu. Mỗi giao dịch trong AXI tương ứng với một khung truyền, do đó còn được gọi là giao dịch theo khung truyền. Đặc tính của cơ chế burst được xác định thông qua các thông tin điều khiển được truyền trên các kênh địa chỉ AR (đọc) hoặc AW (ghi).

Khi bắt đầu một giao dịch, phía nguồn (master) sẽ gửi địa chỉ và các tín hiệu điều khiển qua kênh AR hoặc AW. Địa chỉ được truyền là địa chỉ của byte đầu tiên trong burst, đồng thời kèm theo các thông tin điều khiển như: độ rộng dữ liệu, độ dài của burst, kiểu truyền dữ liệu, cùng các thuộc tính khác. Trong một burst, dữ liệu được chia thành các gói nhỏ. Phía khởi tạo không cần phải tự tính toán địa chỉ của từng gói nhỏ, vì việc này sẽ do phía đích (slave) đảm nhiệm, dựa trên thông tin điều khiển đã nhận được. Các kênh có những đường tín hiệu thực hiện chức năng điều khiển và tín hiệu địa chỉ trong hình 2.19 và 2.20:

- Kênh AR/AW:
 - AxADDR (AWADDR/ARADDR): Địa chỉ của lần truyền dữ liệu đầu tiên trong burst.
 - AxLEN (AWLEN/ARLEN): Độ dài khung truyền, hay là độ dài burst, xác định số lượng gói tin trong khung truyền đó.
 - AxSIZE (AWSIZE/ARSIZE): Độ dài dữ liệu hay độ dài gói tin trong khung truyền đó.

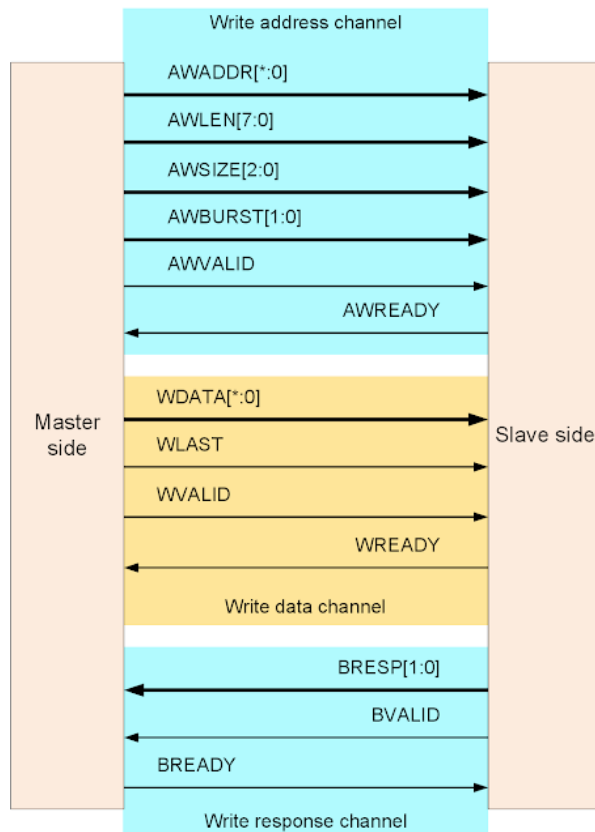
- AxBURST (AWBURST/ARBURST): Xác định kiểu truyền hay loại burst nào được lựa chọn.
- AxVALID: (AWVALID/ARVALID): Tín hiệu valid của kênh địa chỉ.
- AxREADY: (AWREADY/ARREADY): Tín hiệu READY của kênh địa chỉ.



Hình 2.19 Nhóm kênh phục vụ cho giao dịch đọc

- Kênh R:

- RDATA: Đường dữ liệu đọc.
- RRESP: Tín hiệu phản hồi thông báo trạng thái của giao dịch đọc hiện tại.
- RLAST: Tín hiệu thông báo đây là gói tin cuối cùng của một khung truyền đọc.
- RVALID: Tín hiệu VALID của kênh R
- RREADY: Tín hiệu READY của kênh R



Hình 2.20 Nhóm kênh phục vụ cho giao dịch ghi

- Kênh W:

- WDATA: Đường dữ liệu ghi
- WLAST: Tín hiệu thông báo đây là gói tin cuối cùng của một khung truyền ghi.
- WVVALID: Tín hiệu VALID của kênh W

- Kênh B:

- BRESP: Tín hiệu phản hồi báo hiệu trạng thái của giao dịch ghi
- BVALID: Tín hiệu VALID của kênh B
- BREADY: Tín hiệu READY của kênh B

Trong một giao dịch, số lượng gói tin (beat) được truyền đi trong một khung truyền (burst) được quy định bởi tín hiệu AxLEN theo công thức như sau:

$$\text{Số lượng beat (số gói tin)} = \text{AxLEN} + 1.$$

Độ rộng tối đa của một gói tin (một beat) được quy định bởi tín hiệu AxSIZE. Kích thước này không được lớn hơn độ rộng của tín hiệu dữ liệu WDATA và RDATA. Kích thước dữ liệu ứng với giá trị AxSIZE được thể hiện ở bảng 2.2.

Bảng 2.2 Kích thước dữ liệu tương ứng với tín hiệu AxSIZE

AxSIZE[2:0]	Số lượng byte
3'b000	1
3'b001	2
3'b010	4
3'b011	8
3'b100	16
3'b101	32
3'b110	64

Ngoài các tín hiệu điều khiển AxLEN, AxSIZE, còn có tín hiệu điều khiển kiểu truyền AxBURST. Tín hiệu AxBURST xác định loại burst được sử dụng, bao gồm 3 loại sau: FIXED, INCR và WRAP. Các loại burst tương ứng với giá trị AxBurst sẽ được thể hiện ở bảng 2.3.

Bảng 2.3 Các loại burst tương ứng với giá trị của tín hiệu AxBURST

AxBURST[1:0]	Kiểu truyền
2'b00	FIXED
2'b01	INCR
2'b02	WRAP

- FIXED: Trong mỗi giao dịch, địa chỉ chỉ cần được phát một lần duy nhất cho chuyển giao đầu tiên trong khung truyền, và các chuyển giao tiếp theo trong khung truyền sẽ sử dụng lại địa chỉ đó. Tính năng thường dùng để kiểm tra xem FIFO của hệ thống có hoạt động hay không.
- INCR: Địa chỉ của mỗi lần truyền dữ liệu tiếp theo là giá trị của bộ đếm lên theo địa chỉ đầu tiên, nó được quy định bởi AxSIZE. Điều này cho phép các giao dịch trong khung truyền có các địa chỉ liên tiếp và tăng dần dựa trên địa chỉ đầu tiên và kích thước của khung truyền.
- WRAP: Giống với INCR nhưng giá trị các bit nhỏ nhất (LSB) của thông tin địa chỉ mới được tăng số lượng, bit tăng được xác định dựa trên giá trị của tín hiệu AxLEN và AxSIZE.

Với kiểu truyền INCR và WRAP, những địa chỉ ngõ vào AxADDR phải được căn chỉnh phù hợp với tín hiệu AxSIZE, địa chỉ căn chỉnh phải là bội số của tín hiệu AxSIZE. Để minh họa kiểu truyền này dễ hơn thì 2 kiểu truyền sẽ được mô tả qua ví dụ với các thông số là địa chỉ khởi điểm (AxADDR) là 0x1C, độ dài của khung truyền (AxLEN) là 0x03, độ dài của những gói tin trong khung truyền (AxSIZE) = 0x02.

- INCR BURST sẽ làm tăng địa chỉ tuần tự theo kích thước gói tin sau mỗi lần truyền. Địa chỉ tiếp theo sẽ bằng địa chỉ hiện tại thêm 4 vì độ dài gói tin là 4 byte. Vì thế địa chỉ tiếp theo sẽ từ 0x1C đến 0x20, 0x24, 0x28. Trong khung truyền có 4 gói tin vì thế chỉ có 4 địa chỉ được sử dụng.
- WRAP BURST vẫn sẽ tăng địa chỉ nhưng mà chỉ giới hạn trong khung nhất định, gọi là *Wrap Boundary* (vùng giới hạn wrap), khi vượt qua giới hạn thì địa chỉ đó sẽ quay lại đầu *Wrap Boundary*. Kích thước của vùng giới hạn wrap sẽ được tính như sau:
 - $Wrap\ Boundary\ Size = (AxLEN + 1) * (2^{AxSIZE}) = (3+1) * (2^2) = 4 * 4 = 16$. Vậy vùng giới hạn wrap có kích thước là 16 bytes.
 - $Wrap\ Boundary = INT(AxADDR / (2^{AxSIZE} * (AxLEN + 1))) * (2^{AxSIZE} * (AxLEN + 1)) = INT(0x1C / (4 * 4)) * (4 * 4) = 0x10$.
Ta có 0x1C chuyển sang hệ thập phân là 28, lấy cận dưới của 28/16 là 1, sau đó lấy 1*16 là 16 chuyển sang hệ thập lục phân sẽ thành 0x10. Vì thế địa chỉ của kiểu truyền WRAP sẽ từ 0x10 đến 0x1F. Địa chỉ sẽ tăng từ 0x1C, sau đó quay lại 0x10 tăng lên 0x14, 0x18.

Có 2 loại tín hiệu phản hồi, RRESP là tín hiệu phản hồi trong quá trình đọc còn BRESP là tín hiệu phản hồi trong quá trình ghi.

Bảng 2.4 Các loại phản hồi tương ứng với giá trị của RRESP/BRESP

xRESP[1:0]	Phản hồi
2'b00	OKAY
2'b01	EXOKAY
2'b10	SLVERR
2'b11	DECERR

- OKAY: Đã hoàn tất quá trình truyền dữ liệu. Có nghĩa là quá trình truyền dữ liệu đã thành công và không có lỗi nào xảy ra.
- EXOKAY: Truyền dữ liệu thành công với quyền truy cập độc quyền.
- SLVERR: Dữ liệu đã đến được đích, tuy nhiên đã gặp lỗi trong quá trình truyền dữ liệu.
- DECERR: Trong quá trình truyền dữ liệu đã phát sinh lỗi giải mã. Điều này xảy ra nếu địa chỉ của yêu cầu không được thiết bị nhận dạng.

2.4 TỔNG QUAN VỀ MÔI TRƯỜNG KIỂM TRA UVM

Universal Verification Methodology (UVM) là một chuẩn phương pháp luận được sử dụng phổ biến trong lĩnh vực xác minh thiết kế phần cứng, đặc biệt là trong thiết kế mạch số (Digital Design Verification). UVM được phát triển dựa trên SystemVerilog và kế thừa từ các phương pháp luận xác minh trước đó như OVM (Open Verification Methodology) và VMM (Verification Methodology Manual). Mục tiêu chính của UVM là cung cấp một khuôn khổ nhất quán, linh hoạt và có khả năng tái sử dụng cao cho việc xây dựng các testbench xác minh tự động. [5]

Trong quá trình phát triển các hệ thống vi mạch (SoC - System on Chip), việc xác minh thiết kế chiếm một tỷ lệ rất lớn trong tổng thời gian và nguồn lực. Khi các hệ thống ngày càng trở nên phức tạp, đòi hỏi cần có các phương pháp xác minh hiện đại, có khả năng mở rộng và tái sử dụng cao. Các phương pháp xác minh truyền thống như viết testbench thủ công với Verilog hoặc SystemVerilog thường thiếu tính tổ chức, khó bảo trì và không hiệu quả khi mở rộng quy mô. UVM ra đời như một giải pháp để giải quyết các vấn đề đó.

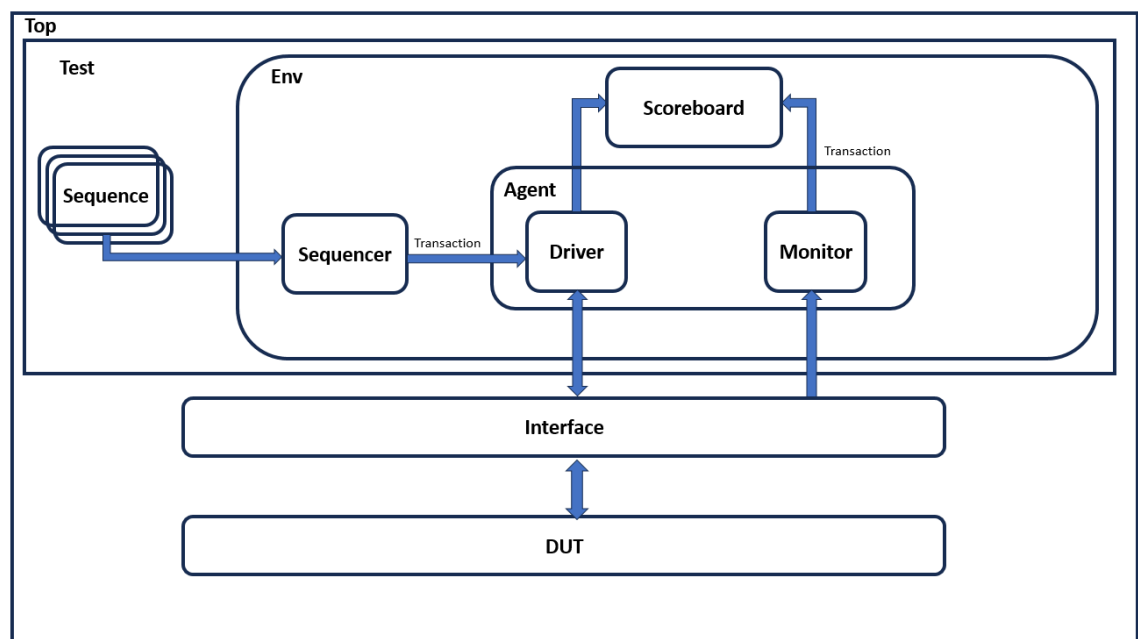
UVM thể hiện được một số ưu điểm như:

- Khả năng tái sử dụng: Các thành phần trong UVM được thiết kế theo kiểu hướng đối tượng (OOP) nên cho phép người dùng có thể sử dụng cho các dự án khác nhau
- Tính bao quát: với các cơ chế randomization và functional coverage, người dùng có thể kiểm tra một cách tự động từ giúp giúp phát hiện các lỗi trong thiết kế nhanh hơn

2.5 CẤU TRÚC MÔI TRƯỜNG UVM

UVM cung cấp bộ thư viện theo các lớp hướng đối tượng (Object-Oriented Classes) được xây dựng trên ngôn ngữ System Verilog. UVM bao gồm các thành phần cơ bản của một môi trường kiểm tra như Test, Environment, Agent, Driver, Monitor, Scoreboard, Sequence. [6]

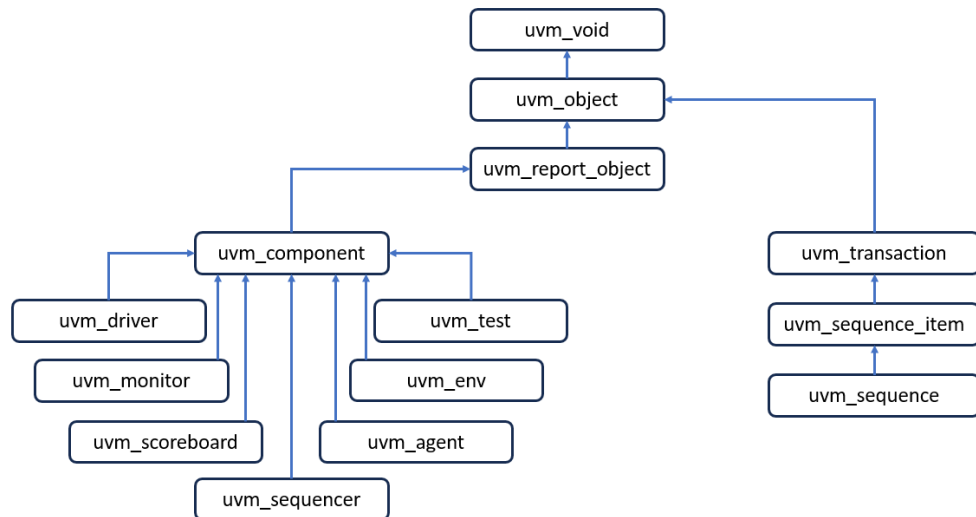
Kiến trúc testbench UVM phổ biến được thể hiện ở hình 2.21:



Hình 2.21 Kiến trúc môi trường UVM

Kiến trúc này mô tả cách các thành phần UVM tương tác với nhau bên trong testbench từ đó cho phép các thủ tục kiểm tra được tiến hành. Trong một UVM testbench, các thành phần được tạo ra từ lớp `uvm_component` gốc có vai trò kiểm soát cấu trúc và hành vi của testbench phục vụ cho phù hợp với yêu cầu kiểm tra ban đầu. Các thành phần này là cố định và nhất quán trong suốt quá trình kiểm tra.

Ở phía ngược lại, các thành phần sequence mang tính tạm thời. Những thành phần này được tạo ra trong quá trình trình test, sau đó chúng được thực thi và cuối cùng là bị tắt khi những nhiệm vụ (task) của chúng hoàn tất. Trong đó, Sequence chứa những tình huống test bao gồm các gói tin truyền (transaction), những gói tin sẽ được truyền đến DUT thông qua UVM_Sequencer và UVM_Driver. Phân cấp các lớp trong UVM được trình bày ở hình 2.22.



Hình 2.22 Sơ đồ phân cấp trong môi trường UVM

Các phân cấp ở hình 2.22 bao gồm `uvm_void` là lớp cơ sở cho các lớp còn lại. Trong khi đó `uvm_object` cung cấp các phương thức `create`, `cast`,... Lớp `uvm_report_object` cung cấp các chức năng báo cáo cho UVM bao gồm: `uvm_info`, `uvm_error`, `uvm_warning`, `uvm_fatal`. Trong lớp `uvm_component` sẽ bao gồm các thành phần như `driver`, `monitor`,... Còn lớp `uvm_transaction` sẽ bao gồm các thành phần sequence, đây là các lớp đối tượng chứa lớp con liên quan đến sequence.

2.5.1 UVM Testbench top

Để sử dụng được UVM thì các kết nối giữa DUT và Interface phải được đảm bảo để các thành phần UVM phía sau có thể nhận được các tín hiệu. Test sẽ được bắt đầu thông qua `run_test()`. UVM Testbench Top sẽ khởi tạo DUT và lần lượt thực thi các test.

2.5.2 UVM Test

UVM_Test là lớp bao gồm các trường hợp kiểm tra tính năng và tính chính xác của DUT. Khi người dùng đã xác định được kế hoạch kiểm tra thì có thể viết một đoạn mã và sử dụng chung cho nhiều trường hợp test thay vì phải viết nhiều đoạn mã, giảm thiểu thời gian kiểm tra.

UVM_Test sẽ tạo, cấu hình các thành phần môi trường và khởi động sequence cho sequencer. Bên cạnh đó, nó cũng được dùng để tạo ra thêm các test case khác có độ chi tiết cao hơn bằng cách mở rộng từ chính UVM_Test, từ đó thể hiện được tính tái sử dụng cao trong việc xây dựng môi trường kiểm tra.

2.5.3 UVM Environment

Khối UVM_Env bao gồm Sequencer và Agent. Khối UVM_Test cấu hình một số tùy chọn xuống môi trường cho các trường hợp test khác nhau. UVM_Env trong mô hình này là một lớp cơ bản và bao gồm các khối khác bên trong. Tuy nhiên, đây cũng có thể là một lớp môi trường phụ cho một môi trường lớn hơn trong nhiều loại test khác nhau.

2.5.4 UVM Agent

UVM_Agent có nhiệm vụ tạo ra các tín hiệu để gửi đến DUT. UVM_Agent có các thành phần dưới cấp như Driver, Monitor và Scoreboard. UVM_Agent giúp quản lý luồng dữ liệu kiểm thử (stimulus) và thu thập dữ liệu đầu ra (response). Agent có thể hoạt động ở hai chế độ: active (gửi stimulus thông qua sequencer và driver) hoặc passive (chỉ quan sát qua monitor). Việc đóng gói logic liên quan đến giao tiếp trong một agent giúp tái sử dụng và cấu trúc testbench một cách rõ ràng, linh hoạt.

2.5.5 UVM Driver

UVM_Driver có nhiệm vụ chuyển luồng dữ liệu kiểm thử (stimulus) thành trình tự đúng để giao tiếp được với DUT. Driver sẽ nhận được sequence item từ phía sequencer và chuyển thành các trình tự thích hợp để truyền vào DUT. Trong

một số trường hợp, driver cũng có thể nhận ngược lại các tín hiệu từ DUT và phản hồi ngược lại đến sequence để hoàn thành một hoạt động truyền (transaction).

2.5.6 UVM Monitor

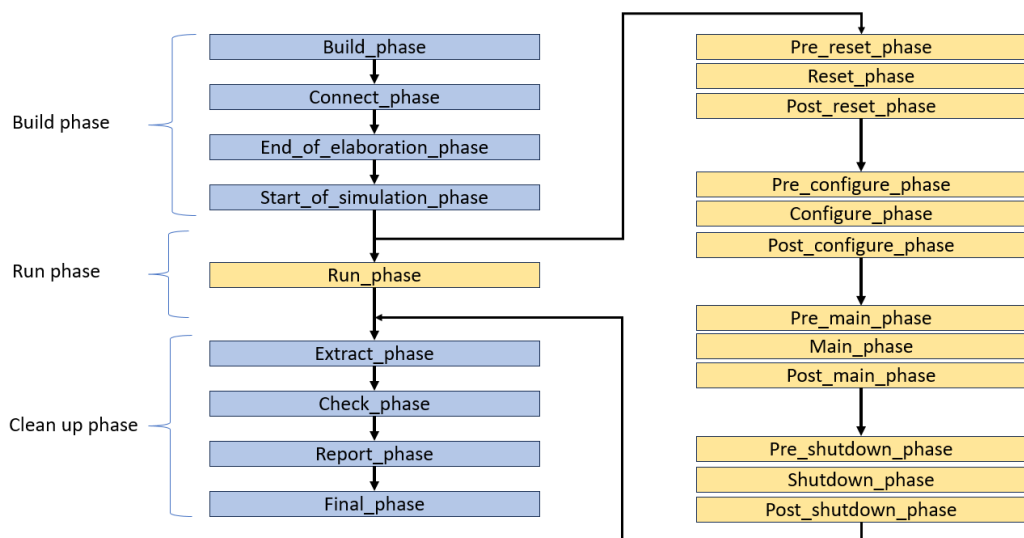
UVM_Monitor là một thành phần để nhận data trả ra từ phía DUT và gửi chúng lên Scoreboard phục vụ mục đích so sánh. Monitor nhận được dữ liệu và tín hiệu từ phía DUT thông qua Interface và chuyển chúng thành dạng “sequence item” và chuyển chúng lên scoreboard thông qua Analysis Port.

2.5.7 UVM Scoreboard

UVM_Scoreboard là thành phần với nhiệm vụ chính là kiểm tra chức năng của DUT. UVM_Scoreboard sẽ nhận những gói tin từ phía Driver và Monitor gửi đến để có thể tiến hành so sánh. UVM_Scoreboard người dùng có thể sử dụng những macro như `uvm_info`, `uvm_error`, `uvm_warning` hay `uvm_fatal` nhằm hiển thị các thông tin cần thiết liên quan đến quá trình so sánh dữ liệu.

2.6 CƠ CHẾ PHA TRONG UVM

Nhằm đảm bảo việc chạy testbench một cách đồng bộ giữa các khối, UVM đã tổ chức các bước chính trong khi chạy mô phỏng thành các pha. Sau đây là danh sách và thứ tự sắp xếp của các pha:



Hình 2.23 Danh sách và trình tự các pha trong UVM

Theo hình 2.23, quá trình mô phỏng của UVM có 3 nhóm pha chính:

- Pha khởi tạo (Build phase): Đây là pha được gọi từ thời điểm bắt đầu mô phỏng. Đây là lúc testbench được xây dựng và cấu hình. Các phương thức được sử dụng trong nhóm pha khởi tạo là function vì các quá trình thực hiện ở pha khởi tạo sẽ không tiêu tốn thời gian trong mô phỏng.
- Pha thực thi (Run phase): đây là pha thực hiện các quá trình đưa dữ liệu hoặc lấy dữ liệu ra khỏi DUT. Phương thức sử dụng trong pha thực thi sẽ là task, vì các quá trình xử lý trong pha thực thi sẽ tiêu tốn thời gian mô phỏng.
- Pha kết thúc (Clean up phase): đây là pha sẽ thu thập những thông tin của các test và tạo báo cáo cho các test này.

CHƯƠNG 3

THIẾT KẾ HỆ THỐNG

3.1 YÊU CẦU CỦA HỆ THỐNG

Thiết kế cầu nối AXI sang APB bao gồm các yêu cầu sau:

- Cầu chuyển đổi từ giao thức AXI tốc độ cao sang bus tốc độ thấp APB
- Hỗ trợ 1 master AXI và hỗ trợ kết nối 4 Slave APB
- Hỗ trợ truyền 2, 4, 8, 16 gói tin dữ liệu trong 1 burst
- Hỗ trợ truyền các gói tin với kích thước 8, 16 và 32 bit

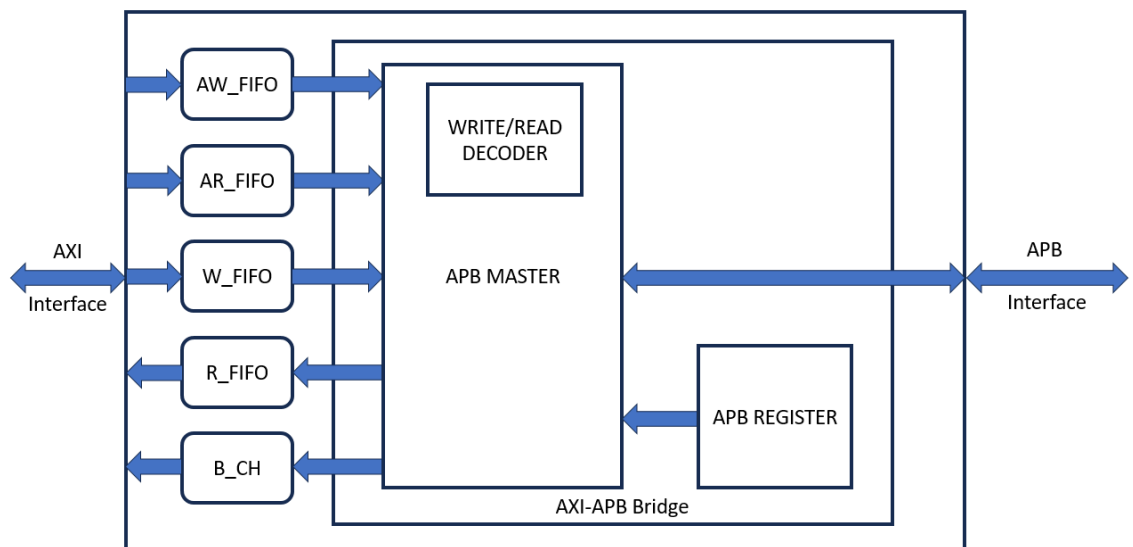
Thiết kế môi trường UVM bao gồm các yêu cầu sau:

- Xây dựng môi trường UVM kiểm tra chức năng của cầu nối AXI-APB
- Kết quả báo cáo (UVM Report) trình bày tường minh, dễ dàng theo dõi

3.2 THIẾT KẾ HỆ THỐNG

3.2.1 Sơ đồ khối tổng quát

Sơ đồ khối tổng quát của thiết kế cầu nối AXI – APB được thể hiện ở hình 3.1 như sau:

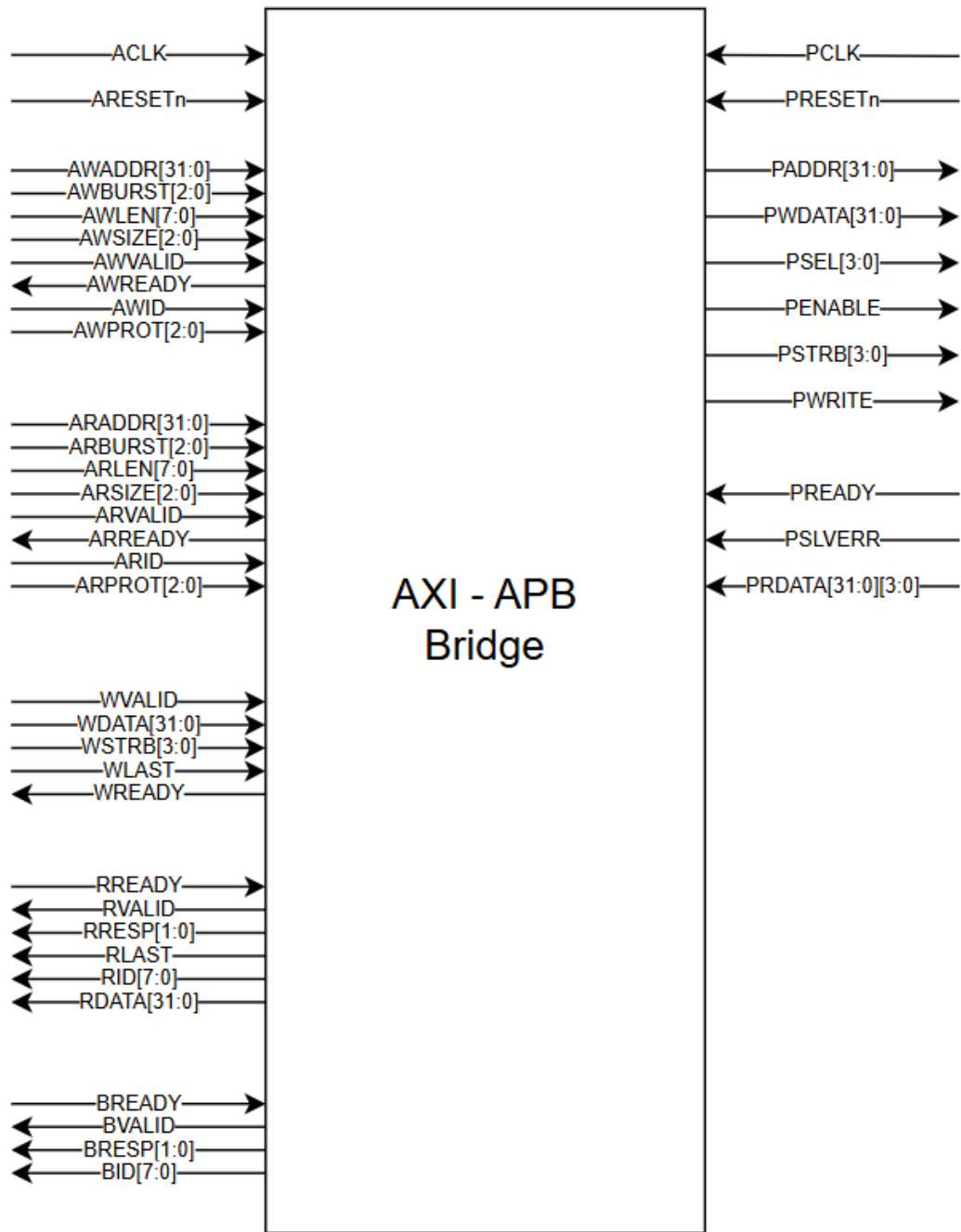


Hình 3.1 Sơ đồ khối tổng quát của cầu nối AXI-APB

Chức năng của từng khối trong thiết kế là:

- Các khối FIFO: nhiệm vụ đồng bộ dữ liệu, tránh bị mất gói tin giữa 2 bên do AXI và APB chạy 2 tần số khác nhau. Thiết kế 4 FIFO này nhận tín hiệu từ các kênh khác nhau phía AXI. Chức năng của từng khối FIFO là:
 - AW_FIFO: là FIFO dùng để lưu thông tin địa chỉ cùng các tín hiệu điều khiển khác trên kênh địa chỉ ghi của FIFO.
 - AR_FIFO: là FIFO dùng để lưu thông tin địa chỉ cùng các tín hiệu điều khiển khác trên kênh địa chỉ đọc của FIFO.
 - W_FIFO: Là FIFO dùng để lưu dữ liệu ghi từ phía AXI và tín hiệu điều khiển
 - R_FIFO: là FIFO dùng để lưu dữ liệu đọc và các tín hiệu phản hồi của giao dịch đọc. Mỗi gói tin được cần phải đi kèm thông tin phản hồi. Vì thế nên tín hiệu phản hồi và dữ liệu đọc sẽ được đi cùng 1 FIFO.
- Khối B_CH: Khối này đại diện cho kênh B của kênh AXI, chỉ là các tín hiệu phản hồi dựa trên dữ liệu ghi và truy cập ghi.
- Khối APB MASTER: Khối này khối trung gian giữa phía AXI và APB, có chức năng giải mã dữ liệu, địa chỉ dựa trên thông tin điều khiển được cung cấp ở phía AXI để trở thành dạng tín hiệu giao tiếp của phía APB.
- Khối WRITE/READ DECODER: Khối này có chức năng phân xử khi có cả 2 giao dịch đọc, ghi được thực hiện cùng lúc. Khi đó bộ phân xử sẽ phân xử xem giao dịch nào sẽ được thực hiện.
- Khối APB REGISTER: Khối này như khối thanh ghi nội của phía APB, lưu giữ các địa chỉ đầu và địa chỉ cuối của các ngoại vi APB. Dựa vào địa chỉ được cung cấp ở phía AXI, APB MASTER sẽ gửi tín hiệu để lấy thông tin địa chỉ từ thanh ghi.

Dưới đây là sơ đồ tín hiệu của cầu nối AXI-APB:



Hình 3.2 Các tín hiệu đầu vào và đầu ra của cầu nối

Các chân tín hiệu của cầu nối AXI – APB sẽ được mô tả ở bảng 3.1 :

Bảng 3.1 Các chân tín hiệu của cầu AXI – APB

STT	Tín hiệu	Số bit	Phân loại	Mô tả
1	ACLK	1	Ngõ vào	Tín hiệu xung đồng hồ ở phía AXI
2	ARESETn	1	Ngõ vào	Tín hiệu reset tích cực thấp của AXI

Kênh địa chỉ ghi (AW)				
3	AWADDR	32	Ngõ vào	Tín hiệu thông tin địa chỉ ghi.
4	AWLEN	8	Ngõ vào	Tín hiệu điều khiển số lượng gói tin có trong giao dịch ghi.
5	AWSIZE	3	Ngõ vào	Tín hiệu điều khiển kích thước dữ liệu của gói tin trong giao dịch ghi.
6	AWBURST	2	Ngõ vào	Tín hiệu xác định kiểu địa chỉ tăng lên giữa các gói tin.
7	AWVALID	1	Ngõ vào	Tín hiệu valid thông báo cho phía slave rằng tín hiệu địa chỉ và thông tin điều khiển đã sẵn sàng gửi đi.
8	AWREADY	1	Ngõ ra	Tín hiệu ready thông báo cho phía master rằng slave sẵn sàng nhận địa chỉ và thông tin điều khiển.
9	AWID	8	Ngõ vào	Tín hiệu chứa ID của giao dịch ghi.
10	AWPROT	3	Ngõ vào	Tín hiệu chứa thông tin định nghĩa giao dịch ghi.
Kênh địa chỉ đọc (AR)				
11	ARADDR	32	Ngõ vào	Tín hiệu thông tin địa chỉ đọc.
12	ARLEN	8	Ngõ vào	Tín hiệu điều khiển số lượng gói tin có trong giao dịch đọc.
13	ARSIZE	3	Ngõ vào	Tín hiệu điều khiển kích thước dữ liệu của gói tin trong giao dịch đọc.
14	ARBURST	2	Ngõ vào	Tín hiệu xác định kiểu địa chỉ tăng lên giữa các gói tin.
15	ARVALID	1	Ngõ vào	Tín hiệu valid thông báo cho phía slave rằng tín hiệu địa chỉ và thông tin điều khiển đã sẵn sàng gửi đi.
16	ARREADY	1	Ngõ ra	Tín hiệu ready thông báo cho phía master rằng slave sẵn sàng nhận địa chỉ và thông tin điều khiển.
17	ARID	8	Ngõ vào	Tín hiệu chứa ID của giao dịch đọc.
18	ARPROT	3	Ngõ vào	Tín hiệu chứa thông tin định nghĩa giao dịch đọc.
Kênh dữ liệu ghi (W)				

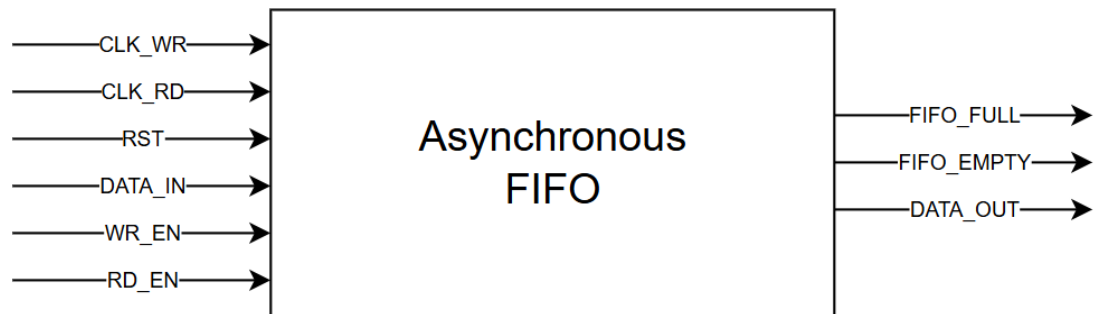
19	WVALID	1	Ngõ vào	Tín hiệu valid của kênh W, thông báo cho slave đến rằng dữ liệu từ master sẵn sàng được truyền.
20	WREADY	1	Ngõ ra	Tín hiệu ready của kênh W, thông báo rằng slave đến sẵn sàng nhận dữ liệu từ master.
21	WDATA	32	Ngõ vào	Tín hiệu dữ liệu ghi.
22	WSTRB	4	Ngõ vào	Tín hiệu mặt nạ dữ liệu ở kênh W trong một lần truyền.
23	WLAST	1	Ngõ vào	Tín hiệu thông báo rằng đây là gói tin cuối cùng trong một giao dịch ghi.
Kênh dữ liệu đọc (W)				
24	RVALID	1	Ngõ ra	Tín hiệu valid của kênh R, thông báo cho slave đến rằng dữ liệu đọc mà master muốn đọc sẵn sàng được truyền.
25	RREADY	1	Ngõ vào	Tín hiệu ready của kênh R, thông báo rằng slave đến sẵn sàng gửi dữ liệu đọc cho master.
26	RDATA	32	Ngõ ra	Tín hiệu dữ liệu đọc.
27	RRESP	4	Ngõ ra	Tín hiệu phản hồi của những gói tin đọc. Dựa vào đây ta xác định được dữ liệu đọc có lỗi hay không.
28	RLAST	1	Ngõ ra	Tín hiệu thông báo rằng đây là gói tin cuối cùng trong một giao dịch đọc.
29	RID	8	Ngõ ra	Tín hiệu ID của giao dịch đọc để xác định xem dữ liệu đọc thuộc giao dịch nào
Kênh phản hồi ghi (B)				
30	BVALID	1	Ngõ ra	Tín hiệu valid kênh B. Thông báo rằng slave đến sẵn sàng để truyền thông tin phản hồi về giao dịch ghi.
31	BREADY	1	Ngõ vào	Tín hiệu ready kênh B. Thông báo rằng master đã sẵn sàng nhận thông tin phản hồi của giao dịch ghi từ slave.
32	BRESP	2	Ngõ ra	Tín hiệu thông báo trạng thái của giao dịch ghi.

33	BID	8	Ngõ ra	Tín hiệu ID của giao dịch ghi. Giúp master phân biệt đây là phản hồi của giao dịch ghi nào.
Tín hiệu giao thức APB				
34	PCLK	1	Ngõ vào	Tín hiệu xung đồng hồ của giao thức APB.
35	PRESETn	1	Ngõ vào	Tín hiệu reset tích cực thấp của APB.
36	PADDR	32	Ngõ ra	Tín hiệu thông tin địa chỉ của APB.
37	PWDATA	32	Ngõ ra	Tín hiệu dữ liệu ghi. Đường tín hiệu này sẽ được sử dụng khi có dữ liệu cần ghi vào phía APB.
38	PSEL	4	Ngõ ra	Tín hiệu lựa chọn ngoại vi APB. Mỗi tín hiệu PSEL sẽ tới 1 ngoại vi APB duy nhất.
39	PENABLE	1	Ngõ ra	Tín hiệu cho phép, tín hiệu trong chu kỳ thứ 2 của tín hiệu giao tiếp APB
40	PSTRB	4	Ngõ ra	Tín hiệu mặt nạ dữ liệu ở phía APB.
41	PWRITE	1	Ngõ ra	Tín hiệu đọc hoặc ghi. Mức cao sẽ là ghi, mức thấp là đọc.
42	PREADY	1	Ngõ vào	Tín hiệu thông báo rằng slave sẵn sàng nhận dữ liệu.
43	PSLVERR	1	Ngõ vào	Tín hiệu thông báo lỗi nhận từ ngoại vi APB.
44	PRDATA	4:32	Ngõ vào	Tín hiệu dữ liệu đọc nhận từ ngoại vi APB. Tới 4 đường PRDATA cho 4 ngoại vi APB
45	PPROT	3	Ngõ ra	Tín hiệu xác định quyền truy cập của giao dịch.

3.2.2 Thiết kế khối FIFO

Do phía AXI và phía APB sử dụng 2 tần số khác nhau, vì thế thiết kế của FIFO sẽ là FIFO không đồng bộ (Asynchronous FIFO) để đồng bộ dữ liệu giữa hai miền xung nhịp khác nhau. Để tối ưu tài nguyên phần cứng tổng thể và đảm bảo tính linh hoạt, thiết kế của FIFO được xây dựng theo kiến trúc mô-đun, cho phép tùy biến độ rộng dữ liệu phù hợp với từng kênh chức năng trong chuẩn AXI. Khối FIFO sẽ sử dụng hai con trỏ đọc và ghi nhằm đảm bảo độ tin cậy trong quá trình

đồng bộ hóa. Ngoài ra, các tín hiệu trạng thái như đầy (full) và rỗng (empty) cũng được xử lý tránh lỗi tràn hoặc mất dữ liệu khi truyền thông tin. Mô hình FIFO bất đồng bộ được thể hiện ở hình 3.3:

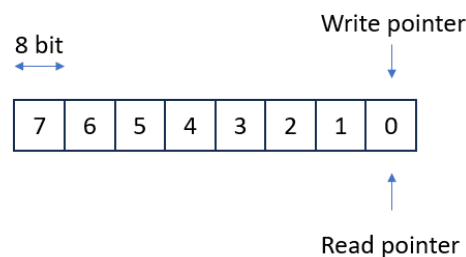


Hình 3.3 Mô hình FIFO bất đồng bộ

Các tín hiệu của FIFO có chức năng như sau:

- CLK_WR: xung đồng hồ đi vào cho tín hiệu ghi dữ liệu.
- CLK_RD: xung đồng hồ đi vào cho tín hiệu đọc dữ liệu.
- RST: tín hiệu reset FIFO.
- WR_EN: Tín hiệu cho phép ghi vào FIFO.
- RD_EN: Tín hiệu cho phép đọc từ FIFO.
- DATA_IN: Đường dữ liệu đi vào FIFO.
- FIFO_FULL: Tín hiệu thông báo rằng FIFO đang đầy, không được ghi vào.
- FIFO_EMPTY: Tín hiệu thông báo rằng FIFO đang trống, không được đọc ra.
- DATA_OUT: Đường dữ liệu ra FIFO.

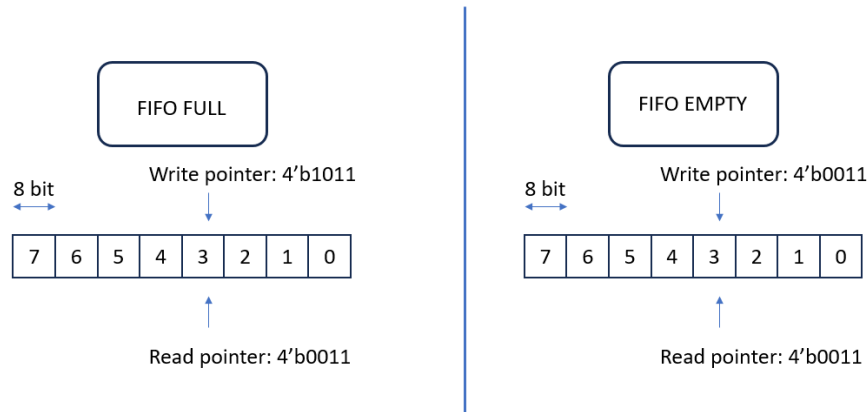
Hình 3.4 dưới đây là một ví dụ cho một hoạt động của một FIFO với độ rộng mỗi ô là 8 bit:



Hình 3.4 Hoạt động của một FIFO độ rộng mỗi ô là 8 bit

Như hình 3.4 mô tả, khi bắt đầu, con trỏ ghi (write pointer) và con trỏ đọc (read pointer), khi có tín hiệu cho phép ghi (write enable) thì con trỏ ghi được tăng 1. Tương tự, con trỏ đọc được tăng 1 khi có tín hiệu cho phép đọc (read enable).

Cơ chế hoạt động của tín hiệu FIFO_FULL và FIFO_EMPTY sẽ được thể hiện ở hình 3.5:



Hình 3.5 Mô tả hoạt động của FIFO_FULL và FIFO_EMPTY

Khi con trỏ đọc trùng vị trí với con trỏ ghi thì sẽ có 2 tình huống như hình 3.5:

- Nếu con trỏ ghi (4'b1011) đã đi được một vòng FIFO so với con trỏ đọc (4'b0011): lúc này FIFO sẽ kích hoạt tín hiệu FIFO_FULL lên 1 để báo hiệu FIFO đã đầy.
- Nếu con trỏ ghi và con trỏ đọc không có chênh lệch về số vòng FIFO đã đi được (đều là 4'b0011): FIFO kích hoạt tín hiệu FIFO_EMPTY lên 1 để thông báo FIFO đang trống.

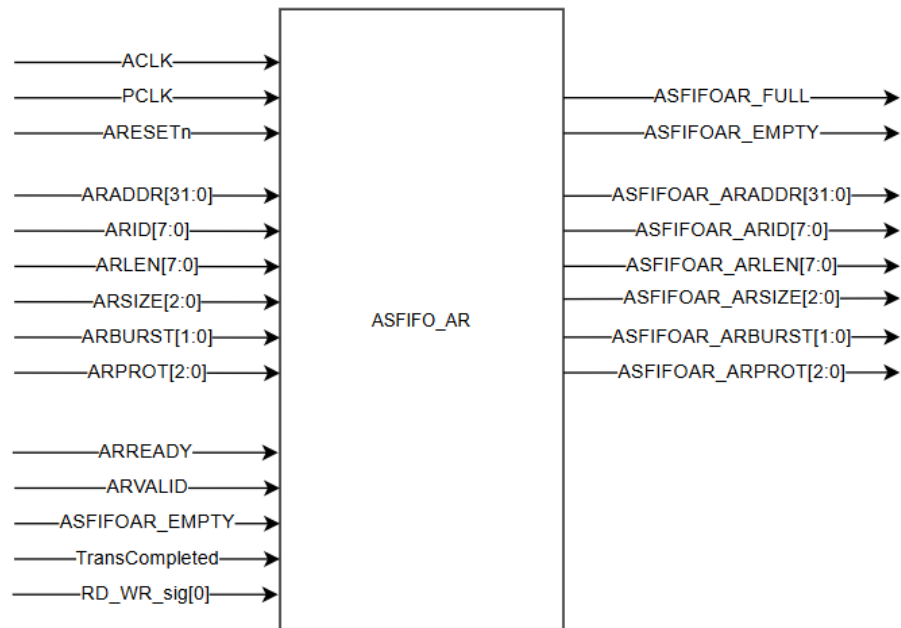
Khi thiết kế FIFO cho các kênh AR, AW, R, W, độ rộng dữ liệu được thiết lập sẽ khác nhau giữa các kênh:

- Kênh AR: Dữ liệu gồm ARADDR[31:0], ARID[7:0], ARLEN[7:0], ARSIZE[2:0], ARBURST[1:0], ARPROT[2:0]. Như vậy tổng độ rộng bit cần thiết lập cho kênh AR là: $32+8+8+3+2+3=56$ bit
- Kênh AW: Dữ liệu gồm AWADDR[31:0], AWID[7:0], AWLEN[7:0], AWSIZE[2:0], AWBURST[1:0], AWPROT[2:0]. Như vậy tổng độ rộng bit cần thiết lập cho kênh AW là: $32+8+8+3+2+3=56$ bit

- Đối với kênh R: Dữ liệu bao gồm PRDATA[31:0], PRESP_INT[1:0], ASFIFOAR_ARID[7:0], RLAST_INT. Như vậy tổng độ rộng bit cần thiết lập cho kênh R là: $32+2+8+1=43$ bit
- Đối với kênh W: Dữ liệu bao gồm WDATA[31:0], WSTRB[7:0], Như vậy tổng độ rộng bit cần thiết lập cho kênh W là: $32+4=36$ bit

3.2.2.1 Thiết kế khối FIFO_AR

Hình 3.6 mô tả sơ đồ kết nối của khối ASFIFO_AR



Hình 3.6 Sơ đồ kết nối của khối ASFIFO_AR

Chức năng của các tín hiệu của khối ASFIFO_AR trong hình 3.6:

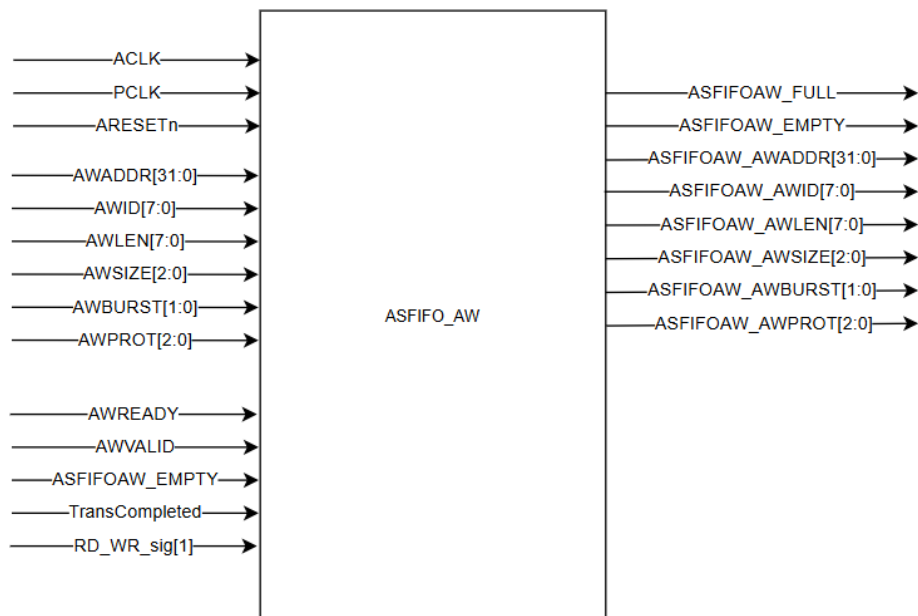
- Tín hiệu cho phép ghi (tín hiệu WR_EN ở hình 3.3) vào FIFO sẽ phụ thuộc 2 tín hiệu bắt tay ARREADY và ARVALID, chỉ khi có cả 2 tín hiệu này quá trình bắt tay xảy ra, khi đó dữ liệu sẽ được truyền vào FIFO_AR.
- Tín hiệu cho phép đọc (tín hiệu RD_EN ở hình 3.3) từ FIFO sẽ phụ thuộc vào việc FIFO_AR không được rỗng, đồng thời phải được khối WRITE/READ DECODER cấp quyền đọc và tín hiệu TransCompleted, thông báo rằng đã có 1 truy nhập được hoàn thành.

- Dữ liệu đầu vào (DATA_IN ở hình 3.3) sẽ là các tín hiệu điều khiển ở kênh AR gồm có ARADDR, ARID, ARLEN, ARSIZE, ARBURST, ARPROT được ghép lại. Độ dài của tín hiệu DATA_IN sẽ bằng tổng độ dài của các đường tín hiệu.
- Dữ liệu đầu ra (DATA_OUT ở hình 3.3) sẽ là ngõ ra của FIFO, giá trị được lấy từ DATA_IN. Độ dài của tín hiệu DATA_OUT sẽ bằng tổng độ dài của các đường tín hiệu trước đó.
- ASFIFOAR_FULL: Tín hiệu thông báo rằng ASFIFO_AR đang đầy,
- ASFIFOAR_EMPTY: Tín hiệu thông báo rằng ASFIFO_AR đang trống.

Xung clock của AXI sẽ được dùng để làm CLK_WR cho ASFIFO_AR vì phía AXI lúc này đang ghi vào ASFIFO_AR. Còn xung clock của APB sẽ được dùng để làm xung CLK_RD cho FIFO vì hiện tại phía APB đang muốn đọc những giá trị trong ASFIFO_AR để xử lý.

3.2.2.2 Thiết kế khối FIFO_AW

Hình 3.7 mô tả sơ đồ kết nối của khối ASFIFO AW



Hình 3.7 Sơ đồ kết nối của khối ASFIFO_AW

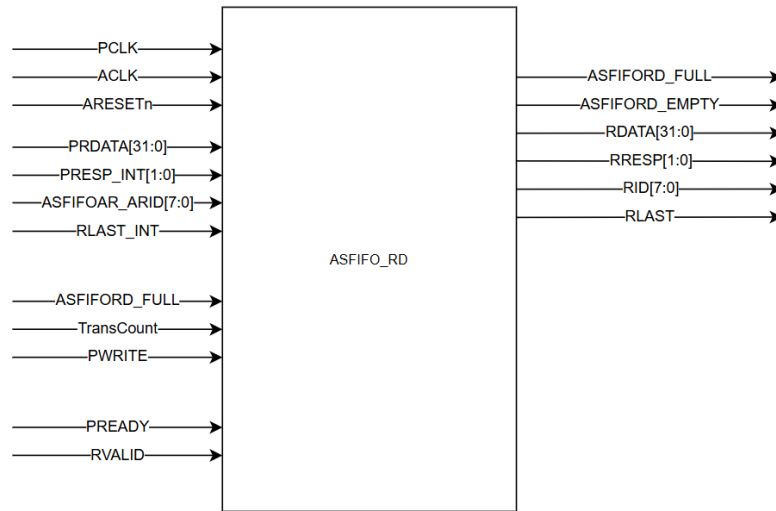
Chức năng của các tín hiệu của khối ASFIFO_AW trong hình 3.7:

- ASFIFOAW_WE: tín hiệu cho phép ghi vào FIFO, sẽ phụ thuộc 2 tín hiệu bắt tay AWREADY và AWVALID, chỉ khi có cả 2 tín hiệu này quá trình bắt tay xảy ra, khi đó dữ liệu sẽ được truyền vào FIFO_AW.
- ASFIFOAW_RE: tín hiệu cho phép đọc vào FIFO, tín hiệu phụ thuộc vào việc FIFO_AW không được rỗng, đồng thời phải được khối WRITE/READ DECODER cấp quyền ghi và tín hiệu TransCompleted, thông báo rằng đã có 1 truy nhập được hoàn thành.
- DATA_IN: sẽ là các tín hiệu điều khiển ở kênh AR gồm có AWADDR, AWID, AWLEN, AWSIZE, AWBURST, AWPROT được ghép lại. Độ dài của tín hiệu DATA_IN sẽ bằng tổng độ dài của các đường tín hiệu.
- DATA_OUT: sẽ là ngõ ra của FIFO, giá trị được lấy từ DATA_IN. Độ dài của tín hiệu DATA_OUT sẽ bằng tổng độ dài của các đường tín hiệu trước đó.
- ASFIFOAW_FULL: Tín hiệu thông báo rằng ASFIFO_AW đang đầy,
- ASFIFOAW_EMPTY: Tín hiệu thông báo rằng ASFIFO_AW đang trống.

Xung clock của AXI sẽ được dùng để làm CLK_WR cho ASFIFO_AR vì phía AXI lúc này đang ghi vào ASFIFO_AW. Còn xung clock của APB sẽ được dùng để làm xung CLK_RW cho FIFO vì hiện tại phía APB đang muốn đọc những giá trị trong ASFIFO_AW để xử lý.

3.2.2.3 Thiết kế khối FIFO_R

Hình 3.8 mô tả sơ đồ kết nối của khối ASFIFO_RD



Hình 3.8 Sơ đồ kết nối của khối ASFIFO_RD

Chức năng của các tín hiệu của khối ASFIFO_RD trong hình 3.8:

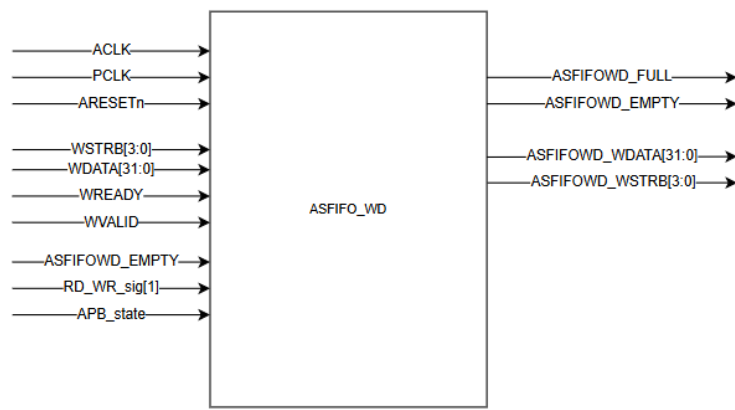
- Tín hiệu cho phép ghi vào FIFO (tín hiệu WR_EN ở hình 3.3) sẽ phụ thuộc vào việc FIFO_RD có đầy hay không, đi kèm phải có tín hiệu TransCount, tín hiệu này là tín hiệu cho phép việc đếm các gói tin truyền vào kênh FIFO_RD, đồng thời giúp xác định cho tín hiệu RLAST. Và phải có tín hiệu ~PWRITE là đang truy cập đọc từ APB, dữ liệu đọc được từ APB sẽ được vào FIFO_RD.
- Tín hiệu cho phép đọc vào FIFO (tín hiệu RD_EN ở hình 3.3) sẽ phụ thuộc vào hai tín hiệu bắt tay RREADY và RVALID, để có thể đọc từ FIFO ra phía AXI.
- Dữ liệu ngõ vào (tín hiệu DATA_IN ở hình 3.3) sẽ là các tín hiệu điều khiển ở kênh RD gồm có PRDATA, RRESP_INT, ASFIFOAR_ARID, RLAST_INT được ghép lại. Độ dài của tín hiệu DATA_IN sẽ bằng tổng độ dài của các đường tín hiệu.
- Dữ liệu ngõ ra của FIFO (tín hiệu DATA_OUT ở hình 3.3) là giá trị được lấy từ DATA_IN. Độ dài của tín hiệu DATA_OUT sẽ bằng tổng độ dài của các đường tín hiệu trước đó.
- ASFIFORD_FULL: Tín hiệu thông báo rằng ASFIFO_RD đang đầy,

- ASFIFORD_EMPTY: Tín hiệu thông báo rằng ASFIFO_RD đang trống.

Xung clock của APB sẽ được dùng để làm CLK_WR cho ASFIFO_RD vì phía APB lúc này đang gửi những dữ liệu đọc đến ASFIFO, tức là ghi vào ASFIFO_AR. Còn xung clock của AXI sẽ được dùng để làm xung CLK_RD cho FIFO vì hiện tại phía AXI đang muốn đọc những giá trị được phía APB ghi vào.

3.2.2.4 Thiết kế khối FIFO_W

Hình 3.9 mô tả sơ đồ kết nối của khối ASFIFO_WD:



Hình 3.9 Sơ đồ kết nối của khối ASFIFO_WD

Chức năng của các tín hiệu của khối ASFIFO_WD trong hình 3.9:

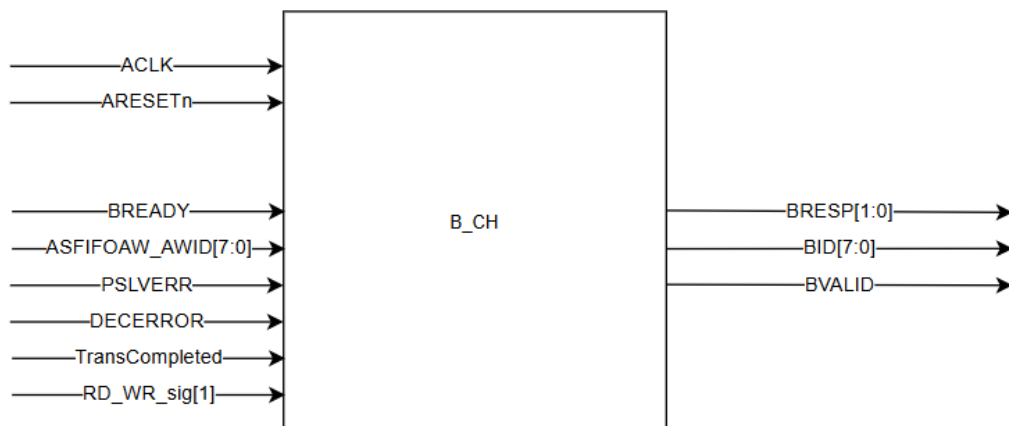
- Tín hiệu cho phép ghi vào FIFO (tín hiệu WR_EN ở hình 3.3) sẽ phụ thuộc 2 tín hiệu bắt tay WREADY và WVALID, chỉ khi có cả 2 tín hiệu này quá trình bắt tay xảy ra, khi đó dữ liệu từ phía AXI sẽ được truyền vào FIFO_WD.
- Tín hiệu cho phép đọc vào FIFO_WD (tín hiệu RD_EN ở hình 3.3) phải đảm bảo rằng FIFO không được rỗng, và phải đang được cấp phép cho truy cập ghi vì đây là FIFO cho dữ liệu ghi, kèm theo đó phải có trạng thái ACCESS từ phía APB vì trạng thái ACCESS là trạng thái truy cập vào APB.
- Dữ liệu ngõ vào (tín hiệu DATA_IN ở hình 3.3) sẽ là các tín hiệu điều khiển ở kênh RD gồm có WDATA, WSTRB được ghép lại. Độ dài của tín hiệu DATA_IN sẽ bằng tổng độ dài của các đường tín hiệu.

- Dữ liệu ngõ ra của FIFO_WD (tín hiệu DATA_OUT ở hình 3.3) giá trị được lấy từ DATA_IN. Độ dài của tín hiệu DATA_OUT sẽ bằng tổng độ dài của các đường tín hiệu trước đó.
- ASFIFOWD_FULL: Tín hiệu thông báo rằng ASFIFO_WD đang đầy,
- ASFIFOWD_EMPTY: Tín hiệu thông báo rằng ASFIFO_WD đang trống.

Xung clock của AXI sẽ được dùng để làm CLK_WR cho ASFIFO_WD vì phía AXI lúc này đang ghi vào ASFIFO_WD. Còn xung clock của APB sẽ được dùng để làm xung CLK_RD vì hiện tại phía APB đang muốn đọc những giá trị trong ASFIFO_WD để xử lý.

3.2.3 Thiết kế khối B_CH

Hình 3.10 mô tả sơ đồ kết nối của kênh B:



Hình 3.10 Sơ đồ kết nối của khối B_CH

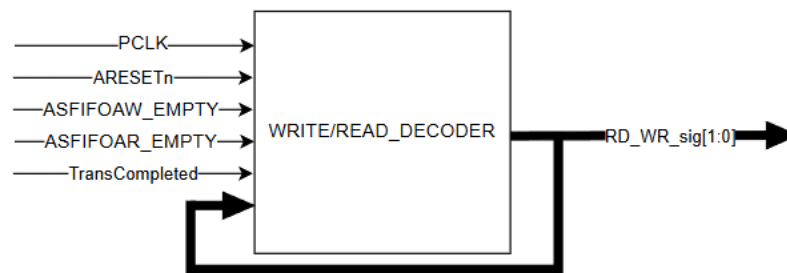
Chức năng của tín hiệu trong khối B_CH trong hình 3.10:

- BRESP[1:0]: Được quyết định bởi giá trị tín hiệu đầu vào PSLVERR và DECERROR. Nếu PSLVERR có giá trị 1 thì BRESP có giá trị là 2'b10, còn nếu DECERROR có giá trị 1 thì BRESP có giá trị là 2'01. Nếu không có lỗi thì BRESP mặc định là 2'b00.
- BID[7:0]: Nếu hoàn tất quá trình ghi thì BID có giá trị bằng với giá trị của ASFIFOAW_AWID

- BVALID: Nếu hoàn tất quá trình ghi thì BVALID có giá trị là 1 trong 1 chu kỳ xung nhịp

3.2.4 Thiết kế khối Write/Read Decoder

Khối Write/Read Decoder đóng vai trò quan trọng trong cả hệ thống, khối này giúp xác định phiên giao dịch hiện tại và giao dịch tiếp theo, phân xử giữa giao dịch ghi và giao dịch đọc. Sơ đồ khối kết nối của khối Write/Read Decoder được mô tả ở hình 3.11:



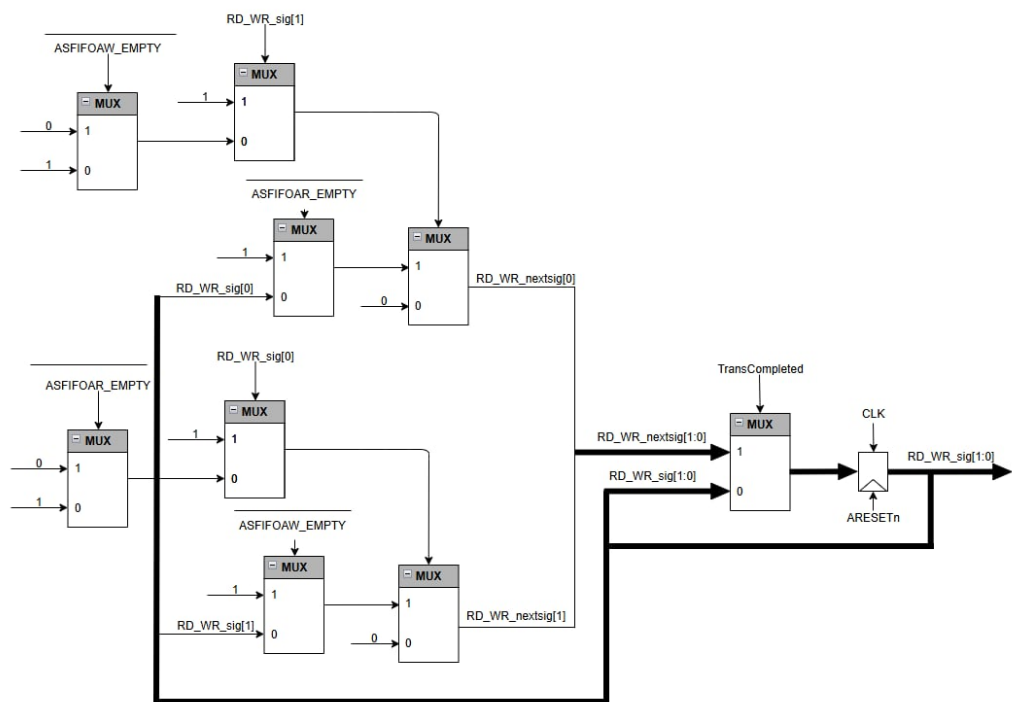
Hình 3.11 Sơ đồ khối Write/Read Decoder.

Chức năng của các tín hiệu của khối Write/Read Decoder ở hình 3.11 là:

- PCLK: Tín hiệu xung vuông của phía APB, để giúp cập nhật trạng thái để xác định phiên giao dịch. Việc sử dụng PCLK mà không phải ACLK là vì có tín hiệu TransCompleted, TransCompleted là tín hiệu xác định đã hoàn thành xong một giao dịch, mà tín hiệu này phụ thuộc vào những việc đếm số lượng gói tin đi qua kênh APB mà kênh APB sử dụng xung clock PCLK nên vì thế khi cập nhật giao dịch mới phải sử dụng xung PCLK.
- ARESETn: Tín hiệu Reset của hệ thống.
- ASFIFOAW_EMPTY: Tín hiệu báo rằng hiệu tại FIFO_AW rỗng, ta cần biết rằng có dữ liệu trong FIFO_AW hay không, để báo hiệu rằng đang có thông tin giao dịch ghi cần được xử lý ở kênh AW.
- ASFIFOAR_EMPTY: Tín hiệu báo rằng hiệu tại FIFO_AR rỗng, ta cần biết rằng có dữ liệu trong FIFO_AR hay không, để báo hiệu rằng đang có thông tin giao dịch đọc cần được xử lý ở kênh AR.

- RD_WR_sig[1:0]: là tín hiệu phân xử giao dịch đọc và ghi. RD_WR_sig[0] là tín hiệu phân xử giao dịch đọc còn RD_WR_sig[1] là tín hiệu phân xử giao dịch ghi. Khi tín hiệu ở mức 1 thì đang cấp phép cho giao dịch đó, còn khi ở mức 0 thì không cấp phép. RD_WR_sig[1:0] chỉ có 2 trạng thái duy nhất là 01 và 10, là chỉ 1 trong 2 giao dịch được cấp phép, chứ không thể có trạng thái cả hai cùng cấp phép, vì trong ngoại vi APB, chỉ có thể đọc hoặc ghi chứ không thể cả hai.

Hình 3.12 dưới đây mô tả chi tiết mạch của khối write/read decoder:

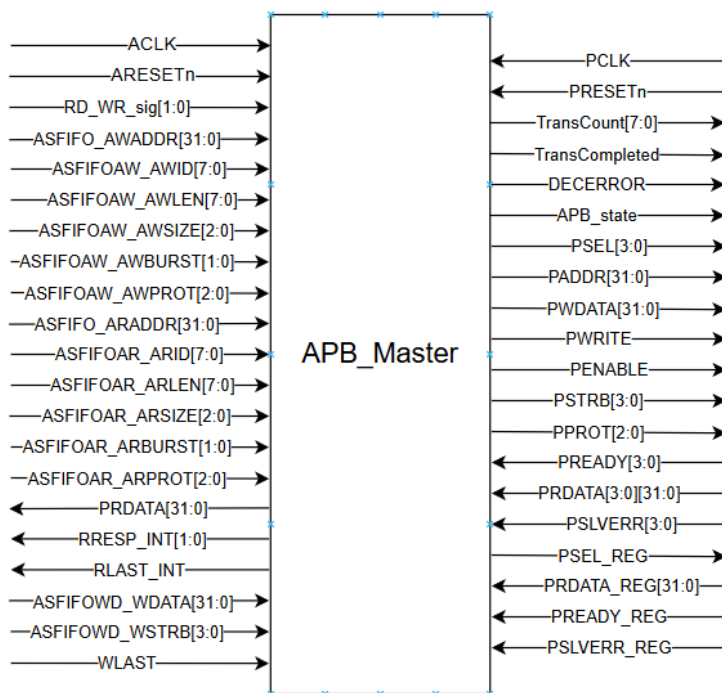


Hình 3.12 Mô tả chi tiết mạch của khối write/read decoder

Theo mô tả từ hình 3.12, để có thể cấp phép cho một giao dịch, trước hết ta phải cần xem tại FIFO_AW và FIFO_AR có thông tin hay không, nếu có tin tức thì có nghĩa là phía AXI đang muốn yêu cầu một giao dịch với phía APB. Tiếp theo khối này sẽ xem xét hiện tại đang cấp quyền cho giao dịch nào, để có thể phân xử cho giao dịch khác dựa trên tín hiệu TransCompleted. Nếu cả 2 FIFO_AR và FIFO_AW có thông tin, hệ thống sẽ phân xử luân phiên cả 2. Nếu chỉ có 1 trong 2 FIFO_AW/AR có thông tin, tín hiệu cấp quyền sẽ được giữ ở tại giao dịch đó.

3.2.5 Thiết kế khối APB Master

Khối APB Master là thành phần cốt lõi cầu nối AXI – APB, khối này là nơi xử lý các tín hiệu điều khiển và dữ liệu từ các kênh FIFO, sau đó chuyển dữ liệu và thông tin điều khiển theo giao thức AXI sang dữ liệu và thông tin điều khiển theo giao thức APB, nhằm giao tiếp được với các ngoại vi APB. Sơ đồ khối kết nối của khối APB Master như hình 3.13:



Hình 3.13 Sơ đồ kết nối khối APB_Master

Bảng mô tả chức năng của các tín hiệu sẽ được thể hiện ở bảng 3.2:

Bảng 3.2: Bảng mô tả chức năng của các tín hiệu khối APB_MASTER

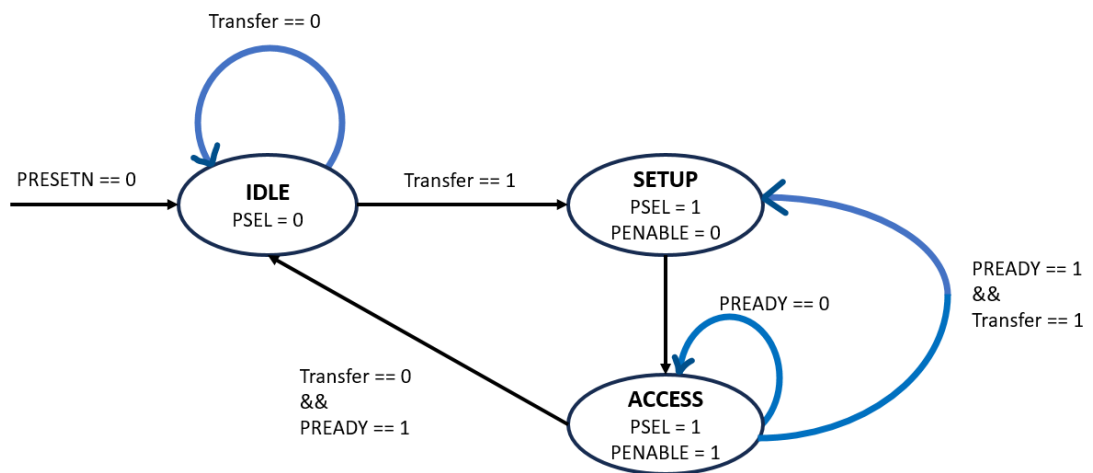
STT	Tín hiệu	Số bit	Phân loại	Mô tả
1	ACLK	1	Ngõ vào	Tín hiệu xung đồng hồ ở phía AXI
2	ARESETn	1	Ngõ vào	Tín hiệu reset tích cực thấp của AXI
3	RD_WR_sig	2	Ngõ vào	Tín hiệu phân quyền giao dịch từ Write/Read Decoder
4	ASFIFO_AWADDR	32	Ngõ vào	Đường địa chỉ kênh AW từ FIFO_AW
5	ASFIFO_AWID	8	Ngõ vào	Tín hiệu ID của giao dịch ghi kênh AW từ FIFO_AW

6	ASFIFO_AWLEN	8	Ngõ vào	Tín hiệu độ dài khung truyền của giao dịch từ FIFO_AW
7	ASFIFO_AWSIZE	3	Ngõ vào	Tín hiệu độ dài gói tin trong khung truyền từ FIFO_AW
8	ASFIFO_AWBURST	2	Ngõ vào	Tín hiệu xác định kiểu truyền từ FIFO_AW
9	ASFIFO_AWPROT	3	Ngõ vào	Tín hiệu xác định kiểu truy cập từ FIFO_AW
10	ASFIFO_ARADDDR	32	Ngõ vào	Đường địa chỉ kênh AR từ FIFO_AR
11	ASFIFO_ARID	8	Ngõ vào	Tín hiệu ID của giao dịch ghi kênh AR từ FIFO_AR
12	ASFIFO_ARLEN	8	Ngõ vào	Tín hiệu độ dài khung truyền của giao dịch từ FIFO_AR
13	ASFIFO_ARSIZE	3	Ngõ vào	Tín hiệu độ dài gói tin trong khung truyền từ FIFO_AR
14	ASFIFO_ARBURST	2	Ngõ vào	Tín hiệu xác định kiểu truyền từ FIFO_AR
15	ASFIFO_ARPROT	3	Ngõ vào	Tín hiệu xác định kiểu truy cập từ FIFO_AR
16	PRDATA	32	Ngõ ra	Đường dữ liệu ra của APB sau khi đã được xử lý
17	RRESP_INT	2	Ngõ ra	Tín hiệu phản hồi đọc sau khi được xử lý
18	RLAST_INT	1	Ngõ ra	Tín hiệu thông báo gói tin cuối sau khi được xử lý
19	ASFIFOWD_WDATA	32	Ngõ vào	Đường dữ liệu ghi từ kênh W sau khi qua FIFO_WD
20	ASFIFOWD_WSTRB	4	Ngõ vào	Tín hiệu mặt nạ dữ liệu ở kênh W ở FIFO_WD
21	WLAST	1	Ngõ vào	Tín hiệu thông báo gói tin cuối cùng trong giao dịch ghi
22	PCLK	1	Ngõ vào	Tín hiệu xung đồng hồ ở phía APB
23	PRESETn	1	Ngõ vào	Tín hiệu Reset ở phía APB
24	TransCount	1	Ngõ ra	Tín hiệu cho phép đếm số lượng gói tin
25	TransCompleted	1	Ngõ ra	Tín hiệu thông báo là đã kết thúc 1 giao dịch

26	DECERROR	1	Ngõ ra	Tín hiệu thông báo rằng lỗi trong quá trình giải mã
27	APB_state	2	Ngõ ra	Trạng thái của APB
28	PSEL	4	Ngõ ra	Tín hiệu lựa chọn ngoại vi APB. Mỗi tín hiệu PSEL sẽ tới 1 ngoại vi APB duy nhất.
29	PADDR	32	Ngõ ra	Tín hiệu thông tin địa chỉ của APB.
30	PWDATA	32	Ngõ ra	Tín hiệu dữ liệu ghi. Đường tín hiệu này sẽ được sử dụng khi có dữ liệu cần ghi vào phía APB.
31	PWRITE	1	Ngõ ra	Tín hiệu đọc hoặc ghi. Mức cao sẽ là ghi, mức thấp là đọc.
32	PENABLE	1	Ngõ ra	Tín hiệu cho phép, tín hiệu trong chu kỳ thứ 2 của tín hiệu giao tiếp APB
33	PSTRB	4	Ngõ ra	Tín hiệu mặt nạ dữ liệu phía APB.
34	PPROT	3	Ngõ ra	Tín hiệu xác định quyền truy cập của giao dịch.
35	PREADY	4	Ngõ vào	Tín hiệu thông báo rằng slave sẵn sàng truyền.
36	PRDATA	32[3:0]	Ngõ vào	Tín hiệu dữ liệu đọc nhận từ ngoại vi APB. Tới 4 đường PRDATA cho 4 ngoại vi APB
37	PSLVERR	4	Ngõ vào	Tín hiệu thông báo ngoại vi APB bị lỗi
38	PSEL_REG	1	Ngõ ra	Tín hiệu lựa chọn APB_Register
39	PRDATA_REG	1	Ngõ vào	Đường dữ liệu của APB_Register
40	PREADY_REG	1	Ngõ vào	Tín hiệu thông báo APB_Register sẵn sàng truyền
41	PSLVERR_REG	1	Ngõ vào	Tín hiệu thông báo APB_Register bị lỗi

Như mô tả trong bảng 3.2, khối APB_Master sẽ tiếp nhận địa chỉ và thông tin điều khiển để xử lý. Công việc APB_Master sẽ xử lý bao gồm có: Chuyển đổi những dữ liệu theo giao thức AXI thành giao thức APB, tính toán số địa chỉ tăng lên dựa vào địa chỉ dựa vào địa chỉ đầu tiên AxADDR và thông tin AxLen, AxSIZE, AxBURST; đếm số lượng những gói tin được truyền qua kênh để phục vụ khối phân xử cho các giao dịch; đóng vai trò là nguồn ở phía APB, điều khiển các tín hiệu giao tiếp đến ngoại vi APB; xử lý các trường hợp quá trình giải mã bị lỗi hoặc ngoại vi APB bị lỗi để phản hồi lại kênh tương ứng ở AXI.

Hình 3.14 dưới đây mô tả về máy trạng thái (state machine) của khối APB Master được tham khảo từ sơ đồ máy trạng thái theo chuẩn APB từ ARM:



Hình 3.14 Sơ đồ máy trạng thái FSM của khối APB Master

Như hình 3.14 mô tả thì khối APB Master có 3 trạng thái chính bao gồm IDLE, SETUP và ACCESS. Mặc định lúc bắt đầu hoặc khi được reset thì khối APB Master sẽ được đưa về trạng thái IDLE. Dưới đây là mô tả cho từng trạng thái:

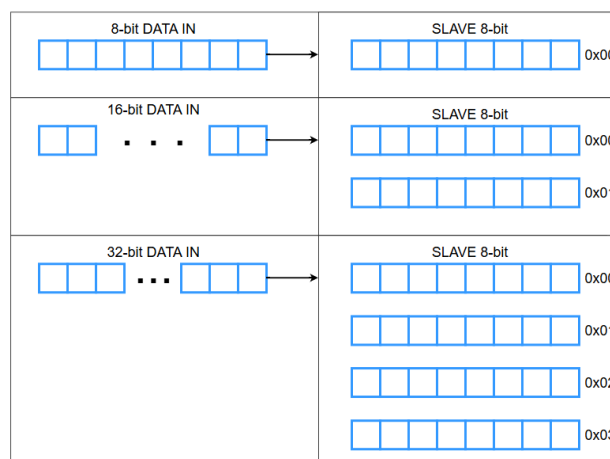
- Trạng thái IDLE: Đây là trạng thái ban đầu không có bất cứ yêu cầu truyền nhận dữ liệu nào. Trong trạng thái IDLE, các dữ liệu bao gồm PSEL, PENABLE, PWRITE, PADDR, PWDATA, PRDATA được đưa xuống giá trị '0' (giá trị mặc định). Điều kiện để chuyển trạng thái (IDLE → SETUP) là có yêu cầu đọc/ghi dữ liệu (transfer == 1).

- **Trạng thái SETUP:** Đây là trạng thái chuẩn bị thiết lập các giao dịch đọc/ghi. Tín hiệu PSEL được thiết lập ở giá trị '1' (chọn APB Slave tương ứng). Địa chỉ PADDR cũng được cập nhật từ địa chỉ phía AXI. PWRITE cũng sẽ được cập nhật dựa trên tín hiệu RD_WR_sig ở khối Write/Read Decoder. Nếu là giao dịch ghi (write transaction) thì PWDATA được cập nhật từ ASFIFOWD_WDDATA. Và PENABLE vẫn được giữ ở giá trị '0'. Sau một chu kỳ xung clock thì APB Master sẽ chuyển từ SETUP sang ACCESS.
- **Trạng thái ACCESS:** Đây là trạng thái thực hiện giao dịch với APB Slave. Tín hiệu PENABLE được thiết lập ở giá trị '1'. Bên cạnh đó APB Master cần chờ thêm tín hiệu PREADY thì phía APB Slave gửi đến để chắc chắn APB Slave sẵn sàng với giao dịch. Nếu PREADY có giá trị '0' thì APB Master sẽ vẫn tiếp tục chờ, nếu PREADY có giá trị là '1' thì tiến hành giao dịch đọc/ghi. Nếu vẫn còn giao dịch đang chờ (transfer == 1) thì khối APB Master sẽ quay lại trạng thái SETUP. Nếu không còn giao dịch thì APB Master sẽ quay lại trạng thái IDLE.

Trong quá trình truyền dữ liệu thì kiểu truyền (burst type) là yếu tố rất quan trọng. Trong mục này, nhóm sẽ tập trung về 2 kiểu truyền INCR và WRAP. Hai cơ chế truyền dữ liệu INCR và WRAP được phát triển nhằm phục vụ các mục tiêu thiết kế khác nhau của SoC, có thể thiên về hiệu suất và độ linh hoạt, tối ưu hóa tài nguyên phần cứng, giúp truyền các khối dữ liệu liên tiếp để đáp ứng các bộ nhớ liên tục như truy xuất mảng, bộ đệm, cache. Kỹ thuật tính toán tăng địa chỉ truyền thống của INCR và WRAP được đề cập trong tài liệu chuẩn của ARM. Tuy nhiên, việc tăng địa chỉ truyền thống có thể chỉ giới hạn trong một số trường hợp cụ thể, ví dụ như ngoại vi của APB sẽ mặc định sẽ có độ dài là 8-bit nên kỹ thuật tính toán được đề cập trong tài liệu ARM chỉ giải quyết được trường hợp này. Để giải quyết trường hợp này, nhóm đã đưa ra phương hướng để có thể khắc phục vấn đề nêu trên.

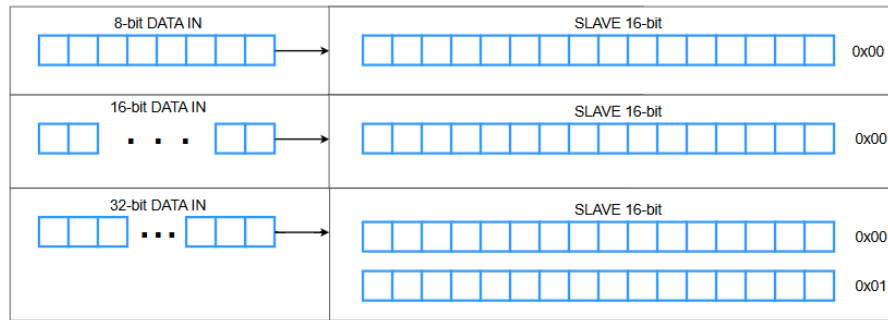
Với thiết kế được đề cập ở trong tài liệu về chuẩn giao tiếp AXI, địa chỉ tăng lên tiếp theo sẽ phụ thuộc vào dữ liệu của gói tin (tín hiệu AxSIZE), tín hiệu ở đây được tính bằng đơn vị bytes. Các ngoại vi sẽ đều được mặc định là ngoại vi 8-bit hoặc 1 bytes. Vì thế nếu tín hiệu AxSIZE trên kênh truyền thay đổi như thế nào thì địa chỉ tăng sau cũng sẽ bằng với tín hiệu AxSIZE. Ví dụ AxSIZE là 2 thì độ rộng của dữ liệu sẽ là 4-byte (đã được đề cập ở phần chương 2) tức là dữ liệu 32-bit, địa chỉ tự động sẽ tăng lên 4. Ở đây bước nhảy địa chỉ sẽ hoàn toàn phụ thuộc vào tín hiệu AxSIZE.

Ở thiết kế của nhóm, khi kênh truyền có gói tin có độ dài dữ liệu là 8-bit, 16-bit, 32-bit thì hệ thống phải tự điều chỉnh tăng địa chỉ làm sao để phù hợp với ngoại vi đang được chọn để truyền. Thiết kế này cho phép truyền các gói tin có độ dài lớn vào các ngoại vi có độ dài dữ liệu lớn, vẫn giữ cơ chế mặc định địa chỉ tăng 0x04.

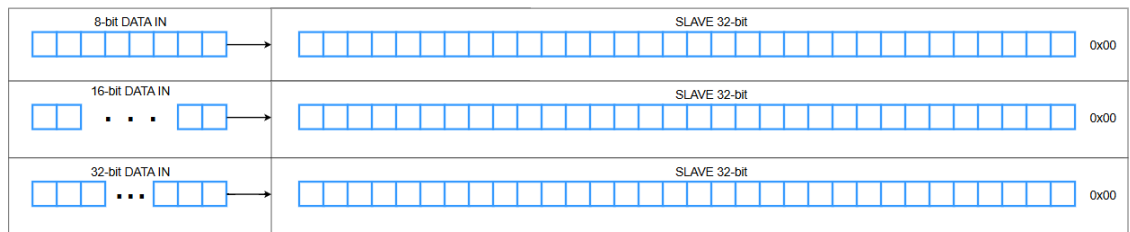


Hình 3.15 Địa chỉ tăng của ngoại vi 8-bit

Hình 3.15 trên minh họa biểu diễn cách tăng địa chỉ cố định chỉ dựa theo tín hiệu AxSIZE, phù hợp với ngoại vi 8-bit. Tuy nhiên khi sử dụng ngoại vi 16-bit và 32-bit, nếu địa chỉ cứ tăng lên theo AxSIZE thì sẽ gây ra hao phí ô nhớ trong ngoại vi. Hình 3.16 và hình 3.17 sẽ minh họa thiết kế tăng địa chỉ của nhóm khi sử dụng ngoại vi 16-bit và 32-bit.



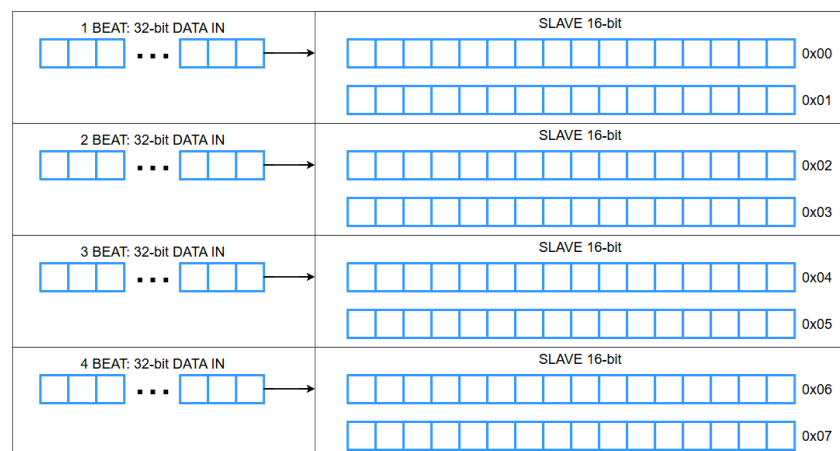
Hình 3.16 Địa chỉ tăng của ngoại vi 16-bit



Hình 3.17 Địa chỉ tăng của ngoại vi 32-bit

Hình 3.16 trên minh họa cách tăng địa chỉ của kiểu truyền INCR với ngoại vi có độ dài dữ liệu là 16-bit. Trường hợp này tương tự với ngoại vi 16-bit đã nêu trên. Với độ dài ô nhớ là 32-bit thì ngoại vi này có thể chứa hoàn toàn được tất cả 3 độ dài gói tin 8-bit, 16-bit, 32-bit.

Với trường hợp cụ thể khi truyền vào ngoại vi 16-bit như sau: $AxADDR = 0x00$, $AxSIZE = 2$, $AxLEN = 3$, $AxBURST = 1$. Ở kênh truyền sẽ có địa chỉ bắt đầu là 0x00, mỗi gói dữ liệu có độ dài 32-bit và có 4 gói tin, kiểu truyền là INCR. Với ô nhớ là 16-bit thì mỗi lần truyền một gói tin chỉ tăng 2 địa chỉ lên, từ 0x00 đến 0x02, tiếp theo là 0x04, 0x06.



Hình 3.18 Minh họa địa chỉ tăng ở ngoại vi 16-bit theo tín hiệu điều khiển

Như đã đề cập ở phần lý thuyết theo tài liệu ARM cơ chế WRAP sẽ thực hiện truyền dữ liệu trong một khoảng địa chỉ (Wrap Boundary Size) nhất định, địa chỉ của WRAP trong 1 burst sẽ quay lại điểm bắt đầu sau khi đạt được giá trị ngưỡng (Wrap Boundary). Giả sử, ta có một burst dữ liệu được truyền được có các thông số được đề cập ở bảng 3.3:

Bảng 3.3 Bảng thông số thiết lập cho burst truyền

AWADDR	AWLEN	AWSIZE	AWTYPE	APB Slave
0x1C	7	0x02	WRAP	8 bit

Dựa trên công thức được đề cập ở chương 2, ta có thể tính được:

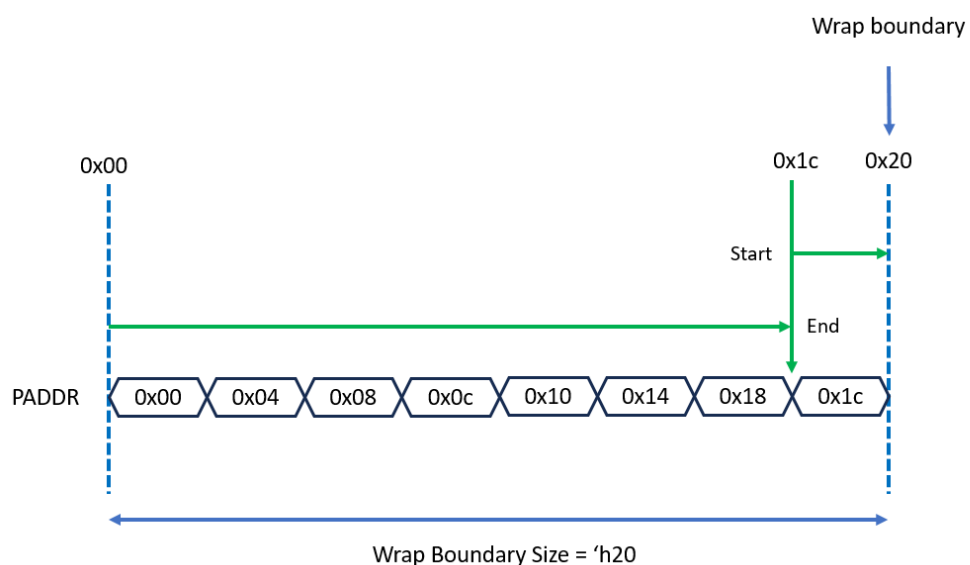
$$\text{Wrap Boundary Size} = (\text{AWLEN} + 1) * (2^{\text{AWSIZE}}) = 8 * 4 = 'b32 = 'h20$$

Wrap Boundary

$$= \text{Int} \left(\frac{\text{AWADDR}}{2^{\text{AWSIZE}} * (\text{AWLEN} + 1)} \right) * 2^{\text{AWSIZE}} * (\text{AWLEN} + 1)$$

$$= \text{Int} \left(\frac{1C}{2^2 * (7 + 1)} \right) * 2^2 * (7 + 1) = 'b32 = 'h20$$

Dựa trên các tính toán ở phía trên ta có được Khoảng địa chỉ wrap (Wrap boundary Size) là 'h20 và Địa chỉ ngưỡng (Wrap Boundary) là 'h20. Vì vậy trình tự địa chỉ thay đổi như hình 3.19:



Hình 3.19 Trình tự thay đổi địa chỉ ở kiểu truyền WRAP

Quá trình truyền bắt đầu tại 0x1c, theo thông thường như kiểu truyền INCR thì địa chỉ tiếp theo là 0x20, tuy nhiên 0x20 đã là địa chỉ ngưỡng vì vậy địa chỉ tiếp sẽ quay về địa chỉ đầu tiên 0x00. Tiếp theo địa chỉ sẽ tiếp tục tăng đến địa chỉ 0x18 và kết thúc truyền do đã đủ độ 8 địa chỉ ($AWLEN = 7$).

Tuy nhiên, nếu thực hiện hàng loạt phép nhân và phép chia trong thiết kế để tính toán Wrap Boundary Size và Wrap Boundary có thể sẽ tăng cao độ phức tạp trong thiết kế. Từ đó nhóm sẽ thực hiện tính toán địa chỉ bằng cách tính toán tại một nhóm bit nhất định của địa chỉ dựa trên AxLEN được thể hiện trong bảng 3.4 sau:

Bảng 3.4 Bảng thể hiện nhóm bit tính toán tương ứng với AxLEN

	AxLEN = 1	AxLEN = 3	AxLEN = 7	AxLEN = 15
Nhóm bit cần tính	PADDR[2:0]	PADDR[3:0]	PADDR[4:0]	PADDR[5:0]

Để tính được địa chỉ tiếp theo (Next_APB_addr) trong ví dụ được đề cập trên, ta sẽ thực hiện tính toán như sau:

$$\text{Next_APB_addr}[31:0] = \{\text{PADDR}[31:5], \text{PADDR}[4:0] + 2^{AWSIZE}\}$$

Dưới đây là danh sách trình tự địa chỉ thay đổi theo cách tính bằng nhóm bit

Bảng 3.5 Trình tự thay đổi địa chỉ theo cách tính bằng nhóm bit

Thứ tự	PADDR (Hex)	PADDR[4:0] (Bit)	2^{AWSIZE}	Next_APB_addr (Hex)
1	0000_001c	1_1100	4	0000_0000
2	0000_0000	0_0000	4	0000_0004
3	0000_0004	0_0100	4	0000_0008
4	0000_0008	0_1000	4	0000_000c
5	0000_000c	0_1100	4	0000_0010
6	0000_0010	1_0000	4	0000_0014
7	0000_0014	1_0100	4	0000_0018
8	0000_0018	1_1000	4	0000_001c

Qua cách tính này, ta cũng sẽ có được trình tự truyền tương tự với cách truyền do ARM đề xuất. Cách tính bằng nhóm bit có độ phức tạp thiết kế thấp hơn so với đề xuất từ ARM do chỉ sử dụng một nhóm bit để tính toán.

Phương pháp tính địa chỉ WRAP trên chỉ phù hợp với những tùy chỉnh mặc định của tín hiệu phía AXI như đã đề cập ở mục 3.2.5.1: cơ chế tính toán địa chỉ cho kiểu truyền INCR rằng tất cả các ngoại vi đều có độ dài 8-bit nên địa chỉ sẽ tự động tăng theo AxSIZE và AxLEN. Vì vậy, để địa chỉ tăng và được vòng lại phù hợp với ngoại vi tương ứng thì bây giờ hệ thống cần phải xem xét được đến địa chỉ ngoại vi đích đến.

Bảng 3.6 Bảng thay đổi các nhóm bit của ngoại vi 8-bit

STT	Độ dài gói tin	Số lượng gói tin	Bit ADDR giữ	Bit ADDR bit thay đổi
1	8	2	[31:1]	[0]
2	16	2	[31:2]	[1:0]
3	32	2	[31:3]	[2:0]
4	8	4	[31:2]	[1:0]
5	16	4	[31:3]	[2:0]
6	32	4	[31:4]	[3:0]
7	8	8	[31:3]	[2:0]
8	16	8	[31:4]	[3:0]
9	32	8	[31:5]	[4:0]
10	8	16	[31:4]	[3:0]
11	16	16	[31:5]	[4:0]
12	32	16	[31:6]	[5:0]

Sự thay đổi về địa chỉ ở số thứ tự thứ 9 chính là phương thức WRAP được đề cập ở bảng 3.4, nó chỉ nói về một trường hợp duy nhất là gói tin có độ dài 32-bit và số lượng gói tin sẽ là 8. Khi đó địa chỉ thay đổi sẽ từ 0x00 đến 0x20, theo mã nhị phân ta sẽ có là 0000_0000 đến 0010_000, chỉ cần giữ nguyên bit thứ 6 trở đi và thay đổi vùng 5 bit nhỏ nhất. Đối với trường hợp 12, ta sẽ có 16 gói tin với độ

dài 32-bit khi truyền vào ngoại vi 8-bit, vậy tổng địa chỉ thay đổi sẽ từ 0x00 đến 0x40, với nhị phân 0000_0000 đến 0100_0000, vì thế ta sẽ thay đổi địa chỉ 6 bit nhỏ nhất. Hãy nhớ rằng độ dài gói tin, số lượng gói tin và loại ngoại vi sẽ cho ra kết quả tăng địa chỉ khác nhau nhưng khoảng địa chỉ thay đổi sẽ như nhau, vì vậy nói về cách địa chỉ hãy tham khảo mục 3.2.5.1: cơ chế tính toán địa chỉ cho kiểu truyền INCR.

Bảng 3.7 và 3.8 sẽ mô tả nhóm bit thay đổi với ngoại vi 16-bit với 32-bit.

Bảng 3.7 Bảng mô tả nhóm bit thay đổi với ngoại vi 16-bit

STT	Độ dài gói tin	Số lượng gói tin	Bit ADDR giữ	Bit ADDR bit thay đổi
1	8	2	[31:1]	[0]
2	16	2	[31:1]	[0]
3	32	2	[31:2]	[1:0]
4	8	4	[31:2]	[1:0]
5	16	4	[31:2]	[1:0]
6	32	4	[31:3]	[2:0]
7	8	8	[31:3]	[2:0]
8	16	8	[31:3]	[2:0]
9	32	8	[31:4]	[3:0]
10	8	16	[31:4]	[3:0]
11	16	16	[31:4]	[3:0]
12	32	16	[31:5]	[4:0]

Với trường hợp ngoại vi 16-bit, nếu độ dài dữ liệu của gói tin bé hơn độ dài dữ liệu ngoại vi, số bit địa chỉ cần thay đổi chỉ phụ thuộc vào số lượng gói tin. Nếu lớn hơn thì ta cần phải tính toán vùng bao phủ của toàn bộ các gói tin khi có độ dài dữ liệu ngoại vi.

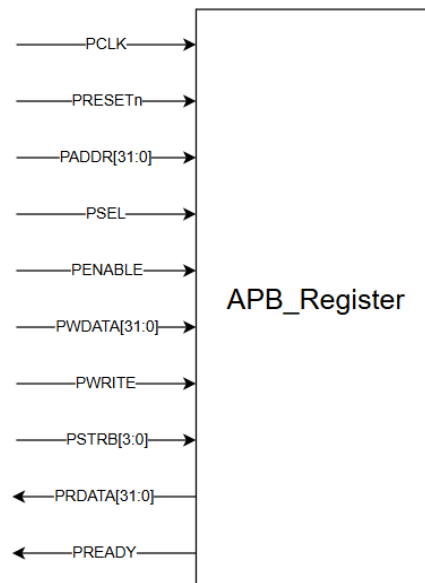
Bảng 3.8 Bảng mô tả nhóm bit thay đổi với ngoại vi 32-bit

STT	Độ dài gói tin	Số lượng gói tin	Bit ADDR giữ	Bit ADDR bit thay đổi
1	8	2	[31:1]	[0]
2	16	2	[31:1]	[0]
3	32	2	[31:1]	[0]
4	8	4	[31:2]	[1:0]
5	16	4	[31:2]	[1:0]
6	32	4	[31:2]	[1:0]
7	8	8	[31:3]	[2:0]
8	16	8	[31:3]	[2:0]
9	32	8	[31:3]	[2:0]
10	8	16	[31:4]	[3:0]
11	16	16	[31:4]	[3:0]
12	32	16	[31:4]	[3:0]

Với những phân tích đã nêu phía trên, giờ đây với ngoại vi 32-bit thì với bất cứ dữ liệu nào nhỏ hơn 32-bit thì bit địa chỉ thay đổi chỉ phụ thuộc vào tín hiệu AxLEN.

3.2.6 Thiết kế khối thanh ghi APB (APB Register)

Hình 3.20 thể hiện mô hình và tín hiệu của APB Register:



Hình 3.20 Mô hình và tín hiệu của khối APB Register

Khối thanh ghi APB Register được đề cập ở hình 3.20 là một APB Slave nhưng đây là một APB Slave chỉ đọc (Read-Only). Thanh ghi APB này chứa địa chỉ đầu và địa chỉ cuối của từng ngoại vi APB và của chính thanh ghi APB. Các ngoại vi APB sẽ được trình bày cụ thể hơn ở mục 3.3.1. Địa chỉ bắt đầu và địa chỉ kết thúc của các ngoại vi APB và thanh ghi được thiết lập như bảng 3.6:

Bảng 3.9 Địa chỉ bắt đầu và kết thúc của ngoại vi APB và thanh ghi APB

	Địa chỉ bắt đầu	Địa chỉ kết thúc
Thanh ghi APB	0x0000_0000	0x0000_0fff
Ngoại vi APB 8 bit	0x0000_1000	0x0000_1fff
Ngoại vi APB 16 bit	0x0000_2000	0x0000_2fff
Ngoại vi APB 32 bit	0x0000_3000	0x0000_3fff

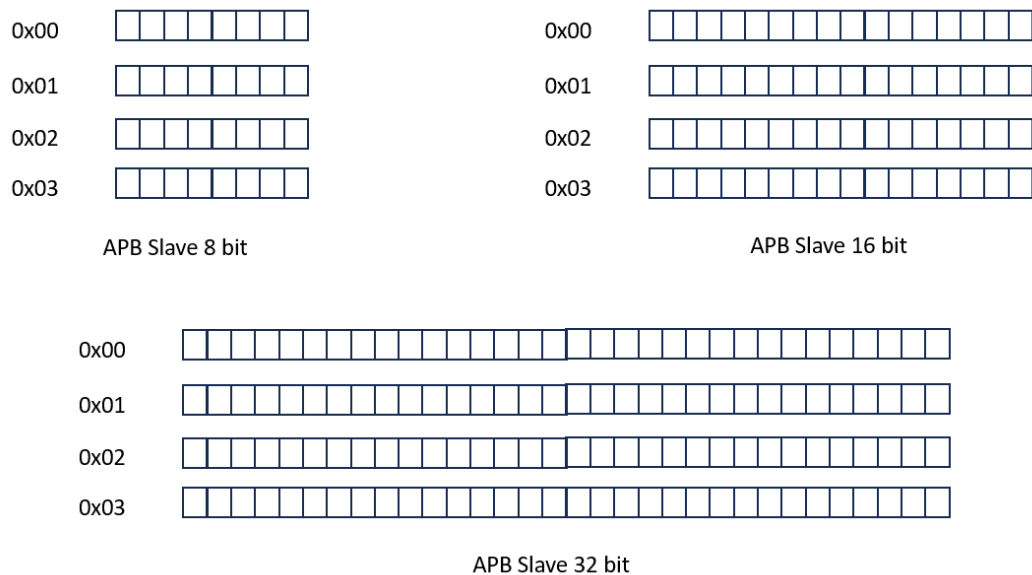
3.3 THIẾT KẾ MÔI TRƯỜNG KIỂM TRA UVM

3.3.1 Xây dựng kế hoạch kiểm tra

Dựa trên cơ sở lý thuyết về AXI và APB, ta có thể thấy rằng một quá trình truyền từ các yếu tố chính bao gồm: địa chỉ (address), kiểu truyền (burst type), kích thước mỗi gói trong 1 burst (burst size) và số lượng gói trong 1 burst (burst length). Vì vậy kế hoạch kiểm tra chức năng sẽ xoay quanh các yếu tố này.

Đối với địa chỉ (address):

Nhóm nghiên cứu sẽ kết nối 3 ngoại vi APB (APB Slave) vào phía APB của cầu nối có độ dài dữ liệu của 1 địa chỉ lần lượt là 8 bit, 16 bit và 32 bit. Hình 3.21 mô tả đặc điểm về độ dài dữ liệu của 1 địa chỉ của các ngoại vi APB:



Hình 3.21 Đặc điểm độ dài dữ liệu của từng ngoại vi APB

Hình 3.21 đã mô tả đặc điểm về độ dài dữ liệu của 1 địa chỉ của các ngoại vi APB. Tiếp theo sẽ là khoảng địa chỉ của từng slave:

APB Slave 8 bit: 32'h0000_1000 đến 32'h0000_1fff

APB Slave 16 bit: 32'h0000_2000 đến 32'h0000_2fff

APB Slave 32 bit: 32'h0000_3000 đến 32'h0000_3fff

Đối với kiểu truyền (burst type):

Môi trường kiểm tra sẽ cần bao quát 3 kiểu truyền là FIXED, INCR, WRAP. Tuy nhiên môi trường sẽ tập trung kiểm tra ở 2 kiểu truyền INCR và WRAP do kiểu FIXED với đặc điểm chỉ ghi vào 1 địa chỉ duy nhất.

Đối với kích thước mỗi gói trong 1 burst (burst size):

Thiết kế của cầu nối hỗ trợ truyền ở kích thước lần lượt là 8 bit, 16 bit và 32 bit, vì vậy nhóm sẽ bao quát cả 3 kích thước truyền này trong quá trình kiểm tra chức năng.

Đối với số lượng gói trong 1 burst (burst length):

Thiết kế của cầu nối hỗ trợ truyền với số lượng gói trong 1 burst lần lượt là 2,4,8 và 16 gói, vì vậy nhóm sẽ bao quát tất cả số lượng gói trong quá trình kiểm tra chức năng.

Từ những yếu tố trong quá trình test, nhóm đã xây dựng tất cả là 72 trường hợp test bao gồm: 3 ngoại vi APB, 2 loại kiểu truyền, 3 loại kích thước gói, 4 loại số lượng gói.

Bảng 3.10 Bảng thống kê hạng mục kiểm thử

Số thứ tự	Ngoại vi	Kiểu truyền	Kích thước mỗi gói	Số lượng gói
1	APB-8bit	INCR	8-bit	2
2	APB-8bit	INCR	8-bit	4
3	APB-8bit	INCR	8-bit	8
4	APB-8bit	INCR	8-bit	16
5	APB-8bit	INCR	16-bit	2
6	APB-8bit	INCR	16-bit	4
7	APB-8bit	INCR	16-bit	8
8	APB-8bit	INCR	16-bit	16
9	APB-8bit	INCR	32-bit	2
10	APB-8bit	INCR	32-bit	4
11	APB-8bit	INCR	32-bit	8
12	APB-8bit	INCR	32-bit	16
13	APB-16bit	INCR	8-bit	2
14	APB-16bit	INCR	8-bit	4
15	APB-16bit	INCR	8-bit	8
16	APB-16bit	INCR	8-bit	16
17	APB-16bit	INCR	16-bit	2
18	APB-16bit	INCR	16-bit	4
19	APB-16bit	INCR	16-bit	8
20	APB-16bit	INCR	16-bit	16

21	APB-16bit	INCR	32-bit	2
22	APB-16bit	INCR	32-bit	4
23	APB-16bit	INCR	32-bit	8
24	APB-16bit	INCR	32-bit	16
25	APB-32bit	INCR	8-bit	2
26	APB-32bit	INCR	8-bit	4
27	APB-32bit	INCR	8-bit	8
28	APB-32bit	INCR	8-bit	16
29	APB-32bit	INCR	16-bit	2
30	APB-32bit	INCR	16-bit	4
31	APB-32bit	INCR	16-bit	8
32	APB-32bit	INCR	16-bit	16
33	APB-32bit	INCR	32-bit	2
34	APB-32bit	INCR	32-bit	4
35	APB-32bit	INCR	32-bit	8
36	APB-32bit	INCR	32-bit	16
37	APB-8bit	WRAP	8-bit	2
38	APB-8bit	WRAP	8-bit	4
39	APB-8bit	WRAP	8-bit	8
40	APB-8bit	WRAP	8-bit	16
41	APB-8bit	WRAP	16-bit	2
42	APB-8bit	WRAP	16-bit	4
43	APB-8bit	WRAP	16-bit	8
44	APB-8bit	WRAP	16-bit	16
45	APB-8bit	WRAP	32-bit	2
46	APB-8bit	WRAP	32-bit	4
47	APB-8bit	WRAP	32-bit	8
48	APB-8bit	WRAP	32-bit	16
49	APB-16bit	WRAP	8-bit	2
50	APB-16bit	WRAP	8-bit	4
51	APB-16bit	WRAP	8-bit	8
52	APB-16bit	WRAP	8-bit	16
53	APB-16bit	WRAP	16-bit	2
54	APB-16bit	WRAP	16-bit	4
55	APB-16bit	WRAP	16-bit	8
56	APB-16bit	WRAP	16-bit	16
57	APB-16bit	WRAP	32-bit	2
58	APB-16bit	WRAP	32-bit	4
59	APB-16bit	WRAP	32-bit	8
60	APB-16bit	WRAP	32-bit	16
61	APB-32bit	WRAP	8-bit	2
62	APB-32bit	WRAP	8-bit	4
63	APB-32bit	WRAP	8-bit	8
64	APB-32bit	WRAP	8-bit	16

65	APB-32bit	WRAP	16-bit	2
66	APB-32bit	WRAP	16-bit	4
67	APB-32bit	WRAP	16-bit	8
68	APB-32bit	WRAP	16-bit	16
69	APB-32bit	WRAP	32-bit	2
70	APB-32bit	WRAP	32-bit	4
71	APB-32bit	WRAP	32-bit	8
72	APB-32bit	WRAP	32-bit	16

Bên cạnh đó một yếu tố được đề cập trong các test chính là tần số hoạt động của AXI và APB. Nhằm thiết lập đa dạng các tần số hoạt động, nhóm quyết định thiết lập như sau:

- Đối với AXI: Có 3 mức tần số hoạt động bao gồm 250 MHz, 200 MHz và 100 MHz
- Đối với APB: Có 3 mức tần số hoạt động bao gồm 25 MHz, 20 MHz và 10 MHz

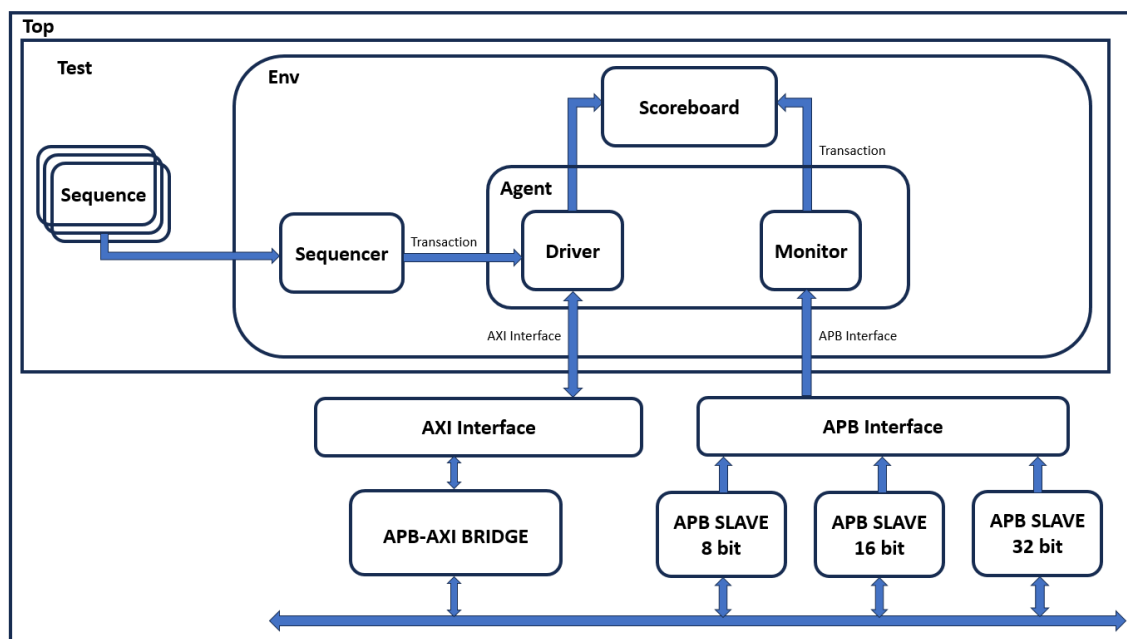
Để xác định được thế nào là một giao dịch mong muốn thì nhóm sẽ thực hiện kiểm tra các hạng mục sau:

- Kiểm tra cơ chế truyền từ AXI và APB bao gồm cơ chế bắt tay Valid – Ready (được trình bày ở mục 2.3.3) và các tín hiệu ngõ ra từ APB Master có tuân theo đúng sơ đồ máy trạng thái FSM đã được trình bày ở mục 3.2.5 hay không. Khâu kiểm tra này được thực hiện bằng cách xác minh trên waveform.
- Kiểm tra các thông tin về burst trong giao dịch bao gồm: Burst length, burst size, burst type. Khâu này sẽ được kiểm tra trong khối scoreboard từ UVM.
- Kiểm tra thông tin về địa chỉ giữa AXI và APB trong các thiết lập truyền khác nhau. Trong các thiết lập truyền khác nhau thì bước nhảy địa chỉ ở phía APB sẽ là khác nhau, vì vậy nhóm sẽ kiểm tra thông tin địa chỉ cùng với thiết lập kiểu truyền trong giao dịch. Khâu này sẽ được kiểm tra trong khối scoreboard từ UVM.

- So sánh kiểm tra thông tin về dữ liệu được truyền giữa AXI và APB trong các thiết lập truyền khác nhau. Khâu này sẽ được kiểm tra trong khối scoreboard từ UVM.

3.3.2 Xây dựng các khối trong môi trường

Thông qua thực hiện mô phỏng với module giúp hiểu rõ hơn cách thức hoạt động của module và kiểm tra những yêu cầu ban đầu đối với module có đạt được đúng như kỳ vọng không. Một môi trường UVM để thực hiện mô phỏng gồm các khối trong hình 3.22 dưới đây:



Hình 3.22 Sơ đồ môi trường UVM

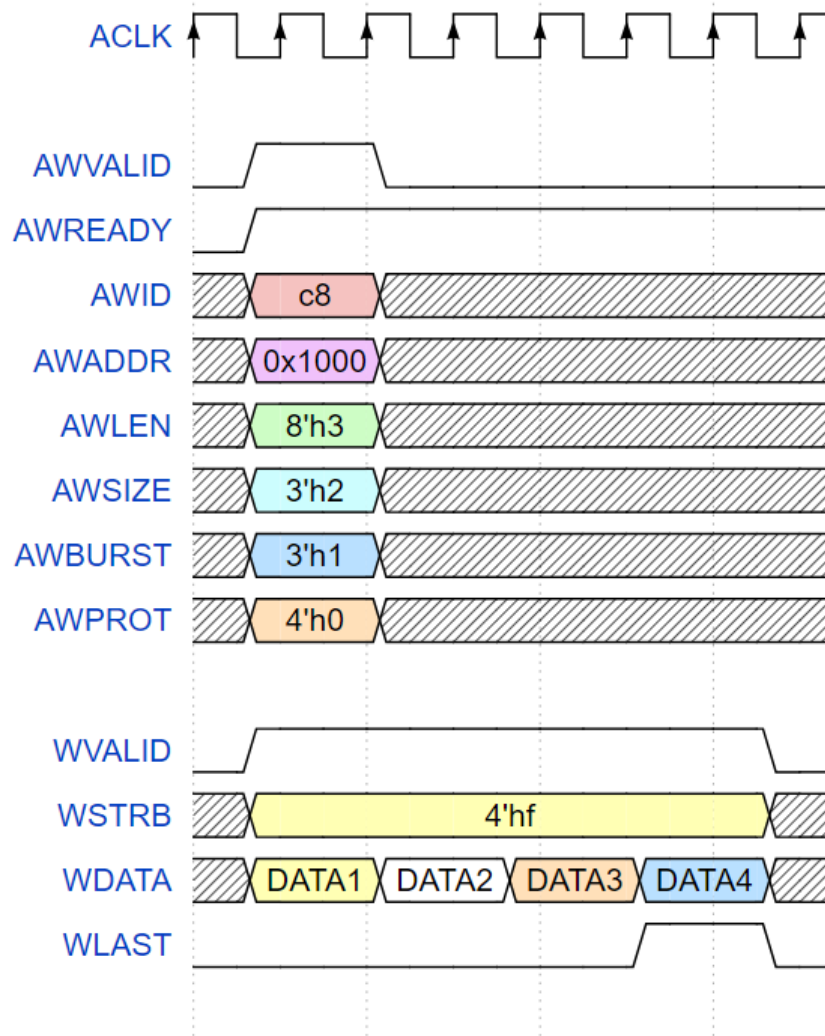
Khối Sequence: Đóng vai trò như một AXI Master để thực hiện gửi các lệnh và dữ liệu theo chuẩn AXI vào bên trong cầu nối. Sequence bao gồm các tín hiệu như sau:

- Đối với giao dịch ghi: AWADDR[31:0], AWSIZE[2:0], AWLEN[7:0], AWBURST[1:0], AWID[7:0], AWPROT[3:0], WDATA[31:0],WSTRB[3:0].
- Đối với giao dịch đọc: ARADDR[31:0], ARSIZE[2:0], ARLEN[7:0], ARBURST[1:0], ARID[7:0], APPROT[3:0], RREADY.

Khối Sequencer: Đóng vai trò là người vận chuyển gói tin vào Driver

Khối Driver: Thực hiện gửi các tín hiệu từ gửi từ Sequencer đến cầu nối AXI-APB thông qua interface. Bên cạnh đó khối Driver còn ghi nhận các dữ liệu cần thiết ở phía AXI và gửi lên scoreboard.

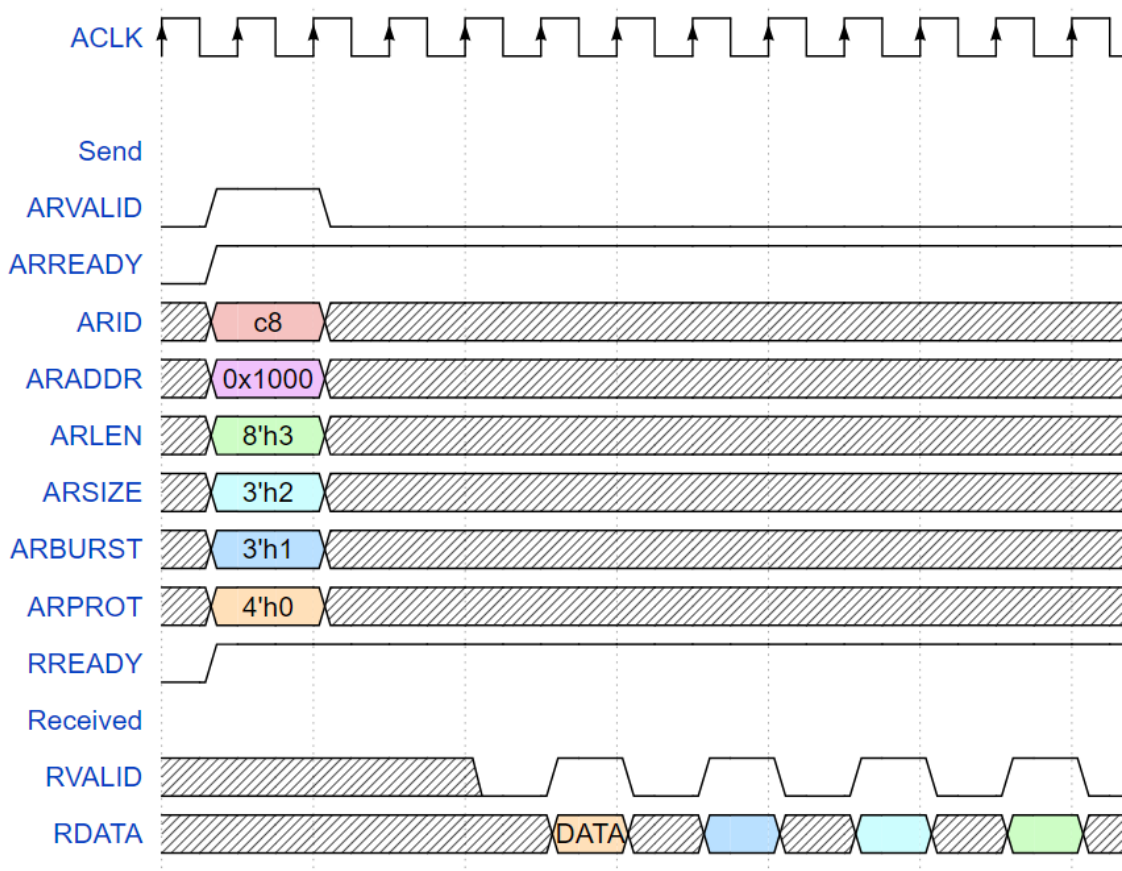
- Đối với hoạt động ghi: Driver sẽ gửi các tín hiệu được mô tả trong một ví dụ ở hình 3.23:



Hình 3.23 Quá trình gửi tín hiệu từ Driver với hoạt động ghi

Như hình 3.23 đã mô tả, Driver sẽ gửi các tín hiệu trên 2 kênh là AW và W đến AXI Interface. Như trong ví dụ này, chúng ta có AWLEN là 8'h3 nên sẽ có 4 khối dữ liệu được truyền đi. Mỗi khối dữ liệu sẽ tạo ra một cách ngẫu nhiên dựa trên cơ chế randomize của UVM.

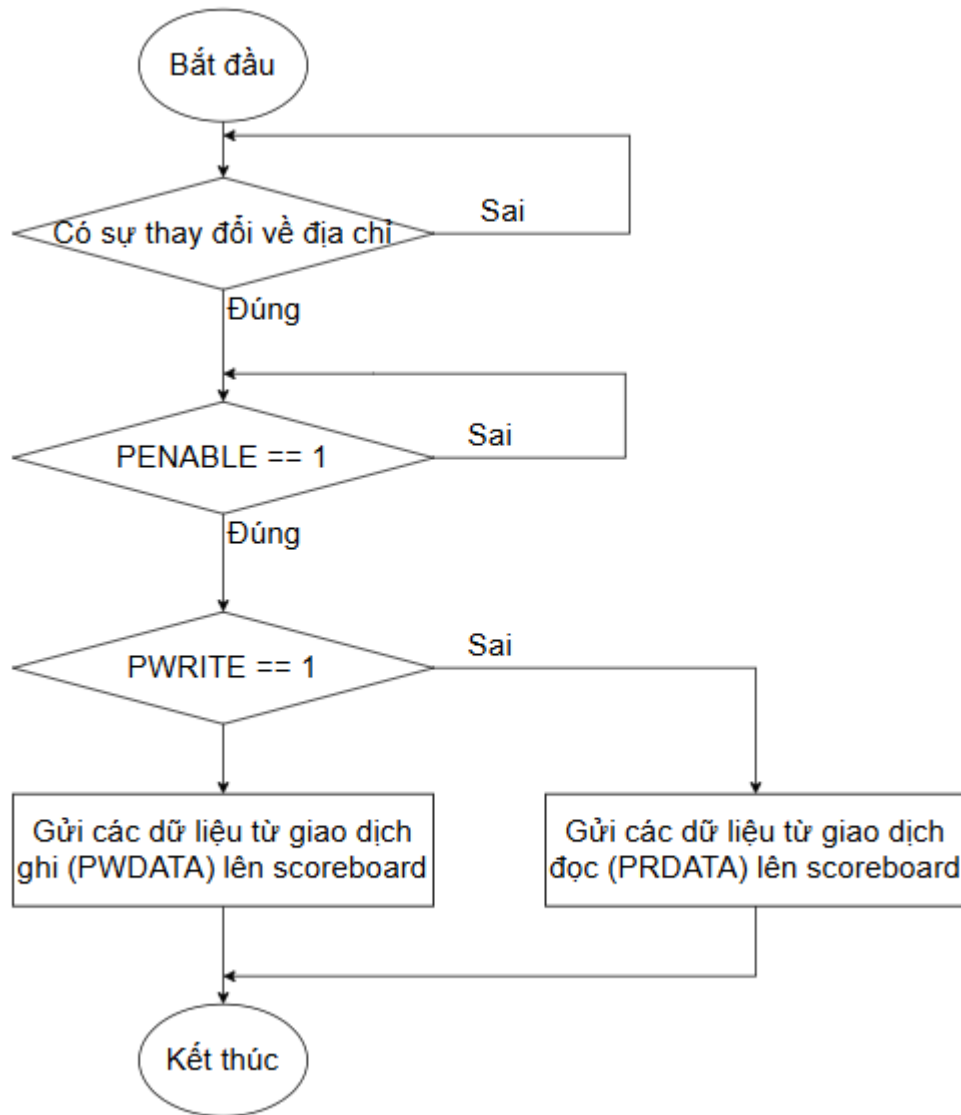
- Đối với hoạt động đọc: Driver sẽ gửi các tín hiệu được mô tả trong một ví dụ ở hình 3.24:



Hình 3.24 Quá trình gửi tín hiệu từ Driver với hoạt động đọc

Như hình 3.24 đã minh họa, Driver sẽ gửi các tín hiệu từ kênh AR đến AXI Interface. Như ở trong ví dụ này, chúng ta có ARLEN là 8'h3 nên khi nhận dữ liệu tương ứng sẽ có 4 khối dữ liệu nhận được. Bên cạnh đó, để nhận được dữ liệu thì kênh R cần kích hoạt tín hiệu RREADY lên mức cao để sẵn sàng nhận dữ liệu.

Khối Monitor: Thực hiện ghi nhận các dữ liệu cần thiết ở phía APB để gửi lên scoreboard. Hình 3.25 dưới đây là lưu đồ của khối Monitor:



Hình 3.25 Lưu đồ hoạt động của khối Monitor

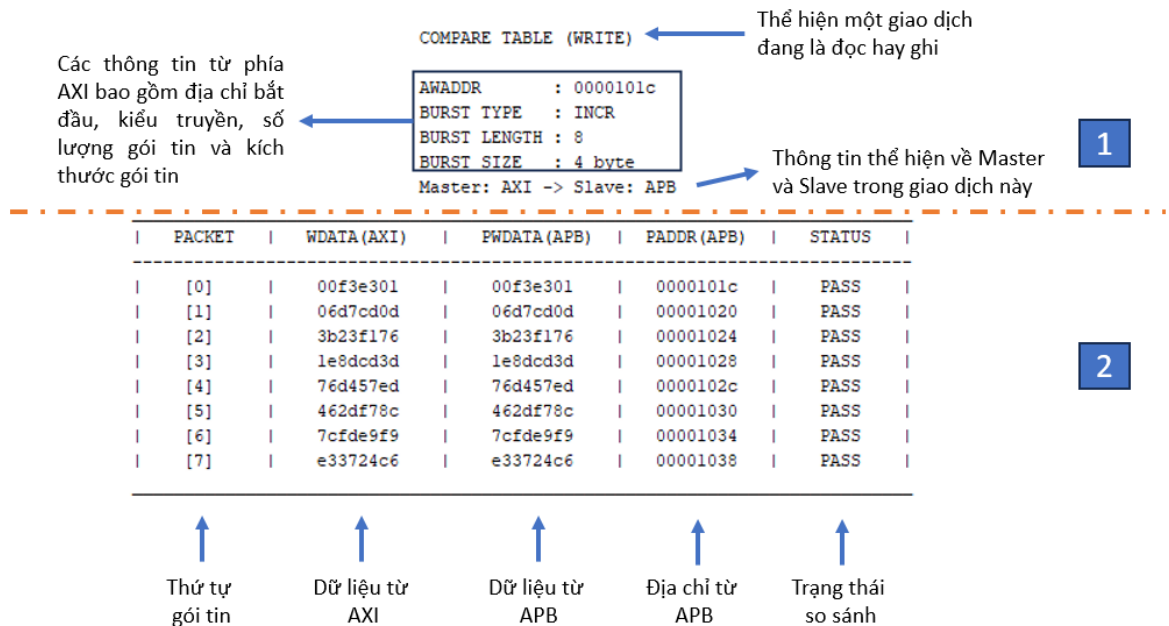
Như hình 3.25 đã mô tả, monitor chịu trách nhiệm theo dõi các tín hiệu APB ở phía ngõ ra. Khi đáp ứng đủ các điều kiện bao gồm sự thay đổi về địa chỉ và tín hiệu PENABLE thì Monitor sẽ dựa vào tín hiệu PWRITE để quyết định chuyển dữ liệu ghi (PWRITE) hoặc dữ liệu đọc (PRDATA) lên scoreboard.

Khối AXI Interface: Nơi tương tác và truyền tín hiệu trực tiếp đến đầu vào AXI của cầu nối AXI-APB.

Khối APB Interface: Nơi tương tác và truyền tín hiệu trực tiếp đến đầu ra APB của cầu nối AXI-APB và các ngoại vi APB.

Khối Scoreboard: Nơi thu thập các dữ liệu gửi về từ Driver và Monitor và tiến hành so sánh và thông báo kết quả. Dựa theo yêu cầu kiểm tra, nhóm quyết định

xây dựng mô hình scoreboard có thể đáp ứng khả năng dễ dàng theo dõi và kiểm tra. Hình 3.26 dưới đây sẽ mô tả ví dụ về cách hiển thị báo cáo của scoreboard:



Hình 3.26 Ví dụ về cách hiển thị báo cáo của scoreboard

Theo hình 3.26, ta thấy có hai khu vực hiển thị thông tin:

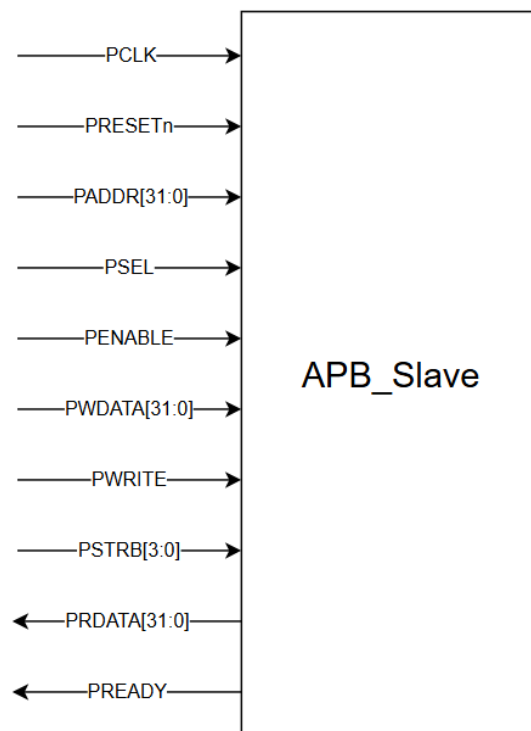
- Khu vực [1]: Hiển thị thông tin về giao dịch bao gồm loại giao dịch (đọc hay ghi), địa chỉ bắt đầu (AWADDR/ARADDR), kiểu truyền (BURST TYPE), số lượng gói tin (BURST LENGTH), kích thước gói tin (BURST SIZE) và thông tin thể hiện vai trò Master và Slave trong giao dịch.
- Khu vực [2]: Hiển thị thông tin về dữ liệu và địa chỉ giữa hai đầu AXI và APB. Cột PACKET hiển thị thứ tự gói tin, cột WDATA hiển thị dữ liệu từ phía AXI, cột PWDATA hiển thị dữ liệu từ phía APB, cột PADDR hiển thị thông tin về địa chỉ của APB và cột STATUS sẽ cập nhật trạng thái so sánh của cặp dữ liệu AXI-APB tương ứng.

Để có thể so sánh ở phía scoreboard, ta cần thu thập đủ số lượng dữ liệu truyền giữa phía AXI và APB của cầu nối, số lượng dữ liệu truyền sẽ phụ thuộc vào AxLEN của giao dịch tương ứng. Các dữ liệu này được ghi vào một hàng đợi, sau đó khi đã đủ số lượng thì ta tiến hành lấy các dữ liệu và so sánh theo từng cặp dữ liệu tương ứng đến từ giao dịch đọc và giao dịch ghi. Hàng đợi này được nhóm sử

dụng dựa trên cơ chế Queue từ System Verilog với các phương thức như `push_back()` và `pop_front()`.

Khối cầu nối AXI-APB: Đây là nơi nhận các tín hiệu và thực hiện các hoạt động truyền hoặc nhận dữ liệu.

Khối APB Slave: Đây là nơi kết nối với phía APB của cầu nối AXI-APB để nhận các tín hiệu và thực thi theo các hoạt động đọc hoặc ghi. Mô hình APB Slave được mô tả như hình 3.27:



Hình 3.27: Mô hình APB Slave

Hình 3.27 mô tả khối APB Slave được sử dụng trong mô phỏng, khối này có các đường tín hiệu và các hoạt động phù hợp theo chuẩn APB.

Phương pháp để thực hiện mô phỏng và kiểm tra hoạt động của cầu nối sẽ là sử dụng môi trường UVM cùng 3 khối APB Slave để truyền và nhận dữ liệu, sau đó so sánh để kiểm tra tính chính xác trong các chức năng.

CHƯƠNG 4

KẾT QUẢ

4.1 CÁC TRƯỜNG HỢP KIỂM THỬ CHỨC NĂNG CỦA QUÁ TRÌNH TRUYỀN

Với kiến thức đã được đề cập ở chương 2 và mô hình thiết kế đã đề xuất ở chương 3, số lượng trường hợp kiểm tra lên đến 72 trường hợp. Vì vậy, nhóm lựa ra 4 trường hợp kiểm thử đặc trưng nhất, có khả năng bao quát hết toàn bộ các phép kiểm thử còn lại. Các trường hợp mà nhóm nghiên cứu đã lựa chọn được trình bày trong bảng dưới đây:

Bảng 4.1 Bảng biểu diễn các hạng mục kiểm tra sẽ được lựa chọn

Số thứ tự	Ngoại vi	Kiểu truyền	Kích thước mỗi gói	Số lượng gói
1	APB-8bit	INCR	32-bit	8
2	APB-8bit	WRAP	32-bit	8
3	APB-16bit	WRAP	32-bit	8
4	APB-32bit	WRAP	32-bit	8

1. Ghi và đọc dữ liệu 32-bit vào ngoại vi APB 8-bit với kiểu truyền tăng địa chỉ tuần tự (INCR).
2. Ghi và đọc dữ liệu 32-bit vào ngoại vi APB 8-bit với kiểu truyền tăng địa chỉ tuần hoàn (WRAP).
3. Ghi và đọc dữ liệu 32-bit vào ngoại vi APB 16-bit với kiểu truyền tăng địa chỉ tuần hoàn (WRAP).
4. Ghi và đọc dữ liệu 32-bit vào ngoại vi APB 32-bit với kiểu truyền tăng địa chỉ tuần hoàn (WRAP).

Nhóm nghiên cứu đã lựa chọn 4 phép kiểm thử trên dựa trên những tiêu chí đã rút ra được từ phần lý thuyết Chương 2:

- Độ dài dữ liệu là 32-bit vì đây là giá trị độ dài dữ liệu lớn nhất mà cầu nối có thể xử lý được. Khi hệ thống xử lý đúng với kích thước dữ liệu lớn nhất, thì các kích thước nhỏ hơn như 8 và 16-bit về mặt kỹ thuật cũng sẽ được đảm bảo hoạt động đúng.
- Cả hai kiểu truyền dữ liệu INCR và WRAP đều sẽ được đưa vào ở ngõ vào, để kiểm tra cách cầu nối xử lý thay đổi địa chỉ trong quá trình truyền.
- Sẽ bao gồm tất cả ngoại vi của APB, để đảm bảo rằng cầu nối AXI-APB sẽ xử lý dữ liệu và địa chỉ tương ứng với mỗi ngoại vi khác nhau.

Nhờ đó, mặc dù chỉ sử dụng 4 trường hợp kiểm thử, nhóm vẫn đảm bảo được khả năng đánh giá toàn diện và tin cậy cho hệ thống. Các trường hợp còn lại (68/72) có thể được suy diễn tính đúng đắn từ các trường hợp đã kiểm tra, nhờ tính chất kế thừa và bao phủ trong hoạt động của bus AXI-APB. Việc rút gọn này giúp tiết kiệm thời gian và công sức, nhưng vẫn đảm bảo chất lượng kiểm tra có bao phủ đầy đủ các tính năng của hệ thống.

Các trường hợp (testcase) kiểm thử sẽ được liệt kê trong bảng 4.2:

Bảng 4.2 Bảng mô tả chi tiết hạng mục kiểm tra

Testcase	Mô tả	Cấu hình
Testcase 1	AXI – APB ở quá trình ghi và đọc với dữ liệu 32-bit và slave 8-bit chế độ INCR	Giai đoạn 1: Truyền tín hiệu địa chỉ ở kênh AW và dữ liệu ở kênh W để ghi vào slave: - Truyền tín hiệu địa chỉ và các tín hiệu điều khiển ở kênh AW: + Tín hiệu thông báo phía master đã sẵn sàng truyền thông tin điều khiển: - AWVALID = 1 + Tín hiệu địa chỉ: - AWADDR = 0x0000101C + Tín hiệu thông báo độ dài của một giao dịch ghi: - AWLEN = 7 + Tín hiệu kích thước của một khung dữ liệu:

		AWSIZE = 2 + Tín hiệu định dạng kiểu truyền địa chỉ: AWBURST = 1 + Tín hiệu mã số cho giao dịch ghi: AWID = 0xC8 - Truyền tín hiệu dữ liệu ở kênh W: + Gửi tín hiệu thông báo sẵn sàng gửi dữ liệu: WVALID = 1 + Tín hiệu mặt nạ dữ liệu: WSTRB = 0x0F + Truyền dữ liệu theo độ dài đã xác định ở kênh AW: WDATA = 0x12153524 WDATA = 0xC0895E81 WDATA = 0x8484D609 WDATA = 0xB1F05663 WDATA = 0x06B97B0D WDATA = 0x46DF998D WDATA = 0xB2C28465 WDATA = 0x89375212 + Tín hiệu thông báo dữ liệu cuối cùng được gửi: WLAST = 1 Giai đoạn 2: Truyền tín hiệu địa chỉ ở kênh AR để thực hiện đọc data: - Truyền tín hiệu địa chỉ và các tín hiệu điều khiển ở kênh AR: + Tín hiệu thông báo phía master đã sẵn sàng truyền thông tin điều khiển: ARVALID = 1 + Tín hiệu địa chỉ:
--	--	---

		ARADDR = 0x0000101C + Tín hiệu thông báo độ dài của một giao dịch ghi: ARLEN = 7 + Tín hiệu kích thước của một khung dữ liệu: ARSIZE = 2 + Tín hiệu định dạng kiểu truyền địa chỉ: ARBURST = 1 + Tín hiệu mã số cho giao dịch ghi: ARID = 0x12
Testcase 2	AXI – APB ở quá trình ghi và đọc với dữ liệu 32-bit và slave 8-bit chế độ WRAP	Giai đoạn 1: Truyền tín hiệu địa chỉ ở kênh AW và dữ liệu ở kênh W để ghi vào slave: - Truyền tín hiệu địa chỉ và các tín hiệu điều khiển ở kênh AW: + Tín hiệu thông báo phía master đã sẵn sàng truyền thông tin điều khiển: AWVALID = 1 + Tín hiệu địa chỉ: AWADDR = 0x0000101C + Tín hiệu thông báo độ dài của một giao dịch ghi: AWLEN = 7 + Tín hiệu kích thước của một khung dữ liệu: AWSIZE = 2 + Tín hiệu định dạng kiểu truyền địa chỉ: AWBURST = 2 + Tín hiệu mã số cho giao dịch ghi: AWID = 0xC8 - Truyền tín hiệu dữ liệu ở kênh W: + Gửi tín hiệu thông báo sẵn sàng gửi dữ liệu: WVALID = 1 + Tín hiệu mặt nạ dữ liệu:

		WSTRB = 0x0F + Truyền dữ liệu theo độ dài đã xác định ở kênh AW: WDATA = 0x12153524 WDATA = 0xC0895E81 WDATA = 0x8484D609 WDATA = 0xB1F05663 WDATA = 0x06B97B0D WDATA = 0x46DF998D WDATA = 0xB2C28465 WDATA = 0x89375212 + Tín hiệu thông báo dữ liệu cuối cùng được gửi: WLAST = 1 Giai đoạn 2: Truyền tín hiệu địa chỉ ở kênh AR để thực hiện đọc data: - Truyền tín hiệu địa chỉ và các tín hiệu điều khiển ở kênh AR: + Tín hiệu thông báo phía master đã sẵn sàng truyền thông tin điều khiển: ARVALID = 1 + Tín hiệu địa chỉ: ARADDR = 0x0000101C + Tín hiệu thông báo độ dài của một giao dịch ghi: ARLEN = 7 + Tín hiệu kích thước của một khung dữ liệu: ARSIZE = 2 + Tín hiệu định dạng kiểu truyền địa chỉ: ARBURST = 2 + Tín hiệu mã số cho giao dịch ghi: ARID = 0x12
--	--	--

Testcase 3	AXI – APB ở quá trình ghi và đọc với dữ liệu 32-bit và slave 16-bit chế độ WRAP	Giai đoạn 1: Truyền tín hiệu địa chỉ ở kênh AW và dữ liệu ở kênh W để ghi vào slave: - Truyền tín hiệu địa chỉ và các tín hiệu điều khiển ở kênh AW: + Tín hiệu thông báo phía master đã sẵn sàng truyền thông tin điều khiển: AWVALID = 1 + Tín hiệu địa chỉ: AWADDR = 0x0000201C + Tín hiệu thông báo độ dài của một giao dịch ghi: AWLEN = 7 + Tín hiệu kích thước của một khung dữ liệu: AWSIZE = 2 + Tín hiệu định dạng kiểu truyền địa chỉ: AWBURST = 2 + Tín hiệu mã số cho giao dịch ghi: AWID = 0xC8 - Truyền tín hiệu dữ liệu ở kênh W: + Gửi tín hiệu thông báo sẵn sàng gửi dữ liệu: WVALID = 1 + Tín hiệu mặt nạ dữ liệu: WSTRB = 0x0F + Truyền dữ liệu theo độ dài đã xác định ở kênh AW: - WDATA = 0x12153524 - WDATA = 0xC0895E81 - WDATA = 0x8484D609 - WDATA = 0xB1F05663 - WDATA = 0x06B97B0D - WDATA = 0x46DF998D
------------	---	--

		WDATA = 0xB2C28465 WDATA = 0x89375212 + Tín hiệu thông báo dữ liệu cuối cùng được gửi: WLAST = 1 Giai đoạn 2: Truyền tín hiệu địa chỉ ở kênh AR để thực hiện đọc data: - Truyền tín hiệu địa chỉ và các tín hiệu điều khiển ở kênh AR: + Tín hiệu thông báo phía master đã sẵn sàng truyền thông tin điều khiển: ARVALID = 1 + Tín hiệu địa chỉ: ARADDR = 0x0000201C + Tín hiệu thông báo độ dài của một giao dịch ghi: ARLEN = 7 + Tín hiệu kích thước của một khung dữ liệu: ARSIZE = 2 + Tín hiệu định dạng kiểu truyền địa chỉ: ARBURST = 2 + Tín hiệu mã số cho giao dịch ghi: ARID = 0x12
Testcase 4	AXI – APB ở quá trình ghi và đọc với dữ liệu 32-bit và slave 32-bit chế độ WRAP	Giai đoạn 1: Truyền tín hiệu địa chỉ ở kênh AW và dữ liệu ở kênh W để ghi vào slave: - Truyền tín hiệu địa chỉ và các tín hiệu điều khiển ở kênh AW: + Tín hiệu thông báo phía master đã sẵn sàng truyền thông tin điều khiển: AWVALID = 1 + Tín hiệu địa chỉ: AWADDR = 0x00003008

		+ Tín hiệu thông báo độ dài của một giao dịch ghi: AWLEN = 7 + Tín hiệu kích thước của một khung dữ liệu: AWSIZE = 2 + Tín hiệu định dạng kiểu truyền địa chỉ: AWBURST = 2 + Tín hiệu mã số cho giao dịch ghi: AWID = 0xC8 - Truyền tín hiệu dữ liệu ở kênh W: + Gửi tín hiệu thông báo sẵn sàng gửi dữ liệu: WVALID = 1 + Tín hiệu mặt nạ dữ liệu: WSTRB = 0x0F + Truyền dữ liệu theo độ dài đã xác định ở kênh AW: WDATA = 0x12153524 WDATA = 0xC0895E81 WDATA = 0x8484D609 WDATA = 0xB1F05663 WDATA = 0x06B97B0D WDATA = 0x46DF998D WDATA = 0xB2C28465 WDATA = 0x89375212 + Tín hiệu thông báo dữ liệu cuối cùng được gửi: WLAST = 1 Giai đoạn 2: Truyền tín hiệu địa chỉ ở kênh AR để thực hiện đọc data: - Truyền tín hiệu địa chỉ và các tín hiệu điều khiển ở kênh AR:
--	--	---

		+ Tín hiệu thông báo phía master đã sẵn sàng truyền thông tin điều khiển: $ARVALID = 1$ + Tín hiệu địa chỉ: $ARADDR = 0x00003008$ + Tín hiệu thông báo độ dài của một giao dịch ghi: $ARLEN = 7$ + Tín hiệu kích thước của một khung dữ liệu: $ARSIZE = 2$ + Tín hiệu định dạng kiểu truyền địa chỉ: $ARBURST = 2$ + Tín hiệu mã số cho giao dịch ghi: $ARID = 0x12$
--	--	---

Nhóm nghiên cứu sẽ thực hiện những phép kiểm thử này với tần số chênh lệch nhau, cụ thể rằng ở bên phía MASTER AXI sẽ hoạt động ở tần số 250MHz và ở phía SLAVE APB sẽ hoạt động ở tần số 25MHz. Tuy rằng, mạch có thể thực hiện với tần số cao hơn đến 750MHz, việc nhóm sử dụng ở tần số thấp để có thể dễ dàng so sánh giữa các tín hiệu diễn ra gần nhau hơn, tín hiệu của AXI và APB sẽ không giống nhau mà sẽ gấp nhau 10 lần để tăng tính trực quan.

4.2 KẾT QUẢ MÔ PHỎNG

4.2.1 Testcase 1: AXI – APB ghi đọc dữ liệu 32-bit vào slave 8-bit chế độ INCR với $BURST_SIZE = 8$, $BURST_LENGTH = 32$

Testcase 1: AXI-APB ở quá trình ghi và đọc dữ liệu 32 bit và slave 8 bit chế độ INCR với $BURST_SIZE = 8$, $BURST_LENGTH = 32$ được mô tả như hình 4.1:



Hình 4.1 Dạng sóng tín hiệu chuẩn bị quá trình ghi và đọc

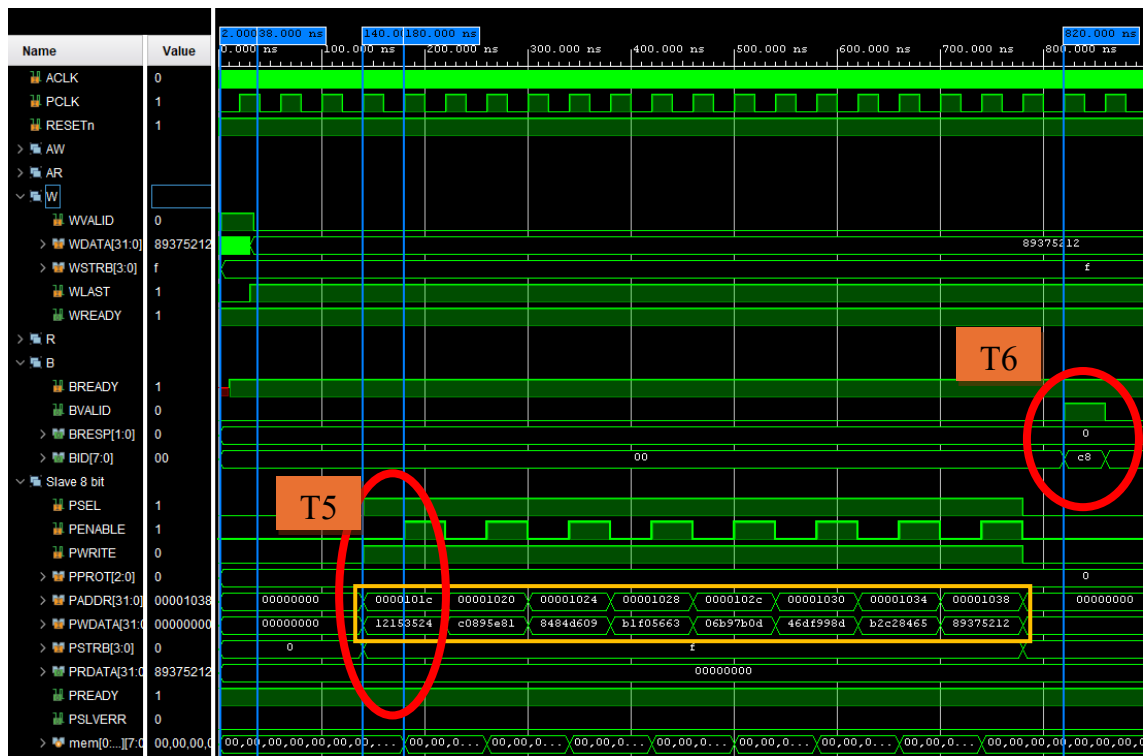
Quá trình của một chu kỳ tín hiệu được gửi đi từ phía AXI được thể hiện ở hình 4.1 như sau:

- (1) Tại ô T1, từ 0ns đến 1ns, $RESETn = 0$ để tiến hành khởi động lại hệ thống, sau đó $RESETn = 1$ để hệ thống bắt đầu hoạt động, $AWREADY$, $ARREADY$, $WREADY$ luôn giữ ở mức cao để cho phép truyền, nhận tín hiệu. Ở thời điểm 2 ns, các tín hiệu thiết lập cho cầu nối thực hiện quá trình ghi được bắt đầu. Với việc có cạnh lên cùng xung clock cùng những tín hiệu điều khiển ở kênh AW, lúc này địa chỉ bắt đầu $AWADDR$ bằng $0x0000101C$, kích thước của gói tin $AWSIZE = 2$ tức là kích thước gói tin là 32-bit (4-byte), $AWLEN = 7$ tức là sẽ có 8 gói tin trong một lần truyền,

AWBURST = 1 tức là chọn kiểu truyền địa chỉ là INCR theo chuẩn AMBA, AWID là 0xC8 để mã giao dịch.

- (2) Ở ô T2, ngay tại thời điểm cạnh lên xung clock đó thì cầu nối cũng sẽ nhận 8 gói tin lần lượt là 0x1215324, 0xC0895E81, 0x8484D609, 0xB1F05663, 0x06B97B0D, 0x46DF998D, 0xB2C28465, 0x89375212, 8 gói tin 32-bit ở kênh W, trùng khớp với thông tin từ kênh AW, AWLEN = 7, AWSIZE = 2. Tín hiệu WSTRB = 0x0F làm mặt nạ 32-bit dữ liệu, tức là sẽ sử dụng hết 32-bit của WDATA. Ở 2ns là gói tin đầu tiên, ở 6ns là gói tin thứ 2 và tiếp tục ở mỗi chu kỳ xung AXI cho đến gói tin thứ 8.
- (3) Khi gửi gói tin thứ 8, gói tin cuối cùng ở vị trí 30ns tại T3, tín hiệu WLAST sẽ được kích hoạt cùng lúc báo hiệu rằng đây là gói tin cuối cùng của khung dữ liệu truyền đi. Ở 34ns, tín hiệu WVALID = 0 đánh dấu cho việc kết thúc việc truyền dữ liệu ghi.
- (4) Tại ô T4, tại thời điểm 38ns, ở phía kết nối AXI sẽ phát tín hiệu rằng phía MASTER muốn đọc dữ liệu với các thông số ở kênh AR như sau: địa chỉ bắt đầu ARADDR bằng 0x0000101C, kích thước của gói tin ARSIZE = 2 là độ dài dữ liệu là 32-bit, ARLEN = 7 trong một chuỗi sẽ đọc 8 gói tin, ARBURST = 1 chọn kiểu truyền địa chỉ là INCR, ARID = 12.

Quá trình ghi ở ngoại vi APB 8 bit được biểu diễn qua hình 4.2



Hình 4.2 Dạng sóng quá trình ghi ở ngoại vi APB 8-bit

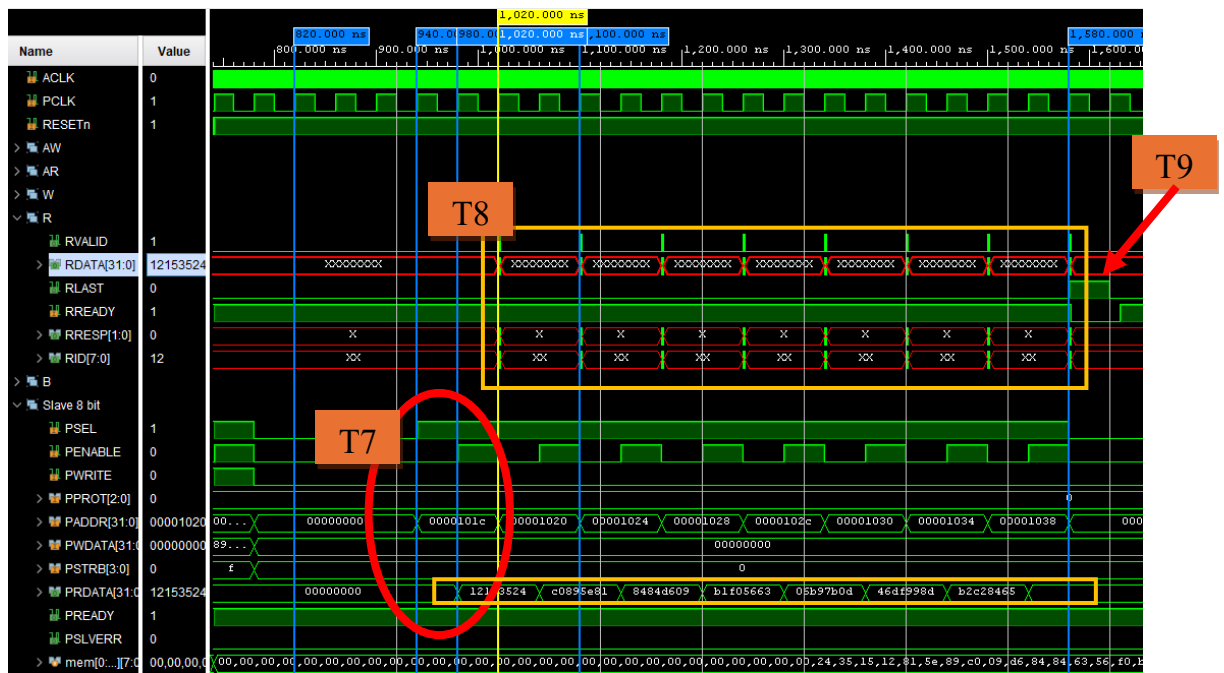
Từ mô tả ở hình 4.2, sau khi đã nhận được tín hiệu ghi dữ liệu vào APB slave, cầu nối AXI – APB sẽ xử lý tín hiệu từ phía AXI sang APB. Vì tần số phía ngoại vi APB là 25MHz so với tần số phía AXI là 250MHz, chênh lệch 10 lần nên việc ghi ở phía APB diễn ra dài hơn.

- (5) Tại thời điểm 140ns ở ô T5, với địa chỉ 0x00001xxx sẽ thuộc về slave 8-bit nên tín hiệu PSEL của APB-SLAVE 8-bit được nâng lên mức 1, đồng thời PWRITE = 1 để cho phép ghi vào SLAVE, địa chỉ PADDR bắt đầu sẽ được lấy từ FIFO của kênh AW, còn PWDATA sẽ được lấy từ FIFO kênh W, tín hiệu PSTRB = F để làm mặt nạ để ghi 32-bit dữ liệu. Tại 180ns, PENABLE được nâng lên mức 1, lúc này data từ kênh PWDATA sẽ được ghi vào bộ nhớ. Ở chu kỳ xung lock tiếp theo 220ns, PENABLE được kích xuống và PWDATA sẽ lấy dữ liệu thứ 2, còn PADDR sẽ tăng lên 4 do ô nhớ của APB-SLAVE 8-bit có độ dài là 8-bit, cần tới 4 ô nhớ để có thể chứa chuỗi dữ liệu 32-bit. Vì thế PADDR = 0x0000101C sẽ lưu gói tin 0x12153524, tăng lên 0x00001020 vì các ô nhớ 0x0000101C sẽ lưu 0x24, 0x0000101D sẽ lưu 0x35, 0x0000101E lưu 0x15, 0x0000101F lưu 0x12 đã chứa đủ một khung dữ liệu 32-bit. Tại 260ns, PENABLE = 1 để tiếp tục

ghi tiếp gói tin thứ 2. Lần lượt là tại thời điểm 340ns, 420ns, 500ns, 580ns, 660ns, 740ns sẽ ghi gói tin thứ 3, thứ 4, thứ 5, thứ 6, thứ 7 và thứ 8 và PADDR tiếp tục tăng lên 4, 0x00001024 là gói tin thứ 3, 0x00001028 là gói tin thứ 4, 0x0000102C là gói tin thứ 5, 0x00001030 là gói tin thứ 6, 0x00001034 là gói tin thứ 7, 0x00001038 là gói tin thứ 8.

- (6) Ô T6 sẽ là tại thời điểm 780ns, thì tất cả các tín hiệu tại APB sẽ được tắt. Tại thời điểm 820ns, ở kênh B thì tín hiệu BVALID được kích lên mức 1 và tín hiệu BID = 0xC8 được truyền đi để phía AXI nhận được báo hiệu tăng đã thực hiện xong quá trình ghi dữ liệu với mã của giao dịch ghi là **0x8C**.

Sau khi đã thực hiện quy trình ghi vào APB-SLAVE, bên phía AXI trước đó đã gửi một truy cập đọc, nên tiếp theo đó bên phía APB-SLAVE bắt đầu gửi những dữ liệu đọc dữ trên những thông tin điều khiển đã có trước đó ở kênh AR như hình 4.3 đã mô tả



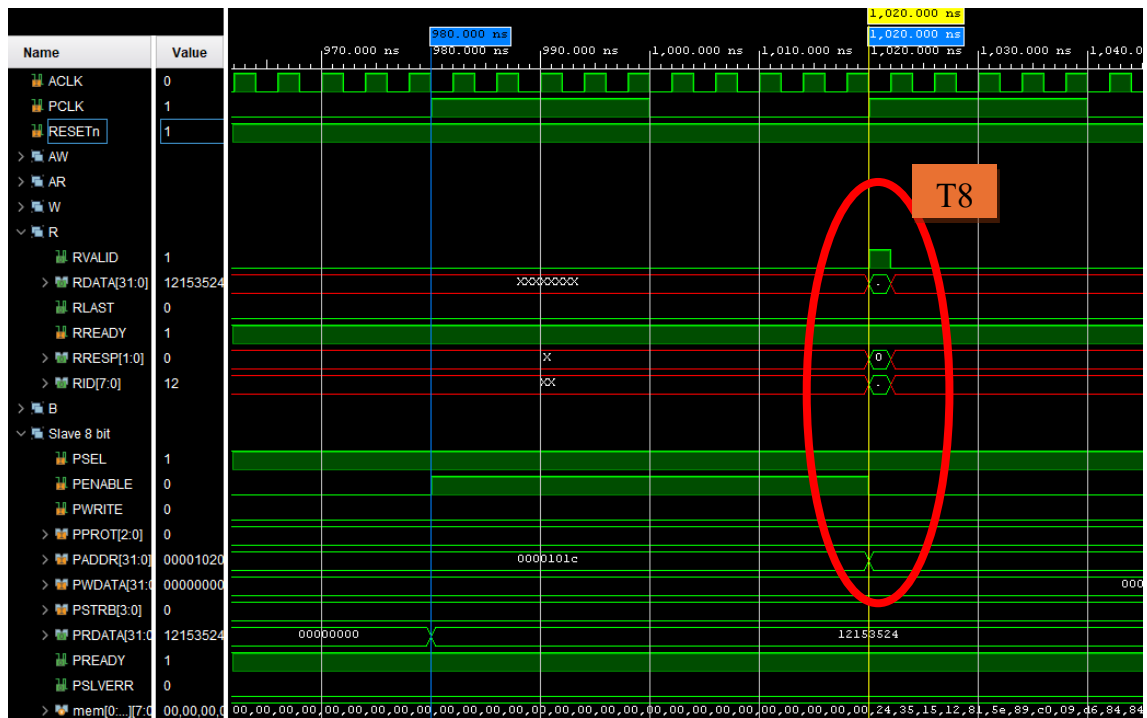
Hình 4.3 Dạng sóng quá trình đọc ở ngoại vi APB 8-bit

- (7) Tại ô T7, PSEL = 1 để việc chọn SLAVE 8-bit này để truy cập do có địa chỉ tương ứng, PADDR = 0x0000101C có được từ thông tin địa chỉ từ kênh AR, PWRITE = 0 để biểu thị quá trình đọc từ APB-SLAVE ra. Tại thời điểm 980ns, PENABLE = 1, báo hiệu rằng SLAVE sẵn sàng truyền thông

tin nên PRDATA sẽ có dữ liệu và PRDATA = 0x12153524, đọc từ địa chỉ ô nhớ đầu tiên bắt đầu từ 0x0000101C. Các địa chỉ tiếp theo sẽ là 0x00001020, 0x00001024, 0x00001028, 0x0000102C, 0x00001030, 0x000034, 0x00001038.

- (8) Tại thời điểm 1020ns ở ô T8, ở bên phía tín hiệu PENABLE được hạ xuống và thông tin địa chỉ ở PADDR cũng thay đổi để phục vụ việc đọc gói tin thứ 2, đồng thời tín hiệu xung clock tại thời gian này cũng truyền dữ liệu từ PRDATA sang kênh RDATA ở phía AXI, với địa chỉ 0x0000101C thì dữ liệu sẽ là 0x12153524, cùng lúc đó tín hiệu RVALID = 1 để báo hiệu rằng có dữ liệu trong FIFO kênh R để có thể đọc ra, RID = 12 là thông tin từ tín hiệu ARID, RRESP = 0 báo hiệu rằng việc đọc không bị lỗi gì. Tuy nhiên gói tin này chỉ tồn tại ở một chu kỳ xung clock ở phía AXI. Sau khi có tín hiệu RVALID cùng với RDATA, ở tại xung clock AXI tiếp theo thì phía AXI sẽ nhận được data. Ở các chu kỳ xung clock PCLK tiếp theo, các địa chỉ mới được nạp vào kênh PADDR sao cho đúng với thông số của kênh AR. 8 gói tin lần lượt sẽ là, gói tin đầu tiên có địa chỉ 0x0000101C là 0x12153524, gói tin thứ 2 có địa chỉ 0x00001020 là 0xC0895E81, gói tin thứ 3 là 0x00001024 có dữ liệu 0x8484D609, gói tin thứ 4 có địa chỉ 0x00001028 là 0xB1F05663, thứ 5 là 0x0000102C có dữ liệu 0x06B97B0D, gói tin thứ 6 có gói tin 0x46DF998D tại ô địa chỉ 0x00001030, gói tin thứ 7 có địa chỉ 0x00001034 có dữ liệu 0xB2C28465, và gói tin thứ 8 tại địa chỉ 0x00001038 mang gói tin 0x89375212.
- (9) Tại T9, sau khi thực hiện đọc 8 gói tin từ trong APB-SLAVE tại thời điểm 1580ns, RLAST được kích lên mức 1 để báo hiệu rằng kết thúc phiên đọc từ APB_SLAVE và tồn tại trong 1 chu kỳ xung clock PCLK. Tín hiệu phản hồi RLAST được kích hoạt theo xung của PCLK vì RLAST được đi chung với kênh FIFO_R, đồng thời dựa vào tín hiệu phát ra khi đã truyền xong số lượng gói tin yêu cầu.

Quá trình ghi ở ngoại vi APB 8 bit được biểu diễn qua hình 4.4



Hình 4.4 Dạng sóng thông tin đọc phóng to ở phía AXI tại thời điểm T8

Hình 4.4 là mô tả gói tin ở tín hiệu RDATA đã được phóng to tại ô T8, với dữ liệu RDATA, PCLK kích lên làm dữ liệu được truyền từ tín hiệu PRDATA đến RDATA. Tuy nhiên chỉ cần gặp cạnh lên của xung clock ACLK từ phía AXI thì dữ liệu sẽ được lấy đi, vì thế RDATA tồn tại trong khoảng thời gian rất ngắn như đã giải thích ở trên.

Bảng kết quả quá trình ghi của cầu nối được thể hiện trong hình 4.5:

COMPARE TABLE (WRITE)						
AWADDR : 0000101c						
BURST TYPE : INCR						
BURST LENGTH : 8						
BURST SIZE : 4 byte						
Master: AXI -> Slave: APB						
PACKET	WDATA (AXI)	PWDATA (APB)	PADDR (APB)	STATUS		
[0]	00f3e301	00f3e301	0000101c	PASS		
[1]	06d7cd0d	06d7cd0d	00001020	PASS		
[2]	3b23f176	3b23f176	00001024	PASS		
[3]	1e8dcd3d	1e8dcd3d	00001028	PASS		
[4]	76d457ed	76d457ed	0000102c	PASS		
[5]	462df78c	462df78c	00001030	PASS		
[6]	7cfde9f9	7cfde9f9	00001034	PASS		
[7]	e33724c6	e33724c6	00001038	PASS		

Hình 4.5 Bảng kết quả quá trình ghi từ mô hình UVM

Hình 4.5 là bảng kết quả cho chu kỳ ghi từ phía AXI đến APB với mô hình kiểm tra UVM. Với các thông tin được liệt kê ở hình trên là AWADDR, BURST TYPE (kiểu truyền tương ứng với tín hiệu AWBURST), BURST LENGTH (số lượng gói tin trong một khung truyền, tương ứng với tín hiệu AWLEN), BURST SIZE (kích thước dữ liệu của gói tin, tương ứng với tín hiệu AWSIZE). Đồng thời cũng hiển thị phía AXI hiện tại đang là nguồn gửi thông tin, phía APB là đích đến ở giao dịch ghi. Ở bảng, ta thấy cột đầu tiên là số thứ tự gói tin, cột thứ hai là dữ liệu từ kênh WDATA, cột thứ ba là dữ liệu từ kênh PWDATA sau khi đã qua cầu nối giao tiếp AXI – APB, cột tiếp theo biểu thị địa chỉ PADDR ở phía ngoại vi tương ứng với từng gói tin, cột cuối cùng cho thấy rằng là việc truyền có bị thất bại hay thành công. Việc truyền thành công hay thất bại dựa vào việc so sánh dữ liệu ở hai kênh PWDATA và WDATA, đồng thời xem xét địa chỉ PADDR có khớp để chứa hoàn toàn được chuỗi dữ liệu 32-bit hay không. Qua bảng trên ta có được cái nhìn tổng quan về chức năng cũng như kiểm tra chức năng của giao dịch ghi của cầu nối AXI – APB.

Bảng kết quả quá trình ghi của cầu nối được thể hiện trong hình 4.6:

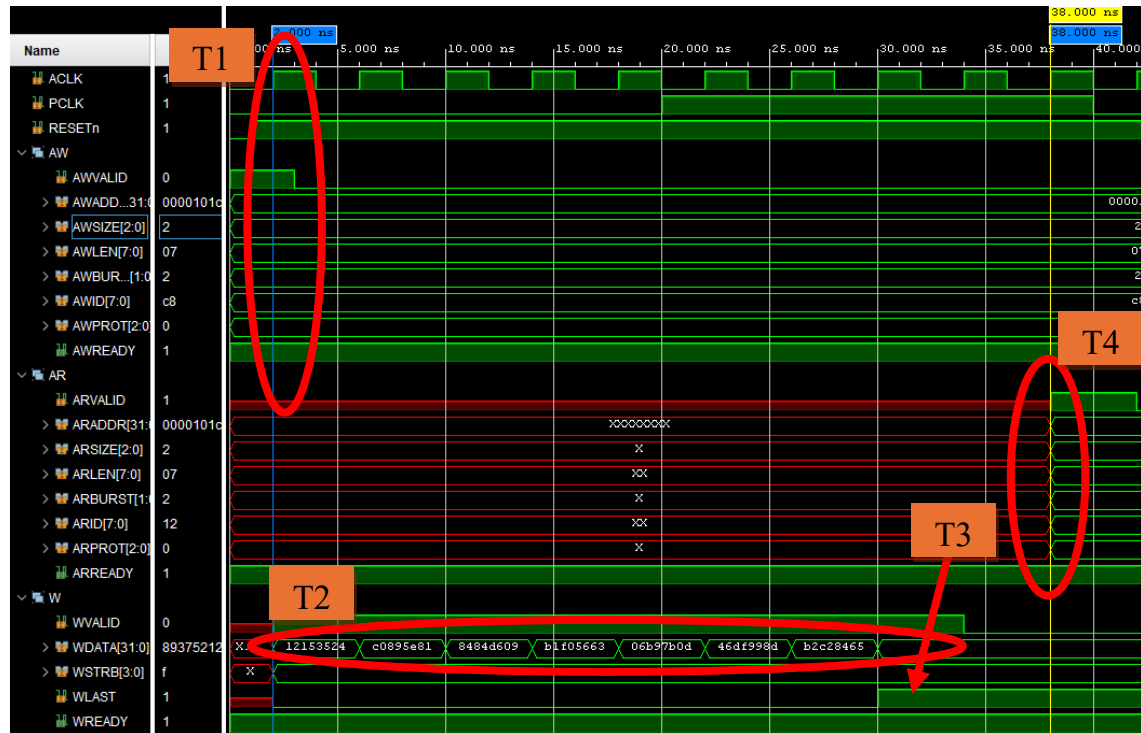
COMPARE TABLE (READ)					
ARADDR : 0000101c					
BURST TYPE : INCR					
BURST LENGTH : 8					
BURST SIZE : 4 byte					
Master: APB -> Slave: AXI					
PACKET	RDATA (AXI)	PRDATA (APB)	PADDR (APB)	STATUS	
[0]	00f3e301	00f3e301	0000101c	PASS	
[1]	06d7cd0d	06d7cd0d	00001020	PASS	
[2]	3b23f176	3b23f176	00001024	PASS	
[3]	1e8dcd3d	1e8dcd3d	00001028	PASS	
[4]	76d457ed	76d457ed	0000102c	PASS	
[5]	462df78c	462df78c	00001030	PASS	
[6]	7cfde9f9	7cfde9f9	00001034	PASS	
[7]	e33724c6	e33724c6	00001038	PASS	

Hình 4.6 Bảng kết quả quá trình đọc của mô hình UVM

Hình 4.6 là bảng kết quả cho chu kỳ đọc từ phía AXI đến APB với mô hình kiểm tra UVM. Với các thông tin được ở hình trên liệt kê là ARADDR, BURST TYPE (kiểu truyền, tương ứng với tín hiệu ARBURST), BURST LENGTH (số lượng gói tin trong một khung truyền, tương ứng với tín hiệu ARLEN), BURST SIZE (kích thước dữ liệu của gói tin, tương ứng với tín hiệu ARSIZE). Đồng thời cũng hiển thị phía APB hiện tại đang là nguồn gửi thông tin, phía AXI là đích đến ở giao dịch đọc. Ở bảng, ta thấy cột đầu tiên là số thứ tự gói tin, cột thứ hai là dữ liệu từ kênh RDATA sau khi đã qua cầu nối giao tiếp AXI – APB, cột thứ ba là dữ liệu từ kênh PRDATA, cột tiếp theo biểu thị địa chỉ PADDR ở phía ngoại vi tương ứng với từng gói tin, cột cuối cùng cho thấy rằng là việc truyền có bị thất bại hay thành công. Việc truyền thành công hay thất bại dựa vào việc so sánh dữ liệu ở hai kênh PRDATA và RDATA. Thêm vào đó, xem xét địa chỉ PADDR có khớp để chứa hoàn toàn được chuỗi dữ liệu 32-bit hay không. Qua bảng trên ta có được cái nhìn tổng quan về chức năng cũng như kiểm tra chức năng của giao dịch đọc của cầu nối AXI – APB.

4.2.2 Testcase2: AXI – APB ghi đọc dữ liệu 32-bit vào slave 8-bit chế độ WRAP với $BURST_SIZE = 8$, $BURST_LENGTH = 32$

Kết quả truyền dữ liệu ở Testcase2 - AXI – APB ở quá trình ghi và đọc với dữ liệu 32-bit và slave 8-bit chế độ WRAP với thông số $BURST_SIZE = 8$, $BURST_LENGTH = 32$ được thể hiện qua hình 4.7:



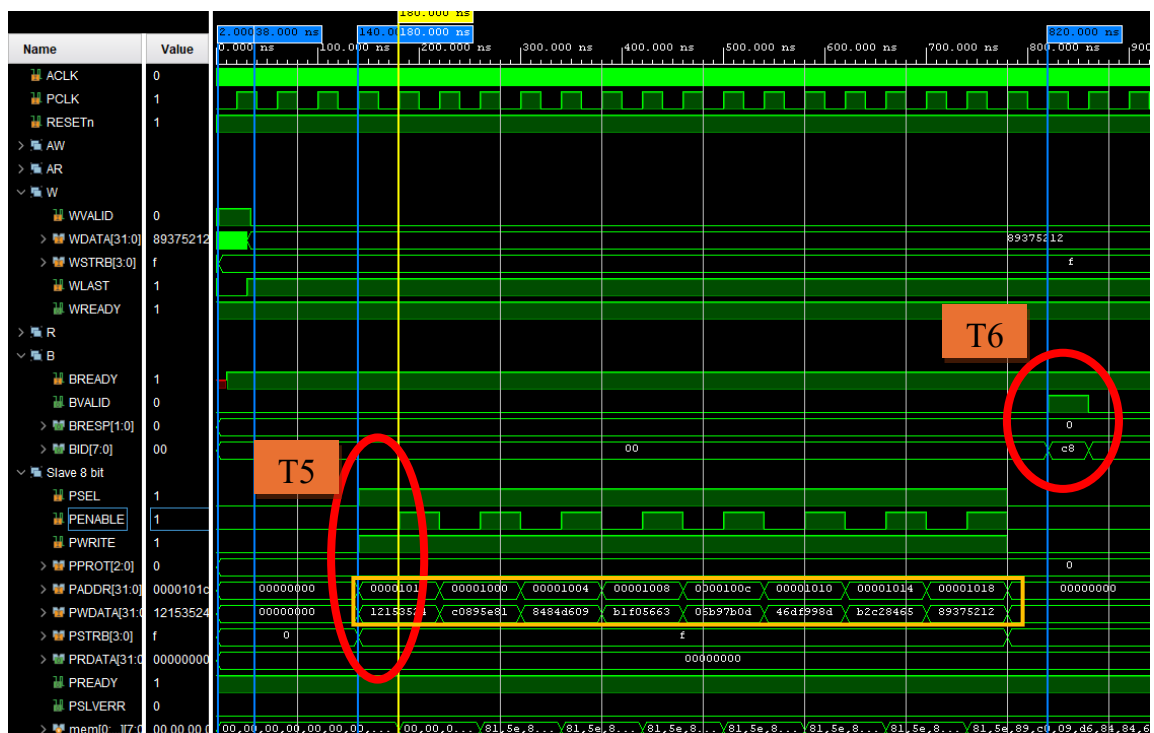
Hình 4.7 Dạng sóng tín hiệu chuẩn bị quá trình đọc và ghi

Quá trình thiết lập tín hiệu được thể hiện ở hình 4.7:

- (1) Tại ô T1, từ 0ns đến 1ns, $RESETn = 0$ để tiến hành khởi động lại hệ thống, sau đó $RESETn = 1$ để hệ thống bắt đầu hoạt động, $AWREADY$, $ARREADY$, $WREADY$ luôn giữ ở mức cao để cho phép truyền, nhận tín hiệu. Ở thời điểm 2 ns, các tín hiệu thiết lập cho câu nối thực hiện quá trình ghi được bắt đầu. Với việc có cạnh lên cùng xung clock cùng những tín hiệu điều khiển ở kênh AW, lúc này địa chỉ bắt đầu $AWADDR$ bằng $0x0000101C$, kích thước của gói tin $AWSIZE = 2$ tức là kích thước gói tin là 32-bit (4-byte), $AWLEN = 7$ tức là sẽ có 8 gói tin trong một khung truyền, $AWBURST = 2$ tức là chọn kiểu truyền địa chỉ là WRAP theo chuẩn AMBA, $AWID$ là $0xC8$ để mã giao dịch.

- (2) Ở tại ô T2, ngay tại thời điểm cạnh lên xung clock đó thì cầu nối cũng sẽ nhận 8 gói tin lần lượt là 0x1215324, 0xC0895E81, 0x8484d609, 0xB1F05663, 0x06B97B0D, 0x46DF998D, 0xB2C28465, 0x89375212, 8 gói tin 32-bit ở kênh W, trùng khớp với thông tin từ kênh AW, AWLEN = 7, AWSIZE = 2. Tín hiệu WSTRB = 0x0F làm mặt nạ 32-bit dữ liệu, tức là sẽ sử dụng hết 32-bit của WDATA. Tại 2ns là gói tin đầu tiên, ở 6ns là gói tin thứ 2 và tiếp tục ở mỗi chu kỳ cho đến gói tin thứ 8.
- (3) Khi gửi gói tin thứ 8, gói tin cuối cùng ở vị trí 30ns tại T3, tín hiệu WLAST sẽ được kích hoạt lên mức 1 cùng lúc báo hiệu rằng đây là gói tin cuối cùng của khung dữ liệu truyền đi. Ở 34ns, tín hiệu WVALID = 0 đánh dấu cho việc kết thúc việc truyền dữ liệu ghi ở phía AXI.
- (4) Tại thời điểm 38ns ở ô T4, phía AXI sẽ phát tín hiệu MASTER muốn đọc dữ liệu với các thông số ở kênh AR như sau: địa chỉ bắt đầu ARADDR bằng 0x0000101C, kích thước của gói tin ARSIZE = 2 độ dài dữ liệu là 32-bit, ARLEN = 7 trong một chuỗi sẽ đọc 8 gói tin, ARBURST = 2 chọn kiểu truyền địa chỉ là WRAP, ARID = 12.

Quá trình ghi ở APB 8 bit được thể hiện ở hình 4.8:



Hình 4.8 Dạng sóng quá trình ghi ở APB 8-bit

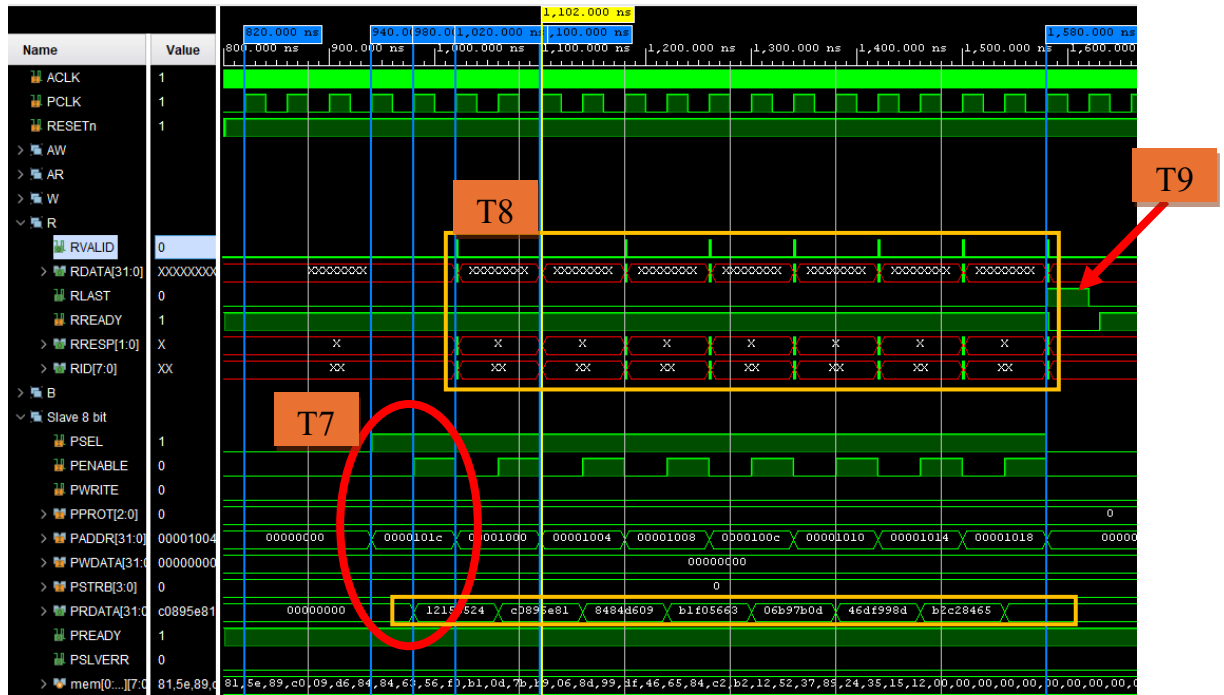
Theo mô tả từ hình 4.8, sau khi đã nhận được tín hiệu ghi dữ liệu vào APB slave, cầu nối AXI – APB sẽ xử lý tín hiệu từ phía AXI sang APB. Vì tần số phía ngoại vi APB là 25MHz so với tần số phía AXI là 250MHz, chênh lệch 10 lần nên việc ghi ở phía APB diễn ra dài hơn.

- (5) Tại thời điểm 140ns tại T4, với địa chỉ 0x00001xxx sẽ thuộc về slave 8-bit nên tín hiệu PSEL của APB-SLAVE 8-bit được nâng lên mức 1, đồng thời PWRITE = 1 để cho phép ghi vào SLAVE, địa chỉ PADDR bắt đầu sẽ được lấy từ FIFO của kênh AW, còn PWDATA sẽ được lấy từ FIFO kênh W, tín hiệu PSTRB = F để làm mặt nạ để ghi 32-bit dữ liệu. Tại 180ns, PENABLE được nâng lên mức 1, lúc này data từ kênh PWDATA sẽ được ghi vào bộ nhớ. Ở chu kỳ xung lock tiếp theo 220ns, PENABLE được kích xuống và PWDATA sẽ lấy dữ liệu thứ 2, còn PADDR sẽ tăng lên 4 do ô nhớ và quay trở lại địa chỉ đầu tiên của một khung là 0x00001000 của APB-SLAVE 8-bit có độ dài là 8-bit, cần tới 4 ô nhớ để có thể chứa chuỗi dữ liệu 32-bit. Dựa vào kiểu truyền WRAP cùng số lượng gói tin được quy định bởi AWLEN, ta có thể tính được rằng là 8 gói tin AXI cần có 0x20 địa chỉ có thể chứa hoàn toàn, cơ chế WRAP không cho địa chỉ đi quá phạm vi đó. Do địa chỉ đầu tiên phía AXI truyền vào là 0x0000101C nằm trong phạm vi từ 0x00001000 đến 0x00001020, do vậy PADDR = 0x0000101C sẽ tăng lên 0x00001000 vì các ô nhớ 0x0000101C, 0x0000101D, 0x0000101E, 0x0000101F đã chứa đủ một khung dữ liệu 32-bit, sau đó quay trở ngược lại 0x00001000. Tại 260ns, PENABLE = 1 để tiếp tục ghi tiếp gói tin thứ 2 tại địa chỉ 0x00001000. Lần lượt là tại thời điểm 340ns, 420ns, 500ns, 580ns, 660ns, 740ns sẽ ghi gói tin thứ 3, thứ 4, thứ 5, thứ 6, thứ 7 và thứ 8 và PADDR tiếp tục tăng lên 4, 0x00001004 là gói tin thứ 3, 0x00001008 là gói tin thứ 4, 0x0000100C là gói tin thứ 5, 0x00001010 là gói tin thứ 6, 0x00001014 là gói tin thứ 7, 0x00001018 là gói tin thứ 8.
- (6) Ở ô T6, thời điểm 780ns, thì tất cả các tín hiệu tại APB sẽ được tắt. Tại thời điểm 820ns, ở kênh B thì tín hiệu BVALID sẽ được đưa lên mức 1 và tín

hiệu BID = 0xC8 được truyền đi để phía AXI nhận được báo hiệu tăng đã thực hiện xong quá trình ghi dữ liệu với mã của giao dịch ghi là 0x8C.

Sau khi đã thực hiện quy trình ghi vào APB-SLAVE, bên phía AXI trước đó đã gửi một truy cập đọc, nên tiếp theo đó bên phía APB-SLAVE bắt đầu gửi những dữ liệu đọc dữ trên những thông tin điều khiển đã có trước đó ở kênh AR.

Quá trình đọc ở ngoại vi APB 8 bit được thể hiện ở hình 4.9:



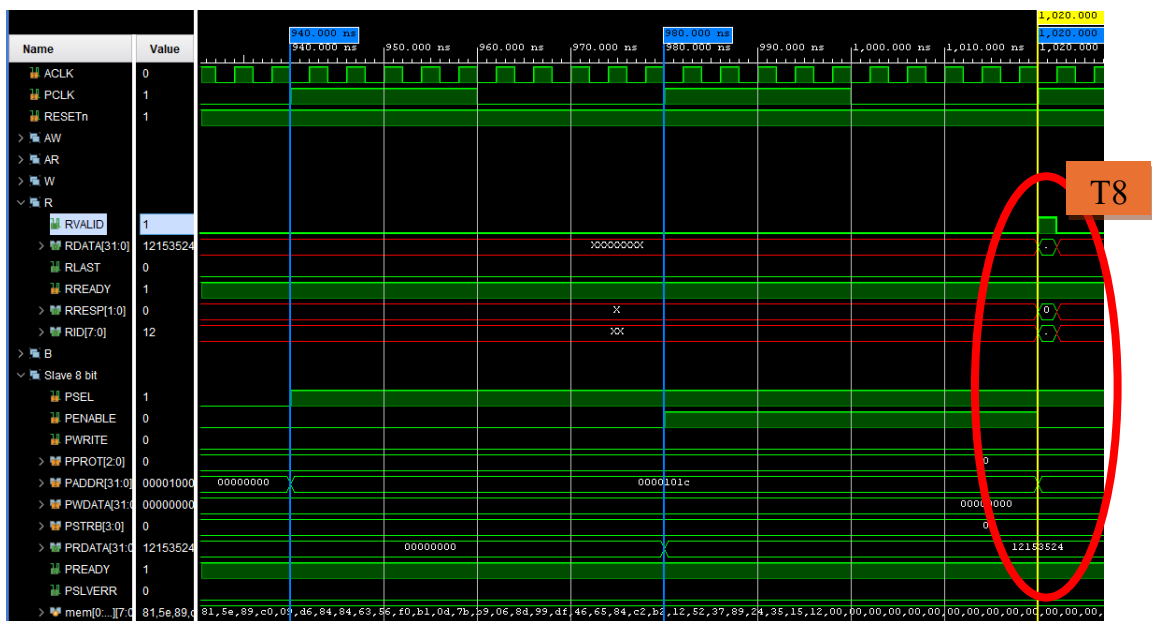
Hình 4.9 Dạng sóng quá trình đọc ở ngoại vi APB 8-bit

(7) Tại thời điểm 940ns, tại T7, PSEL = 1 để việc chọn SLAVE này để truy cập, PADDR = 0x0000101C có được từ thông tin địa chỉ từ kênh AR, PWRITE = 0 để biểu thị quá trình đọc từ APB-SLAVE ra. Tại thời điểm 980ns, PENABLE = 1, báo hiệu rằng SLAVE sẵn sàng truyền thông tin nên PRDATA sẽ có dữ liệu và PRDATA = 0x12153524, đọc từ ô địa chỉ bắt đầu 0x0000101C vì mỗi ô nhớ chỉ chứa được dữ liệu 8-bit.

(8) Tại thời điểm 1020ns ở ô T8, ở bên phía tín hiệu PENABLE được hạ xuống và thông tin địa chỉ ở PADDR cũng thay đổi theo kiểu truyền WRAP đã được đề cập ở truy cập ghi của cầu nối để phục vụ việc đọc gói tin thứ 2, đồng thời tín hiệu xung clock tại thời gian này cũng truyền dữ liệu từ PRDATA sang kênh RDATA ở phía AXI, cùng lúc đó tín hiệu RVALID = 1 để báo hiệu rằng có dữ liệu trong FIFO kênh R để có thể đọc ra, RID =

12 là thông tin từ tín hiệu ARID, BRESP = 0 báo hiệu rằng việc đọc không bị lỗi gì. Ở các chu kỳ xung clock PCLK tiếp theo, các địa chỉ mới được nạp vào kênh PADDR sao cho đúng với thông số của kênh AR và theo kiểu truyền WRAP của AXI. 8 gói tin lần lượt sẽ là, gói tin đầu tiên có địa chỉ 0x0000101C là 0x12153524, gói tin thứ 2 có địa chỉ 0x00001000 là 0xC0895E81, gói tin thứ 3 là 0x00001004 có dữ liệu 0x8484D609, gói tin thứ 4 có địa chỉ 0x00001008 là 0xB1F05663, thứ 5 là 0x0000100C có dữ liệu 0x06B97B0D, gói tin thứ 6 có gói tin 0x46DF998D tại ô địa chỉ 0x00001010, gói tin thứ 7 có địa chỉ 0x00001014 có dữ liệu 0xB2C28465, và gói tin thứ 8 tại địa chỉ 0x00001018 mang gói tin 0x89375212.

- (9) Tại T9, sau khi thực hiện ghi 8 gói tin từ trong APB-SLAVE ra tại thời điểm 1580ns, RLAST được kích lên mức 1 để báo hiệu rằng kết thúc phiên đọc từ APB_SLAVE và tồn tại trong 1 chu kỳ xung clock PCLK.



Hình 4.10 Dạng sóng thông tin đọc phóng to ở phía AXI thời điểm T8

Hình 4.10 là hình một gói tin ở tín hiệu RDATA đã được phóng to tại ô T8. Với dữ liệu RDATA, PCLK kích lên làm dữ liệu được truyền từ tín hiệu PRDATA đến RDATA. Tuy nhiên chỉ cần gặp cạnh lên của xung clock ACLK từ phía AXI thì dữ liệu sẽ được lấy đi, vì thế RDATA tồn tại trong khoảng thời gian rất ngắn.

Kết quả quá trình ghi được thể hiện ở hình 4.11:

COMPARE TABLE (WRITE)					
AWADDR : 0000101c					
BURST TYPE : WRAP					
BURST LENGTH : 8					
BURST SIZE : 4 byte					
Master: AXI -> Slave: APB					
PACKET	WDATA (AXI)	PWDATA (APB)	PADDR (APB)	STATUS	
[0]	00f3e301	00f3e301	0000101c	PASS	
[1]	06d7cd0d	06d7cd0d	00001000	PASS	
[2]	3b23f176	3b23f176	00001004	PASS	
[3]	1e8dcd3d	1e8dcd3d	00001008	PASS	
[4]	76d457ed	76d457ed	0000100c	PASS	
[5]	462df78c	462df78c	00001010	PASS	
[6]	7cfde9f9	7cfde9f9	00001014	PASS	
[7]	e33724c6	e33724c6	00001018	PASS	

Hình 4.11 Bảng kết quả quá trình ghi từ mô hình UVM

Hình 4.11 là bảng kết quả cho chu kỳ ghi từ phía AXI đến APB với mô hình kiểm tra UVM. Với các thông tin được liệt kê ở hình trên là AWADDR, BURST TYPE (kiểu truyền tương ứng với tín hiệu AWBURST), BURST LENGTH (số lượng gói tin trong một khung truyền, tương ứng với tín hiệu AWLEN), BURST SIZE (kích thước dữ liệu của gói tin tương ứng với tín hiệu AWSIZE). Đồng thời cũng hiển thị phía AXI hiện tại đang là nguồn gửi thông tin, phía APB là đích đến ở giao dịch ghi. Ở bảng, ta thấy cột đầu tiên là số thứ tự gói tin, cột thứ hai là dữ liệu từ kênh WDATA, cột thứ ba là dữ liệu từ kênh PWDATA sau khi đã qua cầu nối giao tiếp AXI – APB, cột tiếp theo biểu thị địa chỉ PADDR ở phía ngoại vi tương ứng với từng gói tin, cột cuối cùng cho thấy rằng là việc truyền có bị thất bại hay thành công. Việc truyền thành công hay thất bại dựa vào việc so sánh dữ liệu ở hai kênh PWDATA và WDATA, đồng thời xem xét địa chỉ PADDR có khớp để chứa hoàn toàn được chuỗi dữ liệu 32-bit hay không. Qua bảng trên ta có được cái nhìn tổng quan về chức năng cũng như kiểm tra chức năng của giao dịch ghi của cầu nối AXI – APB.

Bảng kết quả quá trình đọc của cầu nối được thể hiện trong hình 4.12:

COMPARE TABLE (READ)						
ARADDR : 0000101c						
BURST TYPE : WRAP						
BURST LENGTH : 8						
BURST SIZE : 4 byte						
Master: APB -> Slave: AXI						
PACKET	RDATA (AXI)	PRDATA (APB)	PADDR (APB)	STATUS		
[0]	00f3e301	00f3e301	0000101c	PASS		
[1]	06d7cd0d	06d7cd0d	00001000	PASS		
[2]	3b23f176	3b23f176	00001004	PASS		
[3]	1e8dcd3d	1e8dcd3d	00001008	PASS		
[4]	76d457ed	76d457ed	0000100c	PASS		
[5]	462df78c	462df78c	00001010	PASS		
[6]	7cfde9f9	7cfde9f9	00001014	PASS		
[7]	e33724c6	e33724c6	00001018	PASS		

Hình 4.12 Bảng kết quả cho quá trình đọc từ mô hình UVM

Hình 4.12 là bảng kết quả cho chu kỳ đọc từ phía AXI đến APB với mô hình kiểm tra UVM. Với các thông tin được ở hình trên liệt kê là ARADDR, BURST TYPE (kiểu truyền, tương ứng với tín hiệu ARBURST), BURST LENGTH (số lượng gói tin trong một khung truyền, tương ứng với tín hiệu ARLEN), BURST SIZE (kích thước dữ liệu của gói tin, tương ứng với tín hiệu ARSIZE). Ở bảng, ta thấy cột đầu tiên là số thứ tự gói tin, cột thứ hai là dữ liệu từ kênh RDATA sau khi đã qua cầu nối giao tiếp AXI – APB, cột thứ ba là dữ liệu từ kênh PRDATA, cột tiếp theo biểu thị địa chỉ PADDR ở phía ngoại vi tương ứng với từng gói tin, cột cuối cùng cho thấy rằng là việc truyền có bị thất bại hay thành công. Việc truyền thành công hay thất bại dựa vào việc so sánh dữ liệu ở hai kênh PRDATA và RDATA, đồng thời xem xét địa chỉ PADDR có khớp để chứa hoàn toàn được chuỗi dữ liệu 32-bit hay không. Qua bảng trên ta có được cái nhìn tổng quan về chức năng cũng như kiểm tra chức năng của giao dịch đọc của cầu nối AXI – APB.

Để đánh giá hệ thống và kiểm thử chức năng của hệ thống trong trường hợp testcase2, ta cần đánh giá những hạng mục sau:

- Cơ chế bắt tay 2 chiều ở các kênh: Dựa vào lý thuyết ở chương 2 phần giao thức truyền thông AXI, nhóm nghiên cứu đã sử dụng đúng cơ chế bắt tay

VALID và READY, chỉ khi có cả 2 tín hiệu VALID và READY thì mới tiến hành truyền thông tin vào.

- Kiểm tra thông tin điều khiển, xem xét dữ liệu ngõ ra có trùng khớp với dữ liệu ngõ vào. Qua dạng sóng mô phỏng, cùng với những phân tích ở phía trên ta thấy được. Việc tín hiệu bộ nhớ của ngoại vi bị thay đổi chứng tỏ được rằng đã ghi vô được ngoại vi. Đồng thời kiểu truyền là WRAP, dựa vào dạng sóng và bảng kết quả từ UVM, nhóm nghiên cứu rút ra được việc địa chỉ có tăng lên đúng không như đã đề cập ở phần thiết kế hệ thống xử lý tính toán địa chỉ cơ chế INCR và WRAP. Địa chỉ ban đầu đã tăng từ 0x0000101C đến 0x00001018, khoảng độ dài khung truyền là 0x20, lấy giá trị đó thêm vào địa chỉ ban đầu sẽ vượt quá khung nên địa chỉ sẽ quay lại 0x00001000 vì địa chỉ ban đầu là 0x1C sẽ là 0x0001_1100, chỉ thay đổi 5 bit cuối của địa chỉ như đã đề cập ở cơ chế WRAP chương 3. Địa chỉ tiếp theo tiếp tục tăng như thông thường, địa chỉ của gói tin cuối cùng sẽ là 0x00001018, 8 gói tin này sẽ ở địa chỉ từ 0x00001000 đến 0x0000102C, vậy việc tính toán địa chỉ trong kiểu truyền WRAP đã đúng.
- Dữ liệu đọc ghi: Dựa vào tín hiệu, ta đã ghi vào 8 gói tin có dữ liệu như sau: 0x1215324, 0xC0895E81, 0x8484D609, 0xB1F05663, 0x06B97B0D, 0x46DF998D, 0xB2C28465, 0x89375212. Và khi đọc dữ liệu ra từ ngoại vi thì dữ liệu là: 0x1215324, 0xC0895E81, 0x8484D609, 0xB1F05663, 0x06B97B0D, 0x46DF998D, 0xB2C28465, 0x89375212. Do đó, việc ghi vào ngoại vi và đọc ra đã thành công.

4.2.3 Testcase3: AXI – APB ghi đọc dữ liệu 32-bit vào slave 16-bit chế độ WRAP với $BURST_SIZE = 32$, $BURST_LENGTH = 8$

Kết quả truyền dữ liệu ở Testcase3 - AXI – APB ở quá trình ghi và đọc với dữ liệu 32-bit và slave 16-bit chế độ WRAP với thông số $BURST_SIZE = 8$, $BURST_LENGTH = 32$ được thể hiện ở hình 4.13:



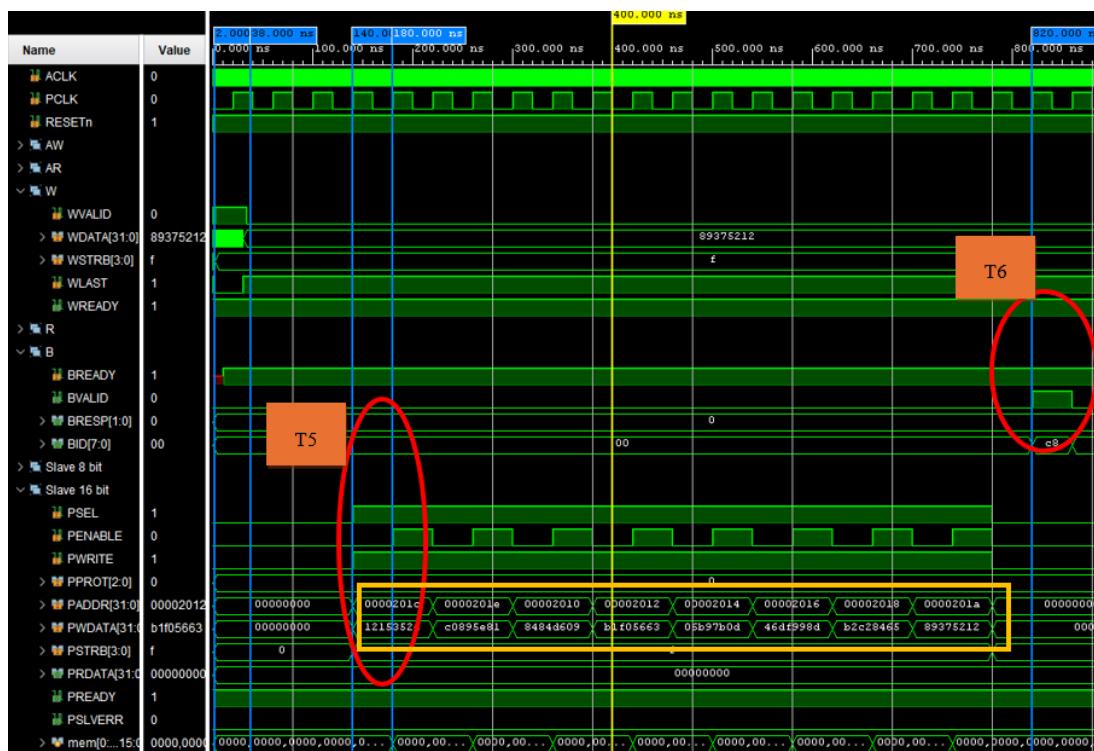
Hình 4.13 Dạng sóng tín hiệu chuẩn bị quá trình ghi/đọc ở ngoại vi 16-bit

Quá trình thiết lập tín hiệu được thể hiện ở hình 4.13:

- (1) Từ 0ns đến 1ns, RESETn = 0 để tiến hành khởi động lại hệ thống, sau đó RESETn = 1 để hệ thống bắt đầu hoạt động, AWREADY, ARREADY, WREADY luôn giữ ở mức cao để cho phép truyền, nhận tín hiệu. Ở thời điểm 2 ns, các tín hiệu thiết lập cho cầu nối thực hiện quá trình ghi được bắt đầu. Với việc có cạnh lên cùng xung clock cùng những tín hiệu điều khiển ở kênh AW, lúc này địa chỉ bắt đầu AWADDR bằng 0x0000201C, địa chỉ của ngoại vi 16-bit, kích thước của gói tin AWSIZE = 2 tức là kích thước gói tin là 32-bit (4-byte), AWLEN = 7 tức là sẽ có 8 gói tin trong một lần truyền, AWBURST = 2 tức là chọn kiểu truyền địa chỉ là WRAP theo chuẩn AMBA, AWID là 0xC8 để mã giao dịch.

- (2) Ngay tại thời điểm cạnh lên xung clock đó thì cầu nối cũng sẽ nhận 8 gói tin 32-bit ở kênh W, lần lượt là lần lượt là 0x1215324, 0xC0895E81, 0x8484d609, 0xB1F05663, 0x06B97B0D, 0x46DF998D, 0xB2C28465, 0x89375212, trùng khớp với thông tin từ kênh AW. Tín hiệu WSTRB = 0x0F làm mặt nạ 32-bit dữ liệu, tức là sẽ sử dụng hết 32-bit của WDATA. Tại 2ns là gói tin đầu tiên, ở 6ns là gói tin thứ 2 và tiếp tục ở mỗi chu kỳ cho đến gói tin thứ 8.
- (3) Khi gửi gói tin cuối cùng ở vị trí T3, tín hiệu WLAST sẽ được kích hoạt lên mức 1 cùng lúc báo hiệu rằng đây là gói tin cuối cùng của khung dữ liệu truyền đi. Ở 34ns, tín hiệu WVALID = 0 đánh dấu cho việc kết thúc việc truyền dữ liệu ghi ở phía AXI.
- (4) Ở T4, tại thời điểm 38ns, ở phía kết nối AXI sẽ phát tín hiệu rằng phía MASTER muốn đọc dữ liệu với các thông số ở kênh AR như sau: địa chỉ bắt đầu ARADDR bằng 0x0000201C, kích thước của gói tin ARSIZE = 2 độ dài dữ liệu là 32-bit, ARLEN = 7 trong một chuỗi sẽ đọc 8 gói tin, ARBURST = 2 chọn kiểu truyền địa chỉ là WRAP, ARID = 12.

Quá trình ghi ở ngoại vi APB 16 bit được thể hiện ở hình 4.14



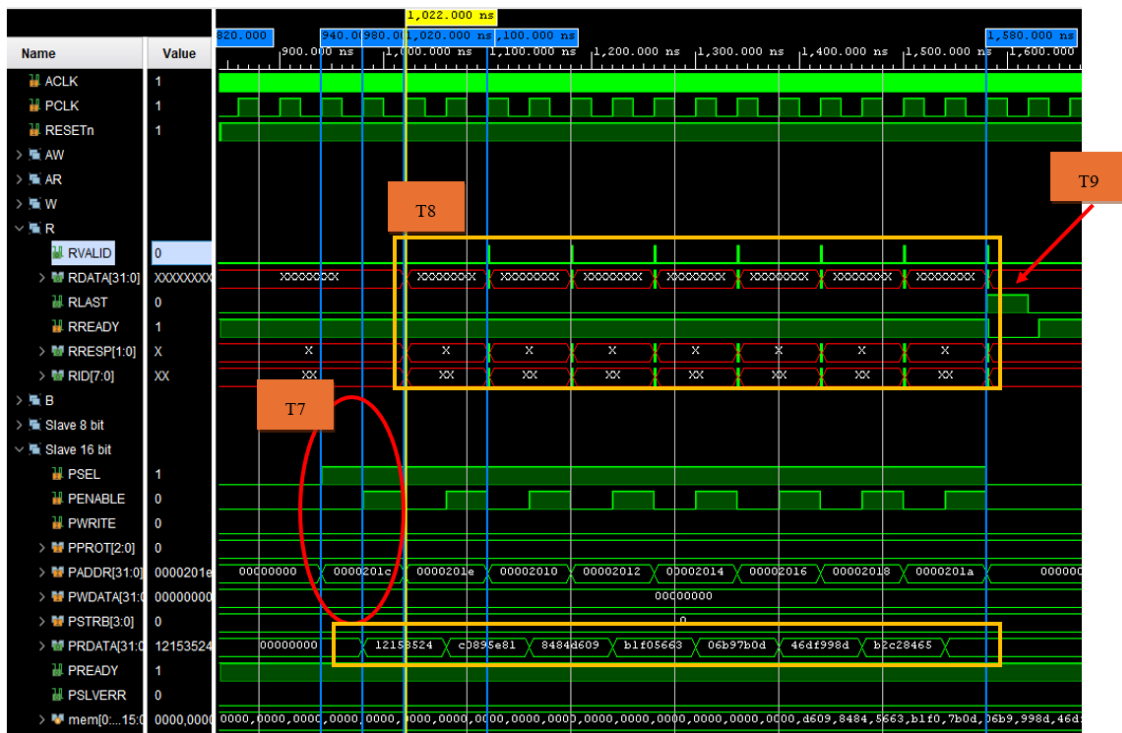
Hình 4.14 Dạng sóng quá trình ghi ở ngoại vi APB 16-bit

Như hình 4.14 đã mô tả, sau khi đã nhận được tín hiệu ghi dữ liệu vào APB slave, cầu nối AXI – APB sẽ xử lý tín hiệu từ phía AXI sang APB. Vì tần số phía ngoại vi APB là 25MHz so với tần số phía AXI là 250MHz, chênh lệch 10 lần nên việc ghi ở phía APB diễn ra dài hơn.

- (5) Tại thời điểm 140ns, với địa chỉ 0x00002xxx sẽ thuộc về slave 16-bit nên tín hiệu PSEL của APB-SLAVE 16-bit được nâng lên mức 1, đồng thời PWRITE = 1 để cho phép ghi vào SLAVE, địa chỉ PADDR bắt đầu sẽ được lấy từ FIFO của kênh AW, còn PWDATA sẽ được lấy từ FIFO kênh W, tín hiệu PSTRB = F để làm mặt nạ để ghi 32-bit dữ liệu. Tại 180ns, PENABLE được nâng lên mức 1, lúc này data từ kênh PWDATA sẽ được ghi vào bộ nhớ. Ở chu kỳ xung lock tiếp theo 220ns, PENABLE được kích xuống và PWDATA sẽ lấy dữ liệu thứ 2, còn PADDR sẽ tăng lên 2 do ô nhớ là 0x0000201E của APB-SLAVE 16-bit. Với độ dài ô nhớ của SLAVE là 16-bit, cần tới 2 ô nhớ để có thể chứa chuỗi dữ liệu 32-bit. Với gói dữ liệu thứ 3, địa chỉ là 0x00002010, dựa vào kiểu truyền WRAP cùng số lượng gói tin được quy định bởi AWLEN, ta có thể tính được rằng là 8 gói tin AXI cần có 0x10 địa chỉ có thể chứa hoàn toàn, cơ chế WRAP không cho địa chỉ đi quá phạm vi đó. Do địa chỉ đầu tiên phía AXI truyền vào là 0x0000201C nằm trong phạm vi từ 0x00002010 đến 0x00002020, vì thế PADDR = 0x0000201C sẽ tăng lên 0x0000202E, vì các ô nhớ 0x0000201C, 0x0000201D, đã chứa đủ một khung dữ liệu 32-bit, sau đó quay trở ngược lại 0x00002010. Tại 260ns, PENABLE = 1 để tiếp tục ghi tiếp gói tin thứ 2 tại địa chỉ 0x0000201E. Lần lượt là tại thời điểm 340ns, 420ns, 500ns, 580ns, 660ns, 740ns sẽ ghi gói tin thứ 3, thứ 4, thứ 5, thứ 6, thứ 7 và thứ 8 và PADDR tiếp tục tăng lên 2, 0x00002010 là gói tin thứ 3, 0x00002012 là gói tin thứ 4, 0x00002014 là gói tin thứ 5, 0x00002016 là gói tin thứ 6, 0x00002018 là gói tin thứ 7, 0x0000201A là gói tin thứ 8.
- (6) Tại thời điểm 780ns, thì tất cả các tín hiệu tại APB sẽ được tắt. Tại thời điểm 820ns, ở kênh B thì tín hiệu BVALID = 1 và tín hiệu BID = 0xC8

được truyền đi để phía AXI nhận được báo hiệu tăng đã thực hiện xong quá trình ghi dữ liệu với mã của giao dịch ghi là 0x8C.

Sau khi đã thực hiện quy trình ghi vào APB-SLAVE, bên phía AXI trước đó đã gửi một truy cập đọc, nên tiếp theo đó bên phía APB-SLAVE bắt đầu gửi những dữ liệu đọc dữ trên những thông tin điều khiển đã có trước đó ở kênh AR như mô tả của hình 4.15

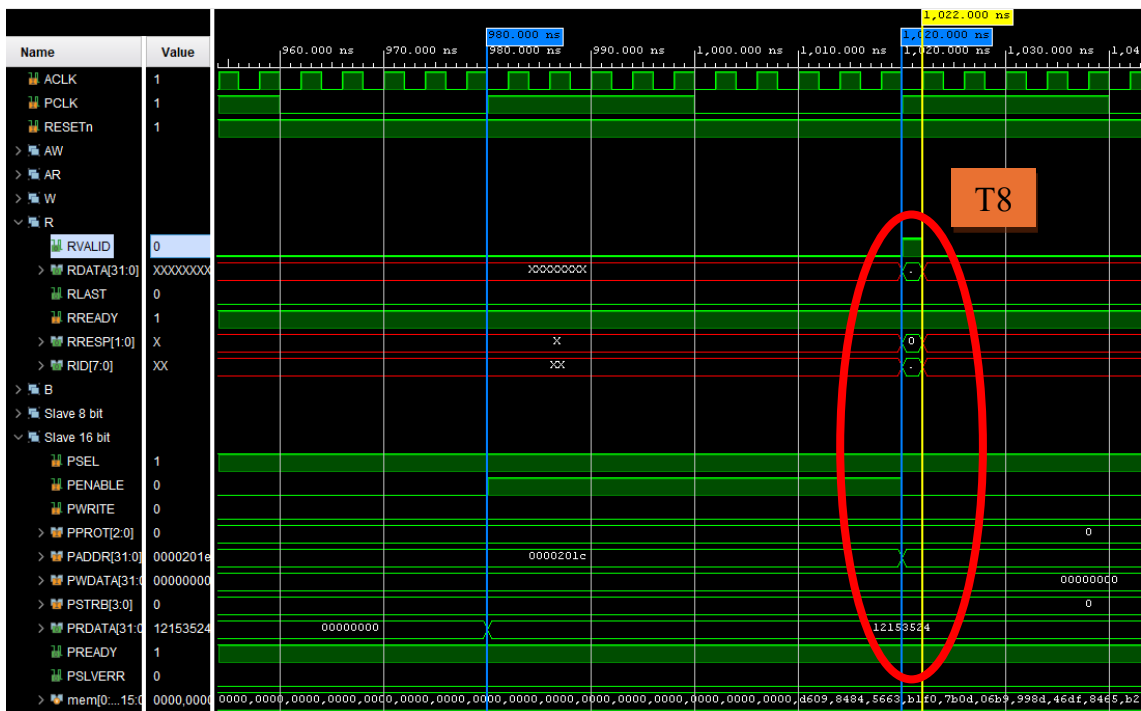


Hình 4.15 Dạng sóng quá trình đọc của ngoại vi APB 16-bit

- (7) Tại thời điểm 940ns tại T7, PSEL = 1 để việc chọn SLAVE 16-bit này để truy cập, PADDR = 0x0000201C có được từ thông tin địa chỉ từ kênh AR, PWRITE = 0 để biểu thị quá trình đọc từ APB-SLAVE ra. Tại thời điểm 980ns, PENABLE = 1, báo hiệu rằng SLAVE sẵn sàng truyền thông tin nên PRDATA sẽ có dữ liệu và PRDATA = 0x12153524, đọc từ ô địa chỉ bắt đầu 0x0000201C vì mỗi ô nhớ chỉ chứa được dữ liệu 16-bit.
- (8) Tại thời điểm 1020ns, ở bên phía tín hiệu PENABLE được hạ xuống và thông tin địa chỉ ở PADDR cũng thay đổi theo kiểu truyền WRAP đã được đề cập ở truy cập ghi của cầu nối để phục vụ việc đọc gói tin thứ 2, đồng thời tín hiệu xung clock tại thời gian này cũng truyền dữ liệu từ PRDATA sang kênh RDATA ở phía AXI, cùng lúc đó tín hiệu RVALID = 1 để báo

hiệu rằng có dữ liệu trong FIFO kênh R để có thể đọc ra, RID = 12 là thông tin từ tín hiệu ARID, BRESP = 0 báo hiệu rằng việc đọc không bị lỗi gì. Ở các chu kỳ xung clock PCLK tiếp theo, các địa chỉ mới được nạp vào kênh PADDR sao cho đúng với thông số của kênh AR và theo kiểu truyền WRAP của AXI. 8 gói tin lần lượt sẽ là, gói tin đầu tiên có địa chỉ 0x0000201C là 0x12153524, gói tin thứ 2 có địa chỉ 0x0000201E là 0xC0895E81, gói tin thứ 3 là 0x00002010 có dữ liệu 0x8484D609, gói tin thứ 4 có địa chỉ 0x00002012 là 0xB1F05663, thứ 5 là 0x00002014 có dữ liệu 0x06B97B0D, gói tin thứ 6 có gói tin 0x46DF998D tại ô địa chỉ 0x00002016, gói tin thứ 7 có địa chỉ 0x00002018 có dữ liệu 0xB2C28465, và gói tin thứ 8 tại địa chỉ 0x0000201A mang gói tin 0x89375212.

- (9) Sau khi thực hiện ghi 8 gói tin từ trong APB-SLAVE ra tại thời điểm 1580ns, RLAST được kích lên mức 1 để báo hiệu rằng kết thúc phiên đọc từ APB_SLAVE và tồn tại trong 1 chu kỳ xung clock PCLK.



Hình 4.16 Dạng sóng đọc được phóng to ở phía AXI tại thời điểm T8

Hình 4.16 là hình một gói tin ở tín hiệu RDATA đã được phóng to tại ô T8, với dữ liệu RDATA, PCLK kích lên làm dữ liệu được truyền từ tín hiệu PRDATA

đến RDATA. Tuy nhiên chỉ cần gấp cạnh lên của xung clock ACLK từ phía AXI thì dữ liệu sẽ được lấy đi, vì thế RDATA tồn tại trong khoảng thời gian rất ngắn.

Bảng kết quả quá trình ghi của cầu nối được thể hiện trong hình 4.17:

COMPARE TABLE (WRITE)					
AWADDR : 0000201c					
BURST TYPE : WRAP					
BURST LENGTH : 8					
BURST SIZE : 4 byte					
Master: AXI -> Slave: APB					
PACKET	WDATA (AXI)	PWDATA (APB)	PADDR (APB)	STATUS	
[0]	00f3e301	00f3e301	0000201c	PASS	
[1]	06d7cd0d	06d7cd0d	0000201e	PASS	
[2]	3b23f176	3b23f176	00002010	PASS	
[3]	1e8dcd3d	1e8dcd3d	00002012	PASS	
[4]	76d457ed	76d457ed	00002014	PASS	
[5]	462df78c	462df78c	00002016	PASS	
[6]	7cfde9f9	7cfde9f9	00002018	PASS	
[7]	e33724c6	e33724c6	0000201a	PASS	

Hình 4.17 Bảng kết quả quá trình ghi từ mô hình UVM

Hình 4.17 là bảng kết quả cho chu kỳ ghi từ phía AXI đến APB với mô hình kiểm tra UVM. Với các thông tin được liệt kê ở hình trên là AWADDR, BURST TYPE (kiểu truyền tương ứng với tín hiệu AWBURST), BURST LENGTH (số lượng gói tin trong một khung truyền, tương ứng với tín hiệu AWLEN), BURST SIZE (kích thước dữ liệu của gói tin tương ứng với tín hiệu AWSIZE). Đồng thời cũng hiển thị phía AXI hiện tại đang là nguồn gửi thông tin, phía APB là đích đến ở giao dịch ghi. Ở bảng, ta thấy cột đầu tiên là số thứ tự gói tin, cột thứ hai là dữ liệu từ kênh WDATA, cột thứ ba là dữ liệu từ kênh PWDATA sau khi đã qua cầu nối giao tiếp AXI – APB, cột tiếp theo biểu thị địa chỉ PADDR ở phía ngoại vi tương ứng với từng gói tin, cột cuối cùng cho thấy rằng là việc truyền có bị thất bại hay thành công. Việc truyền thành công hay thất bại dựa vào việc so sánh dữ liệu ở hai kênh PWDATA và WDATA, đồng thời xem xét địa chỉ PADDR có khớp để chứa hoàn toàn được chuỗi dữ liệu 32-bit hay không. Qua bảng trên ta có được cái nhìn tổng quan về chức năng cũng như kiểm tra chức năng của giao dịch ghi của cầu nối AXI – APB.

Bảng kết quả quá trình đọc của cầu nối được thể hiện trong hình 4.18:

COMPARE TABLE (READ)					
ARADDR : 0000201c					
BURST TYPE : WRAP					
BURST LENGTH : 8					
BURST SIZE : 4 byte					
Master: APB -> Slave: AXI					
PACKET	RDATA (AXI)	PRDATA (APB)	PADDR (APB)	STATUS	
[0]	00f3e301	00f3e301	0000201c	PASS	
[1]	06d7cd0d	06d7cd0d	0000201e	PASS	
[2]	3b23f176	3b23f176	00002010	PASS	
[3]	1e8dcd3d	1e8dcd3d	00002012	PASS	
[4]	76d457ed	76d457ed	00002014	PASS	
[5]	462df78c	462df78c	00002016	PASS	
[6]	7cfde9f9	7cfde9f9	00002018	PASS	
[7]	e33724c6	e33724c6	0000201a	PASS	

Hình 4.18 Bảng kết quả quá trình đọc từ mô hình UVM

Hình 4.18 là bảng kết quả cho chu kỳ đọc từ phía AXI đến APB với mô hình kiểm tra UVM. Với các thông tin được ở hình trên liệt kê là ARADDR, BURST TYPE (kiểu truyền, tương ứng với tín hiệu ARBURST), BURST LENGTH (số lượng gói tin trong một khung truyền, tương ứng với tín hiệu ARLEN), BURST SIZE (kích thước dữ liệu của gói tin, tương ứng với tín hiệu ARSIZE). Ở bảng, ta thấy cột đầu tiên là số thứ tự gói tin, cột thứ hai là dữ liệu từ kênh RDATA sau khi đã qua cầu nối giao tiếp AXI – APB, cột thứ ba là dữ liệu từ kênh PRDATA, cột tiếp theo biểu thị địa chỉ PADDR ở phía ngoại vi tương ứng với từng gói tin, cột cuối cùng cho thấy rằng là việc truyền có bị thất bại hay thành công. Việc truyền thành công hay thất bại dựa vào việc so sánh dữ liệu ở hai kênh PRDATA và RDATA, đồng thời xem xét địa chỉ PADDR có khớp để chứa hoàn toàn được chuỗi dữ liệu 32-bit hay không. Qua bảng trên ta có được cái nhìn tổng quan về chức năng cũng như kiểm tra chức năng của giao dịch đọc của cầu nối AXI – APB.

Để đánh giá hệ thống và kiểm thử chức năng của hệ thống trong trường hợp testcase3, ta cần đánh giá những hạng mục sau:

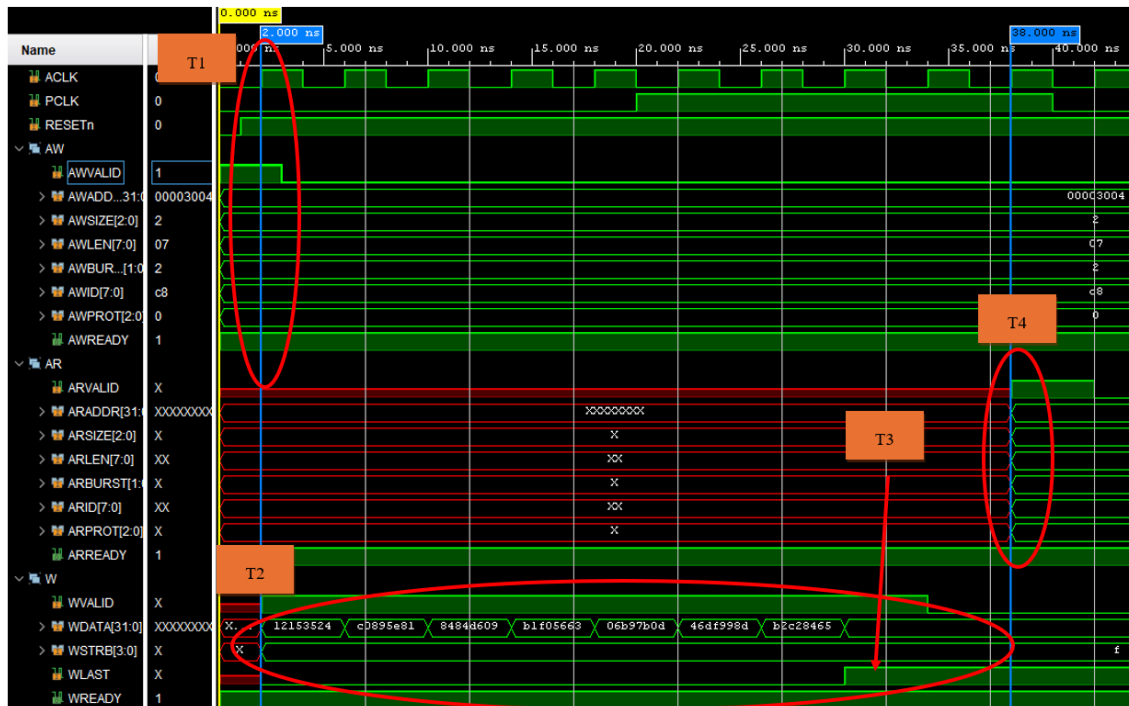
- Cơ chế bắt tay 2 chiều ở các kênh: Dựa vào lý thuyết ở chương 2 phân giao thức truyền thông AXI, nhóm nghiên cứu đã sử dụng đúng cơ chế bắt tay

VALID và READY, chỉ khi có cả 2 tín hiệu VALID và READY thì mới tiến hành truyền thông tin vào.

- Kiểm tra thông tin điều khiển, xem xét dữ liệu ngõ ra có trùng khớp với dữ liệu ngõ vào. Qua dạng sóng mô phỏng, cùng với những phân tích ở phía trên ta thấy được. Việc tín hiệu bộ nhớ của ngoại vi bị thay đổi chứng tỏ được rằng đã ghi vô được ngoại vi. Đồng thời kiểu truyền là WRAP, dựa vào dạng sóng và bảng kết quả từ UVM, nhóm nghiên cứu rút ra được việc địa chỉ có tăng lên đúng không như đã đề cập ở phần thiết kế hệ thống xử lý tính toán địa chỉ cơ chế INCR và WRAP. Địa chỉ ban đầu đã tăng từ 0x0000201C đến 0x00002018. Đây là ngoại vi APB có độ dài ô nhớ 16-bit nên khoảng địa chỉ sẽ chứa dữ liệu, tương đương với khoảng độ dài khung truyền là 0x10, lấy giá trị đó thêm vào địa chỉ ban đầu sẽ vượt quá khung nên địa chỉ sẽ quay lại 0x00002010, địa chỉ ban đầu là 0x1C sẽ là 0x0001_1100, chỉ thay đổi 4 bit cuối của địa chỉ như đã đề cập ở cơ chế WRAP chương 3 của ngoại vi 16-bit. Địa chỉ tiếp theo tiếp tục tăng như thông thường, địa chỉ của gói tin cuối cùng sẽ là 0x00002018, 8 gói tin này sẽ ở địa chỉ từ 0x00002010 đến 0x0000201C, vậy việc tính toán địa chỉ trong kiểu truyền WRAP đã đúng.
- Dữ liệu đọc ghi: Dựa vào tín hiệu, ta đã ghi vào 8 gói tin có dữ liệu như sau: 0x1215324, 0xC0895E81, 0x8484D609, 0xB1F05663, 0x06B97B0D, 0x46DF998D, 0xB2C28465, 0x89375212. Và khi đọc dữ liệu ra từ ngoại vi thì dữ liệu là: 0x1215324, 0xC0895E81, 0x8484D609, 0xB1F05663, 0x06B97B0D, 0x46DF998D, 0xB2C28465, 0x89375212. Do đó, việc ghi vào ngoại vi và đọc ra đã thành công.

4.2.4 Testcase4: AXI – APB ghi đọc dữ liệu 32-bit vào slave 32-bit chế độ WRAP với BURST_SIZE = 8, BURST_LENGTH = 32

Kết quả truyền dữ liệu ở Testcase4 - AXI – APB ở quá trình ghi và đọc với dữ liệu 32-bit và slave 32-bit chế độ WRAP với thông số BURST_SIZE = 8, BURST_LENGTH = 32 được thể hiện ở hình 4.19:



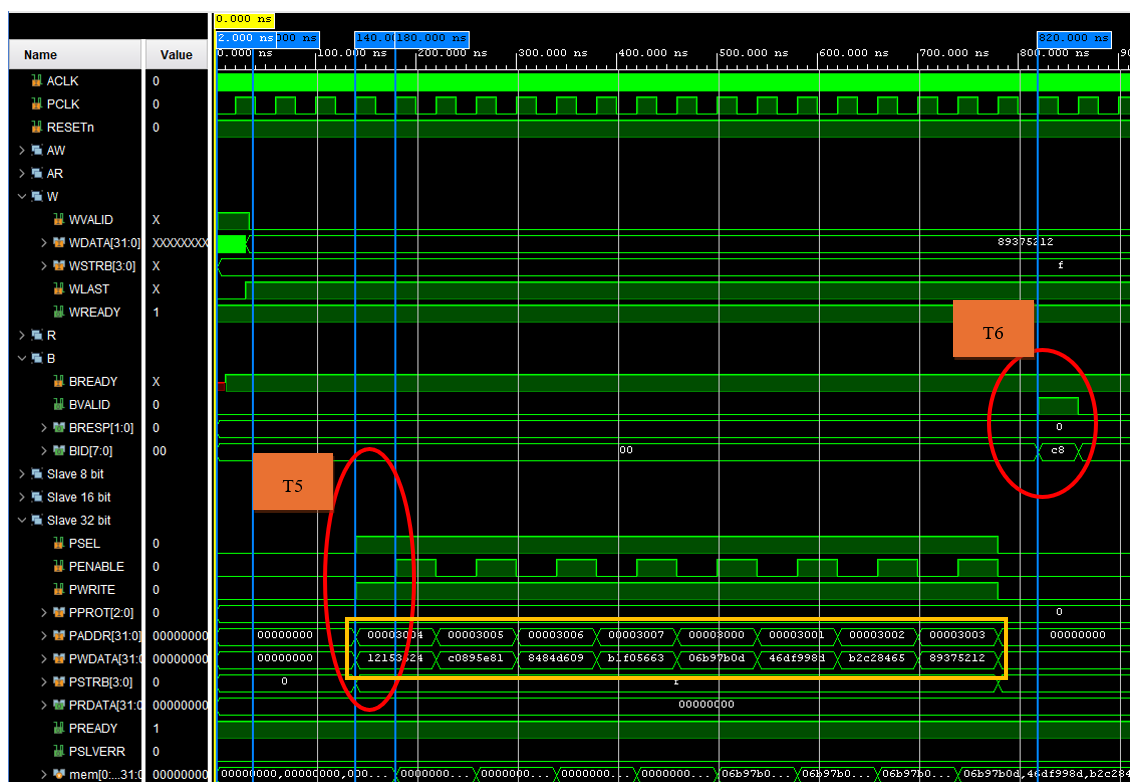
Hình 4.19 Dạng sóng tín hiệu chuẩn bị quá trình ghi và đọc

Quá trình thiết lập tín hiệu được thể hiện ở hình 4.19:

- (1) Tại thời điểm T1, 0ns đến 1ns, RESETn = 0 để tiến hành khởi động lại hệ thống, sau đó RESETn = 1 để hệ thống bắt đầu hoạt động, AWREADY, ARREADY, WREADY luôn giữ ở mức cao để cho phép truyền, nhận tín hiệu. Ở thời điểm 2 ns, các tín hiệu thiết lập cho cầu nối thực hiện quá trình ghi được bắt đầu. Với việc có cạnh lên cùng xung clock cùng những tín hiệu điều khiển ở kênh AW, lúc này địa chỉ bắt đầu AWADDR bằng 0x0003004, địa chỉ của ngoại vi APB 32-bit, kích thước của gói tin AWSIZE = 2 tức là kích thước gói tin là 32-bit (4-byte), AWLEN = 7 tức là sẽ có 8 gói tin trong một lần truyền, AWBURST = 2 tức là chọn kiểu truyền địa chỉ là WRAP theo chuẩn AMBA, AWID là 0xC8 để mã giao dịch.

- (2) Tại khung truyền T2, tại thời điểm cạnh lên xung clock đó thì cầu nối cũng sẽ nhận 8 gói tin 32-bit ở kênh W, trùng khớp với thông tin từ kênh AW. Tín hiệu WSTRB = 0x0F làm mặt nạ 32-bit dữ liệu, tức là sẽ sử dụng hết 32-bit của WDATA. Tại 2ns là gói tin đầu tiên, ở 6ns là gói tin thứ 2 và tiếp tục ở mỗi chu kỳ cho đến gói tin thứ 8.
- (3) Khi gửi gói tin thứ 8 ở vị trí 30ns ở khung truyền T3, tín hiệu WLAST sẽ được kích hoạt lên mức 1 cùng lúc báo hiệu rằng đây là gói tin cuối cùng của khung dữ liệu truyền đi. Ở 34ns, tín hiệu WVALID = 0 đánh dấu cho việc kết thúc việc truyền dữ liệu ghi ở phía AXI.
- (4) Tại thời điểm 38ns ở T4, ở phía kết nối AXI sẽ phát tín hiệu rằng phía MASTER muốn đọc dữ liệu với các thông số ở kênh AR như sau: địa chỉ bắt đầu ARADDR bằng 0x00003004, kích thước của gói tin là 32-bit được quy định bởi ARSIZE = 2, ARLEN = 7 là sẽ có 8 gói tin, ARBURST = 2 chọn kiểu truyền địa chỉ là WRAP, ARID = 12.

Quá trình ghi ở ngoại vi 32 bit được thể hiện ở hình 4.20:

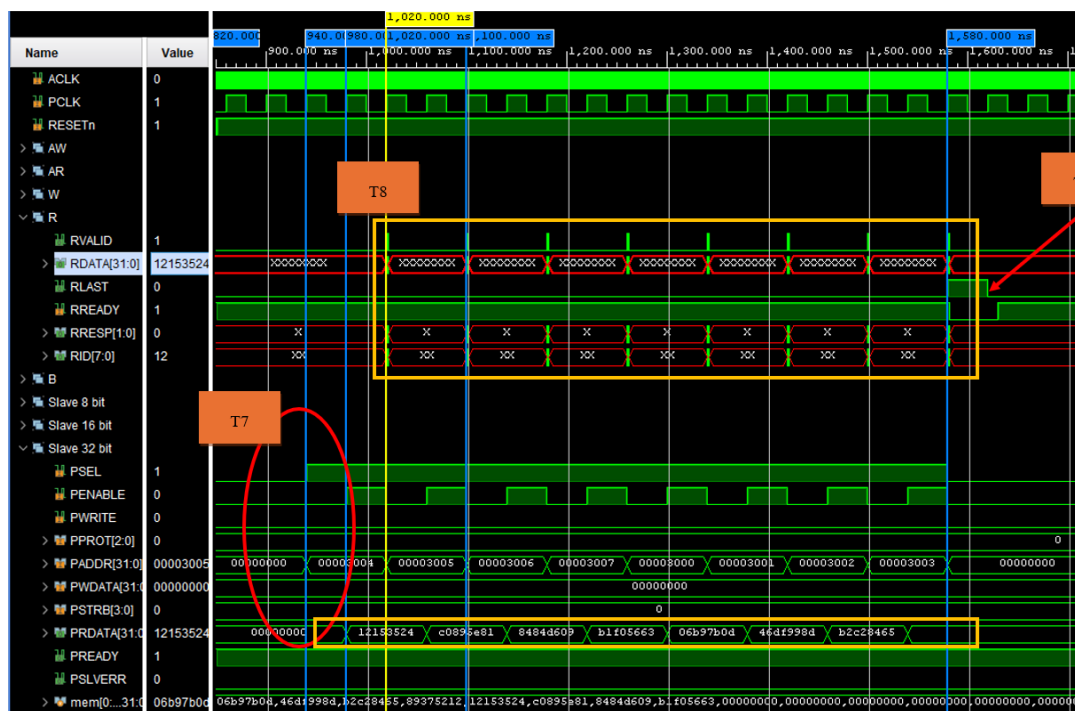


Hình 4.20 Dạng sóng quá trình ghi ở ngoại vi APB 32-bit

Như hình 4.20 đã mô tả, sau khi đã nhận được tín hiệu ghi dữ liệu vào APB slave, cầu nối AXI – APB sẽ xử lý tín hiệu từ phía AXI sang APB. Vì tần số phía ngoại vi APB là 25MHz so với tần số phía AXI là 250MHz, chênh lệch 10 lần nên việc ghi ở phía APB diễn ra dài hơn.

- (5) Tại thời điểm 140ns, với địa chỉ 0x00003xxx sẽ thuộc về slave 32-bit nên tín hiệu PSEL của APB-SLAVE 16-bit được nâng lên mức 1, đồng thời PWRITE = 1 để cho phép ghi vào SLAVE, địa chỉ PADDR bắt đầu sẽ được lấy từ FIFO của kênh AW, còn PWDATA sẽ được lấy từ FIFO kênh W, tín hiệu PSTRB = F để làm mặt nạ để ghi 32-bit dữ liệu. Tại 180ns, PENABLE được nâng lên mức 1, lúc này data từ kênh PWDATA sẽ được ghi vào bộ nhớ. Ở chu kỳ xung lock tiếp theo 220ns, PENABLE được kích xuống và PWDATA sẽ lấy dữ liệu thứ 2, còn PADDR sẽ tăng lên 1 do ô nhớ là 0x00003005 của APB-SLAVE 32-bit. Với độ dài ô nhớ của SLAVE là 32-bit, chỉ cần 1 ô nhớ để đủ có thể chứa chuỗi dữ liệu 32-bit. Lần lượt là tại thời điểm 340ns, 420ns, 500ns, 580ns, 660ns, 740ns sẽ ghi gói tin thứ 3, thứ 4, thứ 5, thứ 6, thứ 7 và thứ 8 và PADDR tiếp tục tăng lên 1, 0x00003006 là gói tin thứ 3, 0x00003007 là gói tin thứ 4. Tuy nhiên sau gói tin thứ 4, địa chỉ sẽ quay về 0x00003000 vì kênh truyền sử dụng kiểu truyền WRAP cùng số lượng gói tin được quy định bởi AWLEN, ta có thể tính được rằng là 8 gói tin AXI cần có 0x08 địa chỉ có thể chứa hoàn toàn, cơ chế WRAP không cho địa chỉ đi quá phạm vi đó. Do địa chỉ đầu tiên phía AXI truyền vào là 0x00003004 nằm trong phạm vi từ 0x00003000 đến 0x00003008, vì thế PADDR = 0x00003004 đến 0x00003007 rồi sẽ quay lại địa chỉ 0x00003000 nên tại thời điểm 500ns, địa chỉ là 0x00003000 có gói tin thứ 5, 0x00003001 là gói tin thứ 6 tại 580ns, 0x00003002 là gói tin thứ 7 lúc 660ns, 0x00003003 là gói tin thứ 8 tại thời điểm 740ns.
- (6) Ở T6, tại thời điểm 780ns, thì tất cả các tín hiệu tại APB sẽ được tắt. Tại thời điểm 820ns, ở kênh B thì tín hiệu BVALID và tín hiệu BID = 0xC8 được truyền đi để phía AXI nhận được báo hiệu tăng đã thực hiện xong quá trình ghi dữ liệu với mã của giao dịch ghi là 0x8C.

Sau khi đã thực hiện quy trình ghi vào APB-SLAVE, bên phía AXI trước đó đã gửi một truy cập đọc, nên tiếp theo đó bên phía APB-SLAVE bắt đầu gửi những dữ liệu đọc dữ trên những thông tin điều khiển đã có trước đó ở kênh AR như hình 4.21 đã mô tả

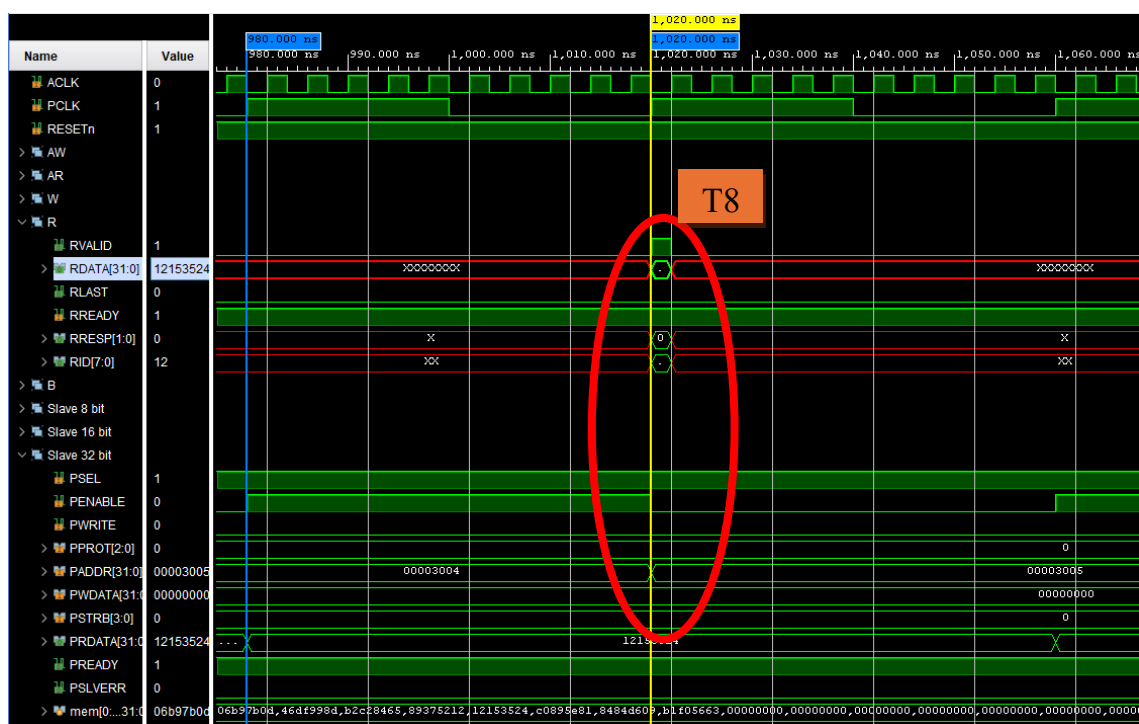


Hình 4.21 Dạng sóng quá trình đọc ở ngoại vi APB 32-bit

- (7) Tại thời điểm 940ns, PSEL = 1 để việc chọn SLAVE 16-bit này để truy cập, PADDR = 0x00003004 có được từ thông tin địa chỉ từ kênh AR, PWRITE = 0 để biểu thị quá trình đọc từ APB-SLAVE ra. Tại thời điểm 980ns, PENABLE = 1, báo hiệu rằng SLAVE sẵn sàng truyền thông tin nên PRDATA sẽ có dữ liệu và PRDATA = 0x12153524, đọc từ ô địa chỉ bắt đầu 0x00003004, địa chỉ từ ngoại vi APB 32-bit.
- (8) Tại thời điểm 1020ns ở ô T8, ở bên phía tín hiệu PENABLE được hạ xuống và thông tin địa chỉ ở PADDR cũng thay đổi theo kiểu truyền WRAP đã được đề cập ở truy cập ghi của cầu nối để phục vụ việc đọc gói tin thứ 2, đồng thời tín hiệu xung clock tại thời gian này cũng truyền dữ liệu từ PRDATA sang kênh RDATA ở phía AXI, cùng lúc đó tín hiệu RVALID = 1 để báo hiệu rằng có dữ liệu trong FIFO kênh R để có thể đọc ra, RID = 12 là thông tin từ tín hiệu ARID, BRESP = 0 báo hiệu rằng việc đọc không

bị lỗi gì. Ở các chu kỳ xung clock PCLK tiếp theo, các địa chỉ mới được nạp vào kênh PADDR sao cho đúng với thông số của kênh AR và theo kiểu truyền WRAP của AXI. 8 gói tin ở kênh R sẽ là: gói tin đầu tiên có địa chỉ 0x00003004 là 0x12153524, gói tin thứ 2 có địa chỉ 0x00003005 là 0xC0895E81, gói tin thứ 3 là 0x00003006 có dữ liệu 0x8484D609, gói tin thứ 4 có địa chỉ 0x00003007 là 0xB1F05663, thứ 5 là 0x00003000 có dữ liệu 0x06B97B0D, gói tin thứ 6 có gói tin 0x46DF998D tại ô địa chỉ 0x00003001, gói tin thứ 7 có địa chỉ 0x00003002 có dữ liệu 0xB2C28465, và gói tin thứ 8 tại địa chỉ 0x00003003 mang gói tin 0x89375212.

- (9) Sau khi thực hiện ghi gói tin cuối cùng từ trong APB-SLAVE ra tại thời điểm 1580ns, RLAST được kích lên mức 1 để báo hiệu rằng kết thúc phiên đọc từ APB_SLAVE và tồn tại trong 1 chu kỳ xung clock PCLK.



Hình 4.22 Dạng sóng dữ liệu đọc phóng to ở phía AXI tại thời điểm T8

Hình 4.22 là hình một gói tin ở tín hiệu RDATA đã được phóng to tại ô T8. Với dữ liệu RDATA, PCLK kích lên làm dữ liệu được truyền từ tín hiệu PRDATA đến RDATA. Tuy nhiên chỉ cần gấp cạnh lên của xung clock ACLK từ phía AXI thì dữ liệu sẽ được lấy đi, vì thế RDATA tồn tại trong khoảng thời gian rất ngắn.

Bảng kết quả quá trình ghi của cầu nối được thể hiện trong hình 4.23:

COMPARE TABLE (WRITE)						
AWADDR : 00003004						
BURST TYPE : WRAP						
BURST LENGTH : 8						
BURST SIZE : 4 byte						
Master: AXI -> Slave: APB						
PACKET	WDATA (AXI)	PWDATA (APB)	PADDR (APB)	STATUS		
[0]	00f3e301	00f3e301	00003004	PASS		
[1]	06d7cd0d	06d7cd0d	00003005	PASS		
[2]	3b23f176	3b23f176	00003006	PASS		
[3]	1e8dcd3d	1e8dcd3d	00003007	PASS		
[4]	76d457ed	76d457ed	00003000	PASS		
[5]	462df78c	462df78c	00003001	PASS		
[6]	7cfde9f9	7cfde9f9	00003002	PASS		
[7]	e33724c6	e33724c6	00003003	PASS		

Hình 4.23 Bảng kết quả quá trình ghi từ mô hình UVM

Hình 4.23 là bảng kết quả cho chu kỳ ghi từ phía AXI đến APB với mô hình kiểm tra UVM. Với các thông tin được liệt kê ở hình trên là AWADDR, BURST TYPE (kiểu truyền tương ứng với tín hiệu AWBURST), BURST LENGTH (số lượng gói tin trong một khung truyền, tương ứng với tín hiệu AWLEN), BURST SIZE (kích thước dữ liệu của gói tin, tương ứng với tín hiệu AWSIZE). Ở bảng, ta thấy cột đầu tiên là số thứ tự gói tin, cột thứ hai là dữ liệu từ kênh WDATA, cột thứ ba là dữ liệu từ kênh PWDATA sau khi đã qua cầu nối giao tiếp AXI – APB, cột tiếp theo biểu thị địa chỉ PADDR ở phía ngoại vi tương ứng với từng gói tin, cột cuối cùng cho thấy rằng là việc truyền có bị thất bại hay thành công. Việc truyền thành công hay thất bại dựa vào việc so sánh dữ liệu ở hai kênh PWDATA và WDATA, đồng thời xem xét địa chỉ PADDR có khớp để chứa hoàn toàn được chuỗi dữ liệu 32-bit hay không. Qua bảng trên ta có được cái nhìn tổng quan về chức năng cũng như kiểm tra chức năng của giao dịch ghi của cầu nối AXI – APB.

Bảng kết quả quá trình ghi của cầu nối được thể hiện trong hình 4.24:

COMPARE TABLE (READ)					
ARADDR : 00003004					
BURST TYPE : WRAP					
BURST LENGTH : 8					
BURST SIZE : 4 byte					
Master: APB -> Slave: AXI					
PACKET	RDATA (AXI)	PRDATA (APB)	PADDR (APB)	STATUS	
[0]	00f3e301	00f3e301	00003004	PASS	
[1]	06d7cd0d	06d7cd0d	00003005	PASS	
[2]	3b23f176	3b23f176	00003006	PASS	
[3]	1e8dcd3d	1e8dcd3d	00003007	PASS	
[4]	76d457ed	76d457ed	00003000	PASS	
[5]	462df78c	462df78c	00003001	PASS	
[6]	7cfde9f9	7cfde9f9	00003002	PASS	
[7]	e33724c6	e33724c6	00003003	PASS	

Hình 4.24 Bảng kết quả quá trình đọc từ mô hình UVM

Hình 4.24 là bảng kết quả cho chu kỳ đọc từ phía AXI đến APB với mô hình kiểm tra UVM. Với các thông tin được ở hình trên liệt kê là ARADDR, BURST TYPE (kiểu truyền, tương ứng với tín hiệu ARBURST), BURST LENGTH (số lượng gói tin trong một khung truyền, tương ứng với tín hiệu ARLEN), BURST SIZE (kích thước dữ liệu của gói tin, tương ứng với tín hiệu ARSIZE). Ở bảng, ta thấy cột đầu tiên là số thứ tự gói tin, cột thứ hai là dữ liệu từ kênh RDATA sau khi đã qua cầu nối giao tiếp AXI – APB, cột thứ ba là dữ liệu từ kênh PRDATA, cột tiếp theo biểu thị địa chỉ PADDR ở phía ngoại vi tương ứng với từng gói tin, cột cuối cùng cho thấy rằng là việc truyền có bị thất bại hay thành công. Việc truyền thành công hay thất bại dựa vào việc so sánh dữ liệu ở hai kênh PRDATA và RDATA, đồng thời xem xét địa chỉ PADDR có khớp để chứa hoàn toàn được chuỗi dữ liệu 32-bit hay không. Qua bảng trên ta có được cái nhìn tổng quan về chức năng cũng như kiểm tra chức năng của giao dịch đọc của cầu nối AXI – APB.

Để đánh giá hệ thống và kiểm thử chức năng của hệ thống trong trường hợp testcase4, ta cần đánh giá những hạng mục sau:

- Cơ chế bắt tay 2 chiều ở các kênh: Dựa vào lý thuyết ở chương 2 phân giao thức truyền thông AXI, nhóm nghiên cứu đã sử dụng đúng cơ chế bắt tay

VALID và READY, chỉ khi có cả 2 tín hiệu VALID và READY thì mới tiến hành truyền thông tin vào.

- Kiểm tra thông tin điều khiển, xem xét dữ liệu ngõ ra có trùng khớp với dữ liệu ngõ vào. Qua dạng sóng mô phỏng, cùng với những phân tích ở phía trên ta thấy được. Việc tín hiệu bộ nhớ của ngoại vi bị thay đổi chứng tỏ được rằng đã ghi vô được ngoại vi. Đồng thời kiểu truyền là WRAP, dựa vào dạng sóng và bảng kết quả từ UVM, nhóm nghiên cứu rút ra được việc địa chỉ có tăng lên đúng không như đã đề cập ở phần thiết kế hệ thống xử lý tính toán địa chỉ cơ chế INCR và WRAP. Địa chỉ ban đầu đã tăng từ 0x00003004 đến 0x00003008. Đây là ngoại vi APB có độ dài ô nhớ 32-bit nên khoảng địa chỉ sẽ chứa dữ liệu, tương đương với khoảng độ dài khung truyền là 0x10, lấy giá trị đó thêm vào địa chỉ ban đầu sẽ vượt quá khung nên địa chỉ sẽ quay lại 0x00003000, địa chỉ ban đầu là 0x04 sẽ là 0x0000_0100, chỉ thay đổi 3 bit cuối của địa chỉ như đã đề cập ở cơ chế WRAP chương 3 của ngoại vi 32-bit. Địa chỉ tiếp theo tiếp tục tăng như thông thường, địa chỉ của gói tin cuối cùng sẽ là 0x00003008, 8 gói tin này sẽ ở địa chỉ từ 0x00003000 đến 0x00003008, vậy việc tính toán địa chỉ trong kiểu truyền WRAP đã đúng.
- Dữ liệu đọc ghi: Dựa vào tín hiệu, ta đã ghi vào 8 gói tin có dữ liệu như sau: 0x1215324, 0xC0895E81, 0x8484D609, 0xB1F05663, 0x06B97B0D, 0x46DF998D, 0xB2C28465, 0x89375212. Và khi đọc dữ liệu ra từ ngoại vi thì dữ liệu là: 0x1215324, 0xC0895E81, 0x8484D609, 0xB1F05663, 0x06B97B0D, 0x46DF998D, 0xB2C28465, 0x89375212. Do đó, việc ghi vào ngoại vi và đọc ra đã thành công.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 KẾT LUẬN

Sau khi thực hiện đề tài “Thiết Kế Cầu Nối AXI-APB Và Kiểm Tra Bằng Môi Trường UVM”, nhóm thực hiện đã đưa ra kết luận sau:

Ưu điểm:

- Thiết kế đã thực hiện được các tính năng truyền dữ liệu theo yêu cầu.
- Cầu nối AXI-APB dựa trên phiên bản AXI-4 và APB-4 nên có thể dùng trong các hệ thống AMBA 4.
- Thiết kế hoạt động được với tần số cao ở phía AXI và tần số thấp ở phía APB với các tỉ lệ khác nhau.
- Môi trường UVM thực hiện kiểm tra đầy đủ các tính năng của cầu nối

Nhược điểm:

- Tín hiệu AWPROT, ARPROT ở giao thức AXI và PPROT ở bus APB chưa được sử dụng.
- Cầu nối chưa được kiểm thử trên một thiết bị FPGA, chưa đưa ra được đánh giá về timing, delay.

5.2 HƯỚNG PHÁT TRIỂN

Với những hạn chế còn tồn đọng ở trên, nhóm xin đề xuất các hướng phát triển để đề tài được hoàn thiện hơn:

- Kiểm tra các tính năng trên FPGA đồng thời đánh giá đầy đủ về vấn đề timing và delay.
- Áp dụng tín hiệu AWPROT, ARPROT ở giao thức AXI và PPROT ở bus APB trong quá trình truyền.
- Xây dựng mô hình hỗ trợ cho nhiều master và nhiều slave

PHỤ LỤC

Folder source code: [RedNubber/AXI-APB-Bridge-and-UVM-Verification: Source code for AXI-APB Bridge and UVM Verification](#)

TÀI LIỆU THAM KHẢO

- [1] V. S. R. V. Anil Kumar, "Design and Implementation of Axi-Apb Bridge based on Amba 4.0," *International Journal for Scientific Research & Development*, vol. Vol. 2, no. Issue 07, pp. 47-50, 2014.
- [2] A. Limited, AMBA® APB Protocol Specification, Cambridge, England: ARM Limited, 2021.
- [3] A. Limited, AMBA® AXI Protocol Specification, Cambridge, England: ARM Limited, 2023.
- [4] B. Q. Minh, "Giới thiệu và So sánh các Giao thức APB, AHB, và AXI," ICTC, 06 2024. [Online]. Available: <https://ictc.edu.vn/gioi-thieu-va-so-sanh-cac-giao-thuc-apb-ahb-va-axi/>. [Accessed 18 02 2025].
- [5] R. A. K. A. M. A. W. T. Y. I. H. M. K. S. Kholoud Mahmoud, "Towards a Generic UVM," *2022 IEEE High Performance Extreme Computing Conference (HPEC)*, vol. Vol.01, p. 6, 2022.
- [6] chipverify, "chipverify," chipverify, [Online]. Available: <https://www.chipverify.com/tutorials/uvm>. [Accessed 03 05 2025].
- [7] H. v. d. S. M. P. T. F. J. S. Michael Horn, *Universal Verification Methodology UVM Cookbook*, USA: Siemens Verification Academy, 2018.
- [8] K. Santhi Priya Sarekokku, "Design And Implementation Of APB Bridge Based On AMBA AXI 4.0," *International Journal of Engineering Research & Technology (IJERT)*, vol. Vol.1, no. Issue 9, pp. 1-4, 2012.

- [9] X. Z. J. S. Lei Liu, "Design of AXI-APB bus bridge verification platform," *2023 5th International Academic Exchange Conference on Science and Technology Innovation (IAECST)*, pp. 138-142, 2012.